COURS COMPLET · TOME 1

PYTHON

De l’algorithmique
à la programmation orientée objet

Un parcours progressif pensé pour les élèves et les professionnels qui veulent acquérir de l’expérience en Python : raisonnement algorithmique, premiers programmes, structures de données, compréhensions, closures, décorateurs, classes, encapsulation, héritage et méthodes spéciales, enfin exceptions et débogage. Chaque partie se conclut par des exercices entièrement corrigés, pas à pas.

AUTEURS

HOUNKPE Mawulikplimi Roland
DJABON Ounimborbitibou

ALGORITHMIQUE
PYTHON 3
POO
DEBUGGING
EXERCICES CORRIGÉS

ÉDITION 2026 · TOME 1 SUR 2

Table des matières

Préface . . . . **iii**

Mode d'emploi du cours . . . . **iii**

Chapitre 1 : Découvrir l'algorithmique 1

Chapitre 2 : Les structures conditionnelles 4

Chapitre 3 : Les boucles et répétitions 7

Chapitre 4 : Tableaux et fonctions algorithmiques 9

Exercices corrigés de la Partie I 11

Chapitre 5 : Découvrir Python et son environnement 17

Chapitre 6 : Variables et types de données 19

Chapitre 7 : Les conditions en Python 21

Chapitre 8 : Les boucles en Python 24

Exercices corrigés de la Partie II 26

Chapitre 9 : Les listes et les tuples 30

Chapitre 10 : La compréhension de listes 32

Chapitre 11 : Dictionnaires et ensembles 35

Chapitre 12 : Les fonctions en Python 37

Chapitre 13 : Portée approfondie, closures et décorateurs 40

Chapitre 14 : Chaînes de caractères et fichiers 43

Exercices corrigés de la Partie III 45

Chapitre 15 : Classes et objets : la première rencontre 51

Chapitre 16 : Encapsulation et propriétés 54

Chapitre 17 : Héritage et polymorphisme 57

Chapitre 18 : Les méthodes spéciales (dunder) 60

Chapitre 19 : Mini-projet final : un système bancaire à trois classes 63

Exercices corrigés de la Partie IV 65

Chapitre 20 : Les erreurs et les exceptions 73

Chapitre 21 : Déboguer et fiabiliser son code 79

Exercices corrigés de la Partie V 84

Conclusion du Tome 1 . . . . **91**

Annexe : Aide-mémoire Python 92

Préface

Apprendre à programmer, c'est d'abord apprendre à penser de manière structurée : décomposer un problème, ordonner les étapes de sa résolution, puis exprimer cette démarche dans un langage que la machine peut exécuter. Trop souvent, les débutants se jettent directement dans la syntaxe d'un langage et décrochent dès que la logique se complique. Ce cours suit le chemin inverse : nous commençons par l'algorithmique, c'est-à-dire l'art de construire des solutions rigoureuses, puis nous transcrivons progressivement ces solutions en Python, l'un des langages les plus appréciés et les plus utilisés au monde.

Ce cours est conçu avec un double objectif : **enseigner les élèves** (qu'ils viennent du secondaire ou de l'université) et accompagner **les professionnels qui veulent acquérir de l'expérience en Python**, que ce soit pour évoluer dans leur métier, automatiser des tâches quotidiennes ou se reconvertir vers les métiers du numérique. Aucune connaissance préalable en programmation n'est exigée : chaque notion est introduite progressivement, illustrée par des exemples concrets tirés de situations réelles, puis consolidée par des exercices entièrement corrigés.

Le présent ouvrage constitue le **Tome 1** de la collection. Il couvre l'intégralité du chemin qui mène du raisonnement algorithmique à la programmation orientée objet : les structures conditionnelles et itératives, les variables et les types de données, les listes et leurs compréhensions, les dictionnaires, les fonctions avec leurs closures et leurs décorateurs, la manipulation des chaînes et des fichiers, la maîtrise des exceptions et de l'art du débogage, jusqu'à la conception de classes, l'encapsulation, l'héritage et le polymorphisme. Chaque partie se termine par une série d'exercices corrigés pas à pas, pensés comme un entraînement progressif : des exercices basiques pour ancrer les réflexes, des exercices intermédiaires pour combiner les notions, et un défi pour dépasser le cours.

Le Tome 2, qui paraîtra dans la même collection, prolongera cette base vers les modules et les packages, la programmation orientée objet avancée, les tests unitaires, les interfaces graphiques et les bases de données. Maîtriser le Tome 1 est le seul prérequis pour l'aborder sereinement.

Nous vous souhaitons une lecture aussi agréable que formatrice. Programmez, vous trompez, recommencez : c'est ainsi que l'on devient développeur.

HOUNKPE Mawulikplimi Roland et DJABON Ounimborbitibou

Mode d'emploi du cours

Ce manuel est organisé en cinq parties qui se lisent de préférence dans l'ordre. La **Partie I** construit votre raisonnement algorithmique sans écrire une seule ligne de Python : vous y apprendrez à décrire des solutions en pseudo-code. La **Partie II** installe Python et transpose chaque notion algorithmique en syntaxe réelle. La **Partie III** enrichit votre boîte à outils avec les structures de données et les fonctions. La **Partie IV** vous initie à la programmation orientée objet, aboutissement naturel du parcours. Enfin, la **Partie V** vous apprend à protéger vos programmes contre les erreurs, à maîtriser les exceptions et à déboguer avec méthode : les compétences qui séparent un script fragile d'un logiciel fiable.

Chaque chapitre alterne explications détaillées, exemples de code commentés ligne à ligne et encadrés pédagogiques. Les exemples de code apparaissent dans des blocs sombres avec coloration syntaxique ; le pseudo-code algorithmique apparaît dans des blocs étiquetés « ALGORITHME ». Nous vous conseillons vivement de taper vous-même chaque exemple dans un fichier Python et de l'exécuter : la programmation s'apprend avec les doigts autant qu'avec les yeux.

Encadré	Rôle
À RETENIR	Synthèse des notions essentielles du chapitre, à relire avant un devoir ou un entretien.
ATTENTION	Piège classique, erreur fréquente du débutant ou point de vigilance du langage.
ASTUCE	Conseil de praticien : raccourci, bonne habitude ou technique de professionnel.

Les trois encadrés qui rythment les chapitres

Les sections « Exercices corrigés » qui closent chaque partie comportent des exercices classés par difficulté croissante : **Basique**, **Intermédiaire** et **Défi**. Cherchez chaque solution seul pendant au moins dix minutes avant de lire la correction : c'est l'effort de recherche, et non la lecture passive, qui construit la compétence. Les corrections détaillent la stratégie, le code commenté et, lorsque c'est utile, une trace d'exécution pas à pas.

PARTIE I

L'algorithmique : apprendre à penser

Avant d'écrire du code, il faut savoir construire une solution. Cette première partie vous entraîne à décomposer un problème, à décrire ses étapes en pseudo-code, à prendre des décisions avec des conditions, à répéter des tâches avec des boucles et à organiser vos données dans des tableaux. Ces notions sont universelles : elles vous serviront quel que soit le langage que vous pratiquerez plus tard.

CHAPITRE 1

Découvrir l'algorithmique

Le mot « algorithme » vient du nom du mathématicien persan Al-Khwarizmi, qui vécut au IXe siècle. Aujourd'hui, un algorithme désigne une suite finie d'instructions non ambiguës qui, à partir de données d'entrée, produit un résultat. Un algorithme n'est pas un programme : c'est une **recette de résolution**, indépendante de tout langage et de toute machine. C'est cette recette que nous apprendrons d'abord à écrire.

1.1 Qu'est-ce qu'un algorithme ?

Vous manipulez des algorithmes tous les jours sans le savoir. Une recette de cuisine est un algorithme : on part d'ingrédients (les données d'entrée), on applique une suite ordonnée d'opérations (mélanger, chauffer, assaisonner) et on obtient un plat (le résultat). Un itinéraire proposé par un GPS est un algorithme : il part d'une position de départ et arrive à une destination en suivant des instructions précises. De même, le calcul d'une moyenne scolaire, la vérification d'un code de carte bancaire ou le tri d'une liste de candidats sont des algorithmes.

Pour être correct, un algorithme doit respecter quatre propriétés fondamentales. Il doit être **fini** : il se termine après un nombre limité d'étapes. Il doit être **non ambigu** : chaque instruction est comprise de la même façon par tout lecteur. Il doit être **ordonné** : les étapes s'exécutent dans un ordre défini, chacune dépendant éventuellement des précédentes. Enfin, il doit être **efficace** : il produit le résultat attendu sans opérations inutiles.

Propriété	Signification	Contre-exemple à éviter
Fini	Se termine toujours, quelle que soit l'entrée	Une boucle qui ne s'arrête jamais
Non ambigu	Aucune instruction ne prête à interprétation	« Ajouter un peu de sel »
Ordonné	Les étapes suivent une séquence logique	Casser les œufs après avoir cuit l'omelette
Efficace	Évite les calculs ou déplacements inutiles	Recopier dix fois la même donnée

Tableau 1.1 : Les quatre propriétés d'un bon algorithme

1.2 Le pseudo-code : écrire avant de coder

Le **pseudo-code** est une façon de décrire un algorithme dans un langage intermédiaire entre le français et le code. Il n'existe pas de norme universelle : l'essentiel est d'être cohérent et clair. Dans ce cours, nous utilisons des mots-clés français (**Variables**, **Lire**, **Afficher**, **SI ... ALORS ... SINON**, **POUR ... FIN POUR**, **TANT QUE ... FIN TANT QUE**) et la flèche ← pour l'affectation d'une valeur à une variable. L'avantage majeur du pseudo-code est qu'il vous oblige à réfléchir à la **logique** sans être distrait par la syntaxe : les erreurs que vous ferez seront de vraies erreurs de raisonnement, pas des fautes de frappe.

Algorithme 1.1 : Calcul de la moyenne de deux notes

```
ALGORITHME MoyenneDeuxNotes
VARIABLES
    note1, note2, moyenne : REEL
DEBUT
    Afficher "Entrez la première note : "
    Lire note1
    Afficher "Entrez la deuxième note : "
    Lire note2
    moyenne ← (note1 + note2) / 2
    Afficher "La moyenne est : ", moyenne
FIN
```

Lisez cet algorithme comme un scénario d'exécution : l'ordinateur affiche une invite, attend une valeur, la range dans une case mémoire nommée **note1**, puis recommence avec **note2**. La ligne de calcul est évaluée de la droite vers la gauche : l'expression (note1 + note2) / 2 est d'abord calculée, puis son résultat est stocké dans **moyenne**. Enfin, le résultat est affiché. Ce schéma « saisir, traiter, afficher » est la colonne vertébrale de presque tous les programmes que vous écrirez.

1.3 Les variables et les types

Une **variable** est une case mémoire étiquetée par un nom, dans laquelle le programme range une valeur susceptible de changer pendant l'exécution. Imaginez une armoire munie de tiroirs : chaque tiroir porte une étiquette (le nom de la variable) et contient un objet (la valeur). Le **type** d'une variable décrit la nature de la valeur qu'elle contient : un entier, un nombre à virgule, du texte ou un résultat logique. Connaître le type est essentiel, car il détermine les opérations possibles : on peut additionner deux entiers, mais additionner un entier et un texte n'a pas de sens.

Type	Contenu	Exemples de valeurs
ENTIER	Nombre sans virgule	-3, 0, 17, 2026
REEL	Nombre à virgule flottante	3.14, -0.5, 12.0
CHAINE	Texte entre guillemets	"Roland", "A +", "12"
BOOLEEN	Vrai ou faux	VRAI, FAUX

Tableau 1.2 : Les types fondamentaux en algorithmique

Algorithme 1.2 : Échange du contenu de deux variables

```
ALGORITHME EchangeDeuxVariables
VARIABLES
    a, b, temporaire : ENTIER
DEBUT
    a ← 5
    b ← 8
    temporaire ← a          // on sauvegarde la valeur de a
    a ← b                   // a reçoit la valeur de b
    b ← temporaire          // b reçoit l'ancienne valeur de a
    Afficher "a vaut ", a, " et b vaut ", b    // a vaut 8, b vaut 5
FIN
```

ATTENTION

L'échange direct « a ← b » suivi de « b ← a » est un piège classique : après la première affectation, l'ancienne valeur de a est définitivement perdue ! Il faut toujours passer par une variable intermédiaire (ici temporaire) pour sauvegarder la donnée avant de l'écraser. Nous verrons au chapitre 5 que Python permet l'échange direct a, b = b, a, mais en algorithmique la technique de la variable temporaire reste la référence.

À RETENIR

Un algorithme est une suite finie et non ambiguë d'instructions qui transforme des données d'entrée en résultat. On l'écrit d'abord en pseudo-code, avec des variables typées et des mots-clés clairs. Le schéma de base de presque tous les programmes est : saisir des données, les traiter, puis afficher le résultat.

CHAPITRE 2

Les structures conditionnelles

Un programme intéressant ne suit pas toujours le même chemin : il réagit aux données. Faut-il accorder un prêt ? La note est-elle suffisante ? Le mot de passe est-il correct ? Autant de questions qui appellent des **tests**. Les structures conditionnelles permettent à l'algorithme de choisir entre plusieurs routes selon qu'une condition est vraie ou fausse. C'est le deuxième pilier de la logique de programmation, avec les boucles que nous verrons au chapitre suivant.

2.1 La structure SI ... ALORS

La forme la plus simple de la condition exécute un bloc d'instructions seulement si une condition est vraie. Si la condition est fausse, le bloc est entièrement sauté et le programme continue après le FIN SI. La condition est toujours une expression booléenne : une comparaison ou une combinaison de comparaisons dont le résultat est VRAI ou FAUX.

Algorithme 2.1 : Vérifier la majorité d'une personne

```
ALGORITHME VerificationMajorite
VARIABLES
    age : ENTIER
DEBUT
    Afficher "Quel est votre age ? "
    Lire age
    SI age >= 18 ALORS
        Afficher "Vous etes majeur."
    FIN SI
    Afficher "Fin du programme."
FIN
```

Observez la dernière instruction : « Fin du programme » s'affiche dans tous les cas, qu'il pleuve ou qu'il fasse beau. C'est un point important : un bloc SI ... ALORS conditionne uniquement les instructions situées entre ALORS et FIN SI. Les instructions situées après FIN SI reprennent leur exécution normale. Ce détail paraît évident, mais de nombreux bugs de débutants viennent d'une instruction placée au mauvais endroit du bloc.

2.2 La structure SI ... ALORS ... SINON

Quand deux cas se partagent l'espace des possibilités (majeur ou mineur, pair ou impair, suffisant ou insuffisant), la structure SI ... ALORS ... SINON est idéale. Exactement l'un des deux blocs s'exécute : celui de la condition si elle est vraie, celui du SINON sinon. Là où deux SI séparés évalueraient deux fois des conditions, la structure SINON exprime clairement l'exclusivité des deux cas.

Algorithme 2.2 : Pair ou impair

```
ALGORITHME PairOuImpair
VARIABLES
```

```
    nombre, reste : ENTIER
DEBUT
    Afficher "Entrez un nombre entier : "
    Lire nombre
    reste ← nombre MOD 2
    SI reste = 0 ALORS
        Afficher "Le nombre est pair."
    SINON
        Afficher "Le nombre est impair."
    FIN SI
FIN
```

L'opérateur **MOD** (modulo) calcule le reste de la division euclidienne : 7 MOD 2 vaut 1, 8 MOD 2 vaut 0, 15 MOD 5 vaut 0. C'est l'outil universel des tests de divisibilité : un nombre est pair si son reste par 2 est nul, divisible par 3 si son reste par 3 est nul, et ainsi de suite. Retenez bien cet opérateur : il reviendra dans les boucles, les jeux et la cryptographie.

2.3 Les conditions imbriquées et le SI MULTIPLE

Les décisions de la vie réelle ont souvent plus de deux issues. Pour attribuer une mention, quatre cas existent : Excellent, Très bien, Passable, Insuffisant. On peut imbriquer des SI dans des SI, mais le code devient vite difficile à lire. Le mot-clé **SINON SI** (parfois écrit SI MULTIPLE) permet d'enchaîner les tests proprement : le premier test vrai déclenche son bloc, et tous les suivants sont ignorés. C'est une cascade de filtres dans laquelle chaque donnée est testée de haut en bas.

Algorithme 2.3 : Attribution d'une mention

```
ALGORITHME AttributionMention
VARIABLES
    note : REEL
DEBUT
    Afficher "Entrez la note (0 a 20) : "
    Lire note
    SI note >= 16 ALORS
        Afficher "Mention Excellent"
    SINON SI note >= 14 ALORS
        Afficher "Mention Tres bien"
    SINON SI note >= 10 ALORS
        Afficher "Passable"
    SINON
        Afficher "Insuffisant"
    FIN SI
FIN
```

ATTENTION

L'ordre des tests dans une cascade SINON SI est crucial. Si l'on teste d'abord « note >= 10 », une note de 17 s'arrêtera au premier filtre et affichera « Passable » ! Le principe est simple : du plus sélectif au plus général. Notez aussi qu'il est inutile d'écrire « note >= 14 ET note < 16 » : le fait d'arriver au deuxième test prouve déjà que le premier a échoué, donc que note est inférieure à 16.

2.4 Les opérateurs de comparaison et les opérateurs logiques

Une condition s'écrit avec des opérateurs de comparaison, éventuellement reliés par des opérateurs logiques. Les opérateurs de comparaison comparent deux valeurs et produisent VRAI ou FAUX ; les opérateurs logiques combinent des conditions entre elles. Ces opérateurs sont les briques élémentaires de toutes les décisions, et vous les retrouverez à l'identique en Python au chapitre 7.

Opérateur	Signification	Exemple (VRAI si ...)
=	égal à	note = 15
!=	différent de	pays != "Togo"
<	strictement inférieur	age < 18
<=	inférieur ou égal	note <= 20
>	strictement supérieur	solde > 0
>=	supérieur ou égal	note >= 10
ET	les deux conditions sont vraies	age >= 18 ET age <= 65
OU	au moins une condition est vraie	jour = "Samedi" OU jour = "Dimanche"
NON	inverse la condition	NON (a = b)

Tableau 2.1 : Opérateurs de comparaison et logiques

À RETENIR

Trois structures conditionnelles suffisent à tout exprimer : SI ... ALORS pour exécuter sous condition, SI ... ALORS ... SINON pour choisir entre deux cas, et la cascade SINON SI pour traiter plusieurs cas en allant du plus sélectif au plus général. L'opérateur MOD teste la divisibilité, et les opérateurs ET / OU / NON combinent les conditions.

CHAPITRE 3

Les boucles et répétitions

Répéter une opération est la raison d'être des ordinateurs : afficher mille factures, additionner les notes d'une classe, vérifier chaque mot d'un dictionnaire. Écrire mille fois la même instruction serait absurde ; les **boucles** permettent de demander à la machine de répéter elle-même un bloc d'instructions. Il existe deux familles de boucles : celles dont on connaît à l'avance le nombre de répétitions, et celles qui s'arrêtent lorsqu'une condition cesse d'être vraie.

3.1 La boucle POUR : quand on connaît le nombre de tours

La boucle POUR exécute un bloc un nombre déterminé de fois, en faisant varier automatiquement une variable de comptage appelée **compteur**. On précise la valeur de départ, la valeur finale et éventuellement le pas. La boucle est utilisée dès que le nombre d'itérations est connu avant de commencer : traiter les 30 jours d'un mois, parcourir les 12 mois de l'année, répéter 5 fois un essai. Le compteur est géré par la boucle elle-même : il s'incrémente à chaque tour, et la boucle se termine automatiquement lorsque le compteur dépasse la valeur finale.

Algorithme 3.1 : Table de multiplication d'un nombre

```
ALGORITHME TableDeMultiplication
VARIABLES
    nombre, i, produit : ENTIER
DEBUT
    Afficher "Table de quel nombre ? "
    Lire nombre
    POUR i DE 1 A 10 FAIRE
        produit ← nombre * i
        Afficher nombre, " x ", i, " = ", produit
    FIN POUR
FIN
```

Trace d'exécution pour nombre = 7 : au premier tour, i vaut 1 et le programme affiche « 7 x 1 = 7 » ; au deuxième tour, i vaut 2 et il affiche « 7 x 2 = 14 » ; et ainsi de jusqu'au dixième tour qui affiche « 7 x 10 = 70 ». Le bloc situé entre POUR et FIN POUR (une seule instruction ici) a donc été exécuté exactement dix fois sans que nous ayons besoin de l'écrire dix fois. Notez que la variable produit évite de recalculer l'expression à l'affichage : c'est une bonne habitude de lisibilité.

3.2 La boucle TANT QUE : quand la fin dépend des données

La boucle TANT QUE répète un bloc tant qu'une condition reste vraie, et s'arrête dès qu'elle devient fausse. Elle est indispensable lorsque le nombre d'itérations n'est pas connu à l'avance : combien de chiffres l'utilisateur va-t-il saisir ? Quand tombera la première commande erronée ? Le test de la condition a lieu **avant** chaque tour : si la condition est fausse dès le départ, le bloc n'est jamais exécuté. La condition doit obligatoirement finir par devenir fausse, sinon la boucle tourne indéfiniment.

Algorithme 3.2 : Somme des nombres saisis (sentinelle 0)

```
ALGORITHME SommeNombresSaisis
VARIABLES
    nombre, somme : ENTIER
DEBUT
    somme ← 0
    Afficher "Entrez un nombre (0 pour arreter) : "
    Lire nombre
    TANT QUE nombre != 0 FAIRE
        somme ← somme + nombre
        Afficher "Entrez un nombre (0 pour arreter) : "
        Lire nombre
    FIN TANT QUE
    Afficher "Somme totale : ", somme
FIN
```

ATTENTION

Oublier de faire évoluer la condition est la cause numéro un de boucle infinie. Dans l'algorithme 3.2, si vous supprimez l'instruction « Lire nombre » à l'intérieur de la boucle, la condition nombre != 0 ne changera jamais et le programme tournera à l'infini. Toute boucle TANT QUE doit contenir une instruction qui rapproche la condition de sa fin : une nouvelle saisie, un compteur incrémenté, une donnée modifiée.

3.3 Choisir la bonne boucle

Les deux boucles ne sont pas interchangeables. La règle de décision tient en une question : **connais-je à l'avance le nombre de répétitions ?** Si oui (traiter N éléments, répéter 10 essais), la boucle POUR s'impose et son compteur protège contre les itérations manquantes ou superflues. Si non (attendre une valeur précise, traiter un flux jusqu'à sa fin, redemander une saisie jusqu'à validation), la boucle TANT QUE est le bon outil. En cas de doute, écrivez la condition d'arrêt en français : si elle mentionne un nombre d'éléments connu, prenez POUR ; si elle mentionne un événement, prenez TANT QUE.

Critère	Boucle POUR	Boucle TANT QUE
Nombre d'itérations	Connu à l'avance	Inconnu, dépend des données
Condition d'arrêt	Compteur atteint la borne finale	Condition devient fausse
Risque principal	Borne de départ ou d'arrivée erronée	Boucle infinie
Exemple typique	Parcourir un tableau de 50 cases	Redemander un mot de passe jusqu'à ce qu'il soit correct

Tableau 3.1 : POUR ou TANT QUE ?

À RETENIR

La boucle POUR répète un nombre de fois connu grâce à un compteur automatique ; la boucle TANT QUE répète tant qu'une condition est vraie et doit obligatoirement contenir une instruction qui fait évoluer cette condition. Avant d'écrire une boucle, identifiez toujours : ce qui se répète (le corps), quand on commence et quand on s'arrête (la condition).

CHAPITRE 4

Tableaux et fonctions algorithmiques

Jusqu'ici, chaque donnée avait son propre nom : note1, note2, age. Comment faire pour gérer les 25 notes d'une classe ? Déclarer 25 variables serait ingérable. Les **tableaux** regroupent sous un seul nom un ensemble de valeurs de même type, accessibles par leur position. Et lorsque le même calcul revient à plusieurs endroits du programme, les **fonctions** permettent de l'écrire une seule fois et de le réutiliser partout. Ces deux outils préparent directement les listes et les fonctions Python des parties suivantes.

4.1 Déclarer et parcourir un tableau

Un tableau est une suite de cases numérotées, toutes de même type. Le numéro d'une case s'appelle son **indice**. Dans nos algorithmes, nous numérotons les cases à partir de 1 pour la lisibilité du pseudo-code ; retenez toutefois dès maintenant que Python, comme la plupart des langages modernes, numérote à partir de 0 : nous y reviendrons longuement au chapitre 9. La déclaration précise le type des éléments et la taille du tableau, c'est-à-dire son nombre de cases, qui peut être lu au moment de l'exécution.

Algorithme 4.1 : Recherche du maximum d'un tableau

```
ALGORITHME MaximumDuTableau
VARIABLES
    T : TABLEAU[1..N] DE REEL
    i : ENTIER
    maximum : REEL
DEBUT
    Afficher "Combien de valeurs ? "
    Lire N
    POUR i DE 1 A N FAIRE
        Afficher "Valeur numero ", i, " : "
        Lire T[i]
    FIN POUR
    maximum ← T[1]
    POUR i DE 2 A N FAIRE
        SI T[i] > maximum ALORS
            maximum ← T[i]
        FIN SI
    FIN POUR
    Afficher "La plus grande valeur est : ", maximum
FIN
```

Cet algorithme introduit une technique fondamentale : la **recherche par candidat courant**. On suppose que le premier élément est le maximum, puis on le confronte à chacun des suivants ; chaque fois qu'un élément le dépasse, il devient le nouveau candidat. À la fin du parcours, le candidat courant est le vrai maximum. Cette stratégie de « champion en titre » s'applique aussi bien à la recherche du minimum, du mot le plus long ou du vendeur le plus performant.

4.2 Les fonctions : écrire une fois, utiliser partout

Une **fonction** est un bloc d'instructions nommé, qui reçoit des valeurs d'entrée appelées **paramètres**, effectue un traitement et renvoie un **résultat** au programme appelant. Le programme principal appelle la fonction par son nom et récupère le résultat grâce au mot-clé RETOURNER. La fonction transforme ainsi un calcul en brique réutilisable : définie une fois, elle peut être appelée dix fois, cent fois, ou par d'autres programmes. Ce principe, la factorisation du code, est l'un des fondements de l'informatique professionnelle.

Algorithme 4.2 : Une fonction de calcul de moyenne pondérée

```
FONCTION MoyennePonderee(note1, note2, coef1, coef2)
    // Retourne la moyenne ponderee de deux notes
    total ← (note1 * coef1) + (note2 * coef2)
    resultat ← total / (coef1 + coef2)
    RETOURNER resultat
FIN FONCTION
ALGORITHME ProgrammePrincipal
VARIABLES
    n1, n2, m : REEL
DEBUT
    n1 ← 12.5
    n2 ← 15
    m ← MoyennePonderee(n1, n2, 2, 3)
    Afficher "Moyenne ponderee : ", m     // affiche 14  (calcul : (25 + 45) / 5)
FIN
```

Suivez le trajet des données : le programme principal envoie 12.5, 15, 2 et 3 à la fonction ; les paramètres note1, note2, coef1 et coef2 reçoivent ces valeurs dans l'ordre d'appel ; la fonction calcule (12.5 × 2 + 15 × 3) / (2 + 3) = (25 + 45) / 5 = 70 / 5 = 14. Prenez le temps de refaire ce calcul vous-même : ce type de vérification manuelle, appelée **jeu de test**, est ce qui vous rendra autonome pour déboguer. Le résultat 14 est retourné et rangé dans la variable m.

ASTUCE

Avant d'écrire un algorithme, listez sur papier les entrées, la sortie attendue et deux ou trois jeux de test avec leurs résultats calculés à la main. Par exemple pour la fonction MoyennePonderee : (12.5, 15, 2, 3) donne 14 ; (10, 10, 1, 1) donne 10. Ces jeux de test deviennent votre filet de sécurité : après avoir écrit l'algorithme, rejouez-les pour valider votre logique.

À RETENIR

Un tableau range plusieurs valeurs de même type sous un seul nom, chacune accessible par son indice. Le parcours complet se fait avec une boucle POUR. La recherche du maximum s'effectue par candidat courant : supposer le premier élément champion, puis remplacer à chaque dépassement. Une fonction regroupe un calcul derrière un nom, reçoit des paramètres et retourne un résultat, rendant le code réutilisable et lisible.

Exercices corrigés de la Partie I

Ces cinq exercices s'effectuent uniquement en pseudo-code. Pour chacun, commencez par identifier les entrées, la sortie et la stratégie, exactement comme nous l'avons fait dans les chapitres, puis rédigez l'algorithme complet avant de lire la solution.

Ex. 1 Périmètre et aire d'un rectangle

BASIQUE

Écrivez un algorithme qui lit la longueur et la largeur d'un rectangle, puis affiche son périmètre et son aire. Rappel : périmètre = 2 × (longueur + largeur) et aire = longueur × largeur. Vérifiez votre algorithme avec le jeu de test suivant : longueur = 5 et largeur = 3 doivent donner un périmètre de 16 et une aire de 15.

Solution

Stratégie : c'est le schéma « saisir, traiter, afficher » pur, sans condition ni boucle. Deux entrées, deux calculs, deux affichages.

Solution de l'exercice 1

```
ALGORITHME Rectangle
VARIABLES
    longueur, largeur, perimetre, aire : REEL
DEBUT
    Afficher "Longueur : "
    Lire longueur
    Afficher "Largeur : "
    Lire largeur
    perimetre ← 2 * (longueur + largeur)
    aire ← longueur * largeur
    Afficher "Perimetre = ", perimetre
    Afficher "Aire = ", aire
FIN
```

Vérification avec le jeu de test : pour 5 et 3, le périmètre vaut 2 × (5 + 3) = 16 et l'aire vaut 5 × 3 = 15, conformément à l'attendu. Les parenthèses dans le calcul du périmètre sont obligatoires : sans elles, la multiplication s'effectuerait avant l'addition et donnerait un résultat faux (2 × 5 + 3 = 13).

Ex. 2 Salaire avec heures supplémentaires

BASIQUE

Un salarié est payé au taux horaire. Au-delà de 40 heures hebdomadaires, chaque heure supplémentaire est payée avec une majoration de 50 % (soit 1,5 fois le taux). Écrivez l'algorithme qui lit le nombre d'heures travaillées et le taux horaire, puis calcule le salaire hebdomadaire. Test : 45 heures à 2000 FCFA de l'heure doivent donner 95000 FCFA.

Solution

Stratégie : deux cas se partagent le problème. Si les heures sont inférieures ou égales à 40, tout est payé au taux normal. Sinon, les 40 premières heures restent au taux normal et seules les heures au-delà de 40 sont majorées. La structure SI ... SINON s'impose naturellement.

`Solution de l'exercice 2`

```
ALGORITHME SalaireHebdomadaire
VARIABLES
    heures : ENTIER
    taux, salaire : REEL
DEBUT
    Afficher "Heures travaillees : "
    Lire heures
    Afficher "Taux horaire : "
    Lire taux
    SI heures <= 40 ALORS
        salaire ← heures * taux
    SINON
        salaire ← 40 * taux + (heures - 40) * 1.5 * taux
    FIN SI
    Afficher "Salaire de la semaine : ", salaire
FIN
```

Vérification : pour 45 heures à 2000 FCFA, on paie 40 × 2000 = 80000 FCFA pour les heures normales, plus 5 heures supplémentaires à 1,5 × 2000 = 3000 FCFA, soit 5 × 3000 = 15000 FCFA. Total : 80000 + 15000 = 95000 FCFA. La formule du SINON traite les deux tranches en une seule ligne : c'est le calcul dit « par tranches », que l'on retrouve dans les impôts, les factures d'électricité et les frais bancaires.

Ex. 3 Le plus grand de trois nombres

INTERMÉDIAIRE

Écrivez un algorithme qui lit trois nombres et affiche le plus grand, **sans utiliser de tableau**. Bonus : réécrivez la solution en créant d'abord une fonction Maximum(a, b) qui renvoie le plus grand de deux nombres, puis en l'utilisant deux fois.

Solution

Stratégie directe : une cascade de comparaisons. Si a est supérieur ou égal à b et à c, c'est lui le champion ; sinon, on départage b et c. La version bonus est plus élégante : comparer deux à deux grâce à une fonction réutilisable, ce qui réduit le risque d'oublier un cas.

Solution de l'exercice 3 : version directe puis version factorisée

```
// Version 1 : comparaisons directes
ALGORITHME PlusGrandDeTrois
VARIABLES
    a, b, c : REEL
DEBUT
    Lire a, b, c
    SI a >= b ET a >= c ALORS
        Afficher "Le plus grand est : ", a
    SINON SI b >= a ET b >= c ALORS
        Afficher "Le plus grand est : ", b
    SINON
        Afficher "Le plus grand est : ", c
    FIN SI
FIN
// Version 2 : avec une fonction reutilisable
FONCTION Maximum(x, y)
    SI x >= y ALORS
        RETOURNER x
    SINON
        RETOURNER y
    FIN SI
FIN FONCTION
ALGORITHME PlusGrandDeTroisV2
VARIABLES
    a, b, c : REEL
DEBUT
    Lire a, b, c
    Afficher "Le plus grand est : ", Maximum(a, Maximum(b, c))
FIN
```

Dans la version 2, l'appel interne Maximum(b, c) est évalué d'abord et renvoie le champion du duo (b, c) ; l'appel externe départage ensuite a et ce champion. Cette composition d'appels, qui passe le résultat d'une fonction comme paramètre d'une autre, est un réflexe de développeur confirmé : elle exprime l'idée « le maximum de trois nombres est le maximum entre le premier et le maximum des deux autres ».

Ex. 4 Somme des nombres pairs de 1 à N

INTERMÉDIAIRE

Écrivez un algorithme qui lit un entier N puis calcule la somme de tous les nombres pairs compris entre 1 et N. Exemple : pour N = 10, la somme vaut 2 + 4 + 6 + 8 + 10 = 30. Proposez une première version qui teste chaque nombre, puis une version améliorée qui n'effectue aucun test.

Solution

Version 1 (stratégie de test) : parcourir tous les entiers de 1 à N avec une boucle POUR et n'ajouter à la somme que ceux dont le reste par 2 est nul. Version 2 (stratégie de pas) : si l'on démarre à 2 et que le compteur avance de 2 en 2, on ne rencontre que des pairs, le test devient inutile.

Solution de l'exercice 4 : deux stratégies de parcours

```
// Version 1 : boucle + test de parite
ALGORITHME SommePairsV1
VARIABLES
    N, i, somme : ENTIER
DEBUT
    Lire N
    somme ← 0
    POUR i DE 1 A N FAIRE
        SI i MOD 2 = 0 ALORS
            somme ← somme + i
        FIN SI
    FIN POUR
    Afficher "Somme des pairs : ", somme
FIN
// Version 2 : pas de 2, aucun test
ALGORITHME SommePairsV2
VARIABLES
    N, i, somme : ENTIER
DEBUT
    Lire N
    somme ← 0
    POUR i DE 2 A N AVEC PAS DE 2 FAIRE
        somme ← somme + i
    FIN POUR
    Afficher "Somme des pairs : ", somme
FIN
```

Les deux versions donnent 30 pour N = 10, mais la seconde fait moitié moins de tours de boucle et ne contient aucun test : elle est donc plus rapide et plus simple à relire. La leçon est importante : avant de tester chaque élément, demandez-vous si un parcours mieux conçu peut éliminer le besoin de tester. Ce raisonnement sur le pas de progression prépare l'utilisation de range(2, N+1, 2) en Python.

Ex. 5 Tester si un nombre est premier DÉFI

Un nombre premier est un entier supérieur ou égal à 2 qui n'est divisible que par 1 et par lui-même (2, 3, 5, 7, 11, ...). Écrivez un algorithme qui lit un entier N et affiche « Premier » ou « Non premier ». Utilisez la technique du **drapeau** (variable booléenne) et justifiez pourquoi il suffit de tester les diviseurs jusqu'à la racine carrée de N.

Solution

Stratégie : tester la divisibilité de N par chaque entier d allant de 2 à la racine carrée de N. Si un diviseur est trouvé, N n'est pas premier et il est inutile de continuer. Le drapeau estPremier mémorise le verdict : initialisé à VRAI avant la boucle, il bascule à FAUX dès la première divisibilité détectée. Pourquoi s'arrêter à la racine carrée ? Si N possède un diviseur d supérieur à sa racine carrée, alors N / d est un diviseur inférieur à la racine carrée, déjà trouvé : tester au-delà est donc du travail en double.

Solution de l'exercice 5 : test de primalité avec drapeau

```
ALGORITHME NombrePremier
VARIABLES
    N, d : ENTIER
    estPremier : BOOLEEN
DEBUT
    Afficher "Entrez un entier superieur ou egal a 2 : "
    Lire N
    estPremier ← VRAI
    d ← 2
    TANT QUE d * d <= N ET estPremier = VRAI FAIRE
        SI N MOD d = 0 ALORS
            estPremier ← FAUX
        SINON
            d ← d + 1
        FIN SI
    FIN TANT QUE
    SI estPremier = VRAI ALORS
        Afficher N, " est premier."
    SINON
        Afficher N, " n'est pas premier."
    FIN SI
FIN
```

Trace d'exécution pour N = 7 : le drapeau démarre à VRAI et d vaut 2. Condition : 2 × 2 = 4 <= 7 et drapeau vrai → on entre dans la boucle. 7 MOD 2 = 1, pas divisible, donc d passe à 3. Condition : 3 × 3 = 9 <= 7 est FAUX → sortie de boucle. Verdict : drapeau toujours VRAI, donc 7 est premier. Observez la double condition du TANT QUE : « d * d <= N » garantit qu'on ne dépasse pas la racine carrée, et « ET estPremier = VRAI » court-circuite la boucle dès qu'un diviseur est trouvé, une optimisation classique dite de sortie anticipée. Le drapeau booléen est une technique universelle : vous la retrouverez dans les recherches, les validations de saisie et les jeux.

PARTIE II

Premiers pas en Python

Le raisonnement est construit, place à l'outil. Cette partie installe Python sur votre machine et transpose en syntaxe réelle tout ce que vous avez appris en pseudo-code : variables et types, conditions, boucles. Vous découvrirez à quel point la préparation algorithmique rend l'apprentissage du langage rapide et serein.

CHAPITRE 5

Découvrir Python et son environnement

Python est un langage de programmation créé par Guido van Rossum et publié en 1991. Sa devise officieuse (la lisibilité compte plus que tout) en fait le langage d'apprentissage préféré des écoles et des universités, tout en étant un outil de production utilisé par Google, Netflix, la NASA ou la banque mondiale des données scientifiques. On l'utilise pour le développement web, l'analyse de données, l'intelligence artificielle, l'automatisation de tâches bureautiques et l'écriture de petits utilitaires. Apprendre Python aujourd'hui, c'est investir dans une compétence demandée partout.

5.1 Installer Python et choisir un éditeur

Rendez-vous sur le site officiel **python.org**, section Downloads, et téléchargez la dernière version stable pour votre système (Windows, macOS ou Linux). Pendant l'installation sous Windows, cochez impérativement la case « Add Python to PATH » : elle permet de lancer Python depuis n'importe quel terminal. Vérifiez ensuite l'installation en ouvrant un terminal et en tapant la commande suivante :

Vérification de l'installation (à taper dans le terminal)

```
python -V
# Exemple de reponse attendue :
# Python 3.12.4
```

Python 3 est livré avec **IDLE**, un éditeur simple parfaitement suffisant pour ce Tome 1. Quand vos projets grandiront, l'éditeur **Visual Studio Code** avec l'extension Python offrira la coloration syntaxique, l'autocomplétion et le débogage pas à pas. Quel que soit l'outil, le cycle de travail reste identique : écrire le code dans un fichier portant l'extension **.py**, l'exécuter, observer le résultat, corriger. C'est en enchaînant ce cycle des dizaines de fois par jour que la maîtrise s'installe.

5.2 La console interactive et les scripts

Python offre deux modes d'utilisation complémentaires. La **console interactive** (ou REPL) s'ouvre en tapant python dans un terminal : chaque ligne saisie est exécutée immédiatement et son résultat s'affiche. Elle est idéale pour tester une expression, découvrir le comportement d'une fonction ou faire office de calculatrice améliorée. Le mode **script** consiste à écrire un programme complet dans un fichier .py, puis à l'exécuter d'un bloc : c'est le mode de production, celui de tous les exemples de ce cours.

Code 5.1 : Le traditionnel premier programme

```
# Mon premier programme Python
print("Bonjour le monde !")
print("J'apprends Python avec M. HOUNKPE et M. DJABON.")
```

Trois choses méritent votre attention. D'abord, la fonction **print()** affiche à l'écran les valeurs qu'on lui passe entre parenthèses, l'équivalent du mot-clé Afficher de notre pseudo-code. Ensuite, la première ligne commence par un croisillon # : tout ce qui suit ce symbole, jusqu'à la fin de la ligne, est un **commentaire** ignoré par l'interpréteur, destiné au lecteur humain. Enfin, il n'y a ni déclaration de variables ni point-virgule : Python est conçu pour être bref.

5.3 L'indentation : une règle d'or en Python

Dans la plupart des langages, les blocs d'instructions sont délimités par des accolades ; l'indentation n'est qu'une question d'esthétique. Python fait le choix radical : **l'indentation définit les blocs**. Toutes les lignes appartenant au même bloc doivent commencer au même niveau, avec le même retrait (la convention universelle est de quatre espaces). Cette contrainte, surprenante au début, garantit une lisibilité homogène de tout le code Python du monde : là où un pseudo-code écrivait SI ... ALORS ... FIN SI, Python écrit une ligne if terminée par deux points, puis un bloc indenté en dessous.

Code 5.2 : Premier aperçu d'un bloc indenté

```
age = 20
if age >= 18:
    print("Vous etes majeur.")       # bloc execute si la condition est vraie
    print("Bienvenue !")             # meme bloc, meme indentation
print("Fin du programme.")           # hors du bloc : toujours execute
```

ATTENTION

Mélanger des tabulations et des espaces dans un même fichier produit l'erreur TabError, et un décalage d'un seul espace provoque le fameux IndentationError. Configurez votre éditeur pour remplacer automatiquement la touche Tab par quatre espaces, et ne modifiez jamais l'indentation « à la main » ligne par ligne : sélectionnez les lignes et utilisez l'indentation automatique de l'éditeur.

À RETENIR

Python s'installe depuis python.org et se vérifie avec python -V. Le mode script écrit des programmes dans des fichiers .py ; la console interactive sert à tester des expressions isolées. print() affiche, # commente, et l'indentation de quatre espaces délimite les blocs : en Python, l'indentation est une obligation syntaxique, pas une préférence.

CHAPITRE 6

Variables et types de données

Reprenez l'image du tiroir étiqueté du chapitre 1 : en Python, créer une variable revient à coller une étiquette sur une valeur. La syntaxe tient en un signe : l'opérateur d'affectation =, lu « reçoit la valeur ». Pas de déclaration de type, pas de mot-clé : la variable naît au moment de sa première affectation et son type est déduit automatiquement de la valeur. On appelle cela le **typage dynamique**.

6.1 Créer et nommer ses variables

Un nom de variable ne peut contenir que des lettres, des chiffres et le tiret bas _ ; il ne peut pas commencer par un chiffre et ne doit pas être un mot réservé du langage (if, for, class...). La convention universelle des développeurs Python, documentée dans la norme PEP 8, est le **snake_case** : des lettres minuscules reliées par des tirets bas. Préférez des noms descriptifs : note_moyenne plutôt que nm, prix_total plutôt que x. Un bon nom de variable est déjà la moitié d'un bon commentaire.

Code 6.1 : Affectations et conventions de nommage

```python
prenom = "Roland"             # chaine de caracteres
age = 27                      # entier
taille = 1.78                 # nombre a virgule (float)
etudiant = True               # booleen : True ou False
# Affectation multiple sur une seule ligne
x, y = 3, 8
print("Prenom :", prenom, "| Age :", age, "| x + y =", x + y)
# Reaffectation : la variable change de valeur (et meme de type !)
age = age + 1                 # age vaut maintenant 28
age = "vingt-neuf ans"        # legale en Python, mais deconseille
```

ATTENTION

Le signe = n'est pas une égalité mathématique mais une affectation : x = x + 1 signifie « calculer x + 1, puis ranger le résultat dans x ». L'égalité de comparaison s'écrit avec deux signes : x == y. Confondre les deux est l'erreur la plus fréquente des six premiers mois d'apprentissage.

6.2 Les quatre types de base

Les types fondamentaux du pseudo-code se retrouvent en Python sous de nouveaux noms : ENTIER devient int, REEL devient float, CHAINE devient str et BOOLEEN devient bool. La fonction intégrée **type()** révèle le type de n'importe quelle valeur : utilisez-la en console interactive pour explorer, c'est un microscope permanent sur vos données.

Type	Contenu	Exemples de littéraux
int	Nombres entiers, taille illimitée	0, 42, -7, 2026
float	Nombres à virgule flottante	3.14, -0.5, 2.0, 1e6
str	Textes entre guillemets simples ou doubles	'Roland', "Tome 1", '12'
bool	Valeurs logiques (première lettre en majuscule)	True, False

Tableau 6.1 : Les types de base et leurs littéraux

Un point de vigilance s'impose pour les floats : la virgule décimale s'écrit avec un **point**, jamais avec une virgule : 1.78 est un float, mais 1,78 est interprété comme deux valeurs distinctes. Par ailleurs, la notation scientifique 1e6 désigne 1 million : le e se lit « fois dix puissance ». Quant au booléen, il s'écrit True ou False avec une majuscule initiale : true en minuscule provoquerait une erreur de nom inconnu.

6.3 Les conversions de type

Les fonctions int(), float(), str() et bool() convertissent une valeur vers le type correspondant. La conversion est indispensable dans un cas que vous rencontrerez dès votre premier programme interactif : la fonction **input()**, qui lit une saisie au clavier, renvoie **toujours** une chaîne de caractères, même si l'utilisateur a tapé un nombre. Avant tout calcul, il faut donc convertir la saisie avec int() ou float().

Code 6.2 : Saisir des nombres et calculer : le trio input / conversion / print

```python
# Programme : moyenne de deux notes
note1 = input("Entrez la premiere note : ")   # input renvoie une CHAINE
note2 = input("Entrez la deuxieme note : ")
note1 = float(note1)                          # conversion en nombre
note2 = float(note2)
moyenne = (note1 + note2) / 2
print("La moyenne est :", moyenne)
# Comparaison avec le pseudo-code de l'algorithme 1.1 :
#   Lire note1        devient    note1 = float(input(...))
#   moyenne <- ...    devient    moyenne = (note1 + note2) / 2
#   Afficher ...      devient    print(...)
# Erreur typique a eviter :
#   moyenne = (note1 + note2) / 2   AVANT conversion
#   -> "12.5" + "15" vaut "12.515" (concatenation) et non 27.5 !
```

6.4 Afficher élégamment avec les f-strings

Le passage de plusieurs arguments à print(), séparés par des virgules, insère des espaces entre eux mais donne peu de contrôle sur la mise en forme. Les **f-strings** (chaînes formatées, introduites par la lettre f devant le guillemet) permettent d'imbriquer des expressions directement dans le texte entre accolades. C'est la méthode moderne d'affichage, à la fois plus lisible et plus précise, avec des formats de présentation intégrés comme l'arrondi à deux décimales :.2f.

Code 6.3 : Les f-strings en action

```
prix_unitaire = 1500.0
quantite = 3
total = prix_unitaire * quantite
print(f"Article : {quantite} pieces a {prix_unitaire} F l'unite")
print(f"Total a payer : {total:.2f} FCFA")     # arrondi a 2 decimales
print(f"Avec remise de 10 pourcent : {total * 0.9:.2f} FCFA")
```

À RETENIR

Une variable Python naît à sa première affectation et son type suit sa valeur (typage dynamique). Les quatre types de base sont int, float, str et bool ; type() les révèle. input() renvoie toujours une chaîne : convertissez avant de calculer. Pour l'affichage, privilégiez les f-strings, qui intègrent expressions et formats comme :.2f.

CHAPITRE 7

Les conditions en Python

Le SI ... ALORS du pseudo-code devient if en Python, suivi d'une condition, de deux points et d'un bloc indenté. Toute la mécanique décisionnelle du chapitre 2 se transpose en quelques minutes : c'est la récompense de la préparation algorithmique. Nous enrichirons l'outillage avec les opérateurs anglais and, or, not et l'expression conditionnelle ternaire.

7.1 if, elif et else

La structure complète comporte trois niveaux : if teste la première condition ; elif (contraction de else if) teste des conditions alternatives, uniquement si toutes les précédentes ont échoué ; else, optionnel, rattrape tous les cas restants. Exactement comme dans la cascade SINON SI, le premier test vrai déclenche son bloc et la structure se termine : l'ordre des tests, du plus sélectif au plus général, reste crucial.

Code 7.1 : Attribution d'une mention (transposition de l'algorithme 2.3)

```
note = float(input("Entrez la note (0 a 20) : "))
if note >= 16:
    print("Mention Excellent")
elif note >= 14:
    print("Mention Tres bien")
elif note >= 10:
    print("Passable")
else:
    print("Insuffisant")
# Comparaison avec le pseudo-code :
#   SI note >= 16 ALORS     devient    if note >= 16:
#   SINON SI note >= 14     devient    elif note >= 14:
```

```
#   SINON                   devient    else:
#   FIN SI                  disparait : c'est la fin de l'indentation
```

Notez la disparition du FIN SI : en Python, la fin du bloc est matérialisée par le retour à l'indentation précédente. C'est un gain de concision, mais aussi une source classique d'erreur : une instruction que l'on croit dans le bloc alors que son indentation diffère. Relisez toujours vos conditions en vérifiant l'alignement vertical des lignes de chaque bloc.

7.2 Les opérateurs de comparaison et les opérateurs logiques

Les comparaisons en Python utilisent les mêmes symboles que le pseudo-code, avec une nuance : l'égalité s'écrit == et la différence !=. Les opérateurs logiques deviennent des mots anglais en minuscules : and remplace ET, or remplace OU, not remplace NON. Ces opérateurs évaluent leur résultat de façon paresseuse : dans a and b, si a est faux, b n'est même pas évaluée, propriété utile pour écrire des conditions de garde comme if n != 0 and total / n > 10.

Pseudo-code	Python	Exemple	Résultat
=	==	5 == 5	True
!=	!=	3 != 4	True
<	<	2 < 1	False
<=	<=	10 <= 10	True
ET	and	age >= 18 and age <= 65	dépend de age
OU	or	jour == 'Samedi' or jour == 'Dimanche'	dépend de jour
NON	not	not (a == b)	inverse du test

Tableau 7.1 : Opérateurs de comparaison et logiques en Python

7.3 Imbrication ou elif : le choix de la lisibilité

Techniquement, on peut toujours imbriquer un if dans un autre. Mais chaque niveau d'imbrication ajoute un cran d'indentation et demande au lecteur de retenir une condition de plus. La règle de lisibilité : privilégiez elif pour des cas exclusifs au même niveau, et réservez l'imbrication aux décisions réellement hiérarchiques (par exemple tester d'abord que la saisie est valide, puis traiter les sous-cas valides). Un code qui s'aligne à droite sur quatre niveaux est presque toujours un code à restructurer.

Code 7.2 : Imbrication justifiée : validation puis traitement

```python
note = float(input("Note : "))
if 0 <= note <= 20:                   # Python accepte les comparaisons chained !
    if note >= 10:
        print("Admis.")
    else:
        print("Ajourne.")
else:
    print("Erreur : la note doit etre entre 0 et 20.")
```

ASTUCE

Python accepte les comparaisons enchaînées : 0 <= note <= 20 se lit naturellement « note comprise entre 0 et 20 » et évite d'écrire note >= 0 and note <= 20. Cette syntaxe, absente de la plupart des langages, rend les tests d'intervalle limpides. Utilisez-la systématiquement pour les bornes.

7.4 L'expression conditionnelle ternaire

Pour les affectations qui dépendent d'une condition simple, Python propose une écriture compacte sur une seule ligne : valeur_si_vrai if condition else valeur_si_faux. Cette expression, dite ternaire, se lit de gauche à droite comme une question-réponse. Elle ne remplace pas les if complets : réservez-la aux choix courts entre deux valeurs, par exemple pour formater un affichage ou fixer une valeur par défaut.

Code 7.3 : L'expression conditionnelle

```
age = int(input("Age : "))
statut = "majeur" if age >= 18 else "mineur"
print(f"Vous etes {statut}.")
# Equivalent avec if/else classique :
# if age >= 18:
#     statut = "majeur"
# else:
#     statut = "mineur"
```

À RETENIR

if / elif / else transposent SI / SINON SI / SINON : deux points, bloc indenté, premier test vrai retenu. L'égalité s'écrit ==, les opérateurs logiques sont and, or et not, et les comparaisons peuvent s'enchaîner (0 <= x <= 20). L'expression ternaire valeur_si_vrai if condition else valeur_si_faux condense un choix simple sur une ligne.

CHAPITRE 8

Les boucles en Python

Python possède deux boucles, fidèles au pseudo-code : for pour les répétitions dont le nombre est connu, while pour les répétitions conditionnelles. La boucle for de Python, cependant, a une saveur particulière : elle parcourt des **séquences** (les éléments d'une liste, les caractères d'une chaîne, les nombres produits par range()) plutôt que de manipuler un compteur manuel. C'est ce qui la rend à la fois sûre et élégante.

8.1 La boucle for et la fonction range()

La fonction intégrée **range()** génère une suite de nombres entiers. Elle s'emploie sous trois formes : range(fin) produit les entiers de 0 à fin - 1 ; range(debut, fin) part de debut ; range(debut, fin, pas) avance avec un incrément donné. La borne de fin est **toujours exclue** : range(5) produit 0, 1, 2, 3, 4, soit exactement cinq valeurs. Cette convention « borne haute exclue » paraît déroutante au début, mais elle garantit que range(n) produit précisément n valeurs et se marie parfaitement avec les index de listes qui démarrent à 0.

Appel	Valeurs produites	Nombre de valeurs
range(5)	0, 1, 2, 3, 4	5
range(1, 6)	1, 2, 3, 4, 5	5
range(0, 10, 2)	0, 2, 4, 6, 8	5
range(10, 0, -2)	10, 8, 6, 4, 2	5

Tableau 8.1 : Les trois formes de range()

Code 8.1 : Table de multiplication (transposition de l'algorithme 3.1)

```python
nombre = int(input("Table de quel nombre ? "))
for i in range(1, 11):                # i prend les valeurs 1 a 10
    produit = nombre * i
    print(f"{nombre} x {i} = {produit}")
# Version compacte sans variable intermediaire :
# for i in range(1, 11):
#     print(f"{nombre} x {i} = {nombre * i}")
```

Comparez avec l'algorithme 3.1 : la transposition est immédiate. POUR i DE 1 A 10 FAIRE devient for i in range(1, 11), avec la subtilité de la borne 11 exclue, nécessaire pour atteindre 10. L'exercice 4 de la Partie I vous a fait écrire un parcours de 2 en 2 : sa version Python s'écrit for i in range(2, n + 1, 2), et la boucle descendante s'écrit range(10, 0, -1). Le pas négatif est la façon idiomatique de compter à rebours.

8.2 La boucle while

La boucle while transpose TANT QUE mot pour mot : une condition, deux points, un bloc indenté répété tant que la condition reste vraie. Elle excelle lorsque la fin dépend des données (validation de saisie, sentinelles, jeux avec tour par tour). La règle du chapitre 3 reste intacte : le corps de la boucle doit, à chaque tour, faire progresser la situation vers l'arrêt, sinon la boucle devient infinie.

Code 8.2 : Somme des saisies (transposition de l'algorithme 3.2)

```python
somme = 0
nombre = int(input("Entrez un nombre (0 pour arreter) : "))
while nombre != 0:                     # repete tant que la saisie n'est pas 0
    somme += nombre                    # equivalent a : somme = somme + nombre
    nombre = int(input("Entrez un nombre (0 pour arreter) : "))
print(f"Somme totale : {somme}")
```

8.3 break, continue et la clause else

Deux mots-clés affinent le contrôle des boucles. **break** provoque la sortie immédiate de la boucle englobante : dès qu'un objectif est atteint (un élément trouvé, une erreur détectée), il est inutile de finir les tours restants. **continue** saute le reste du corps et passe directement au tour suivant : il exprime « cet élément ne m'intéresse pas, passons ». La boucle for possède enfin une clause else optionnelle, exécutée uniquement si la boucle s'est achevée **sans break** : idéale pour signaler « aucune correspondance trouvée » après une recherche complète.

Code 8.3 : Recherche dans une liste avec break et la clause else

```python
fruits = ["mangue", "ananas", "papaye", "banane"]
recherche = input("Quel fruit cherchez-vous ? ")
for fruit in fruits:                   # parcours direct de la sequence
    if fruit == recherche:
        print(f"{recherche} est dans la liste !")
        break                          # trouve : inutile de continuer
else:
    print(f"{recherche} ne figure pas dans la liste.")
```

ATTENTION

while True suivi d'un break est un patron légitime pour les boucles à sortie multiple, mais un while dont la condition ne varie jamais est un bug : while nombre != 0 sans nouvelle saisie dans le corps tourne indéfiniment. Si votre programme « ne répond plus », il est probablement piégé dans une boucle infinie : interrompez-le avec Ctrl + C, puis vérifiez que la condition d'arrêt évolue à chaque tour.

8.4 Les boucles imbriquées

Une boucle peut en contenir une autre : c'est la **boucle imbriquée**, indispensable pour traiter les grilles, les tables et les combinaisons. Pour chaque tour de la boucle externe, la boucle interne effectue l'intégralité de ses tours. Le dessin de triangles en étoiles est l'exercice classique qui sculpte l'intuition : la ligne externe avance ligne par ligne, la ligne interne imprime le bon nombre d'étoiles.

Code 8.4 : Triangle d'étoiles

```
taille = int(input("Taille du triangle : "))
for ligne in range(1, taille + 1):          # lignes 1 a taille
    for etoile in range(ligne):             # ligne etoiles sur cette ligne
        print("*", end="")                  # end="" evite le retour a la ligne
    print()                                 # retour a la ligne de la ligne
```

> **À RETENIR**
>
> for i in range(...) gère les répétitions comptées (la borne haute est exclue) ; for element in sequence parcourt directement une liste ou une chaîne. while répète tant que sa condition est vraie et doit évoluer vers l'arrêt. break sort de la boucle, continue saute au tour suivant, et la clause else d'une boucle s'exécute si aucun break n'a eu lieu. Les boucles imbriquées multiplient les parcours : ligne par ligne, colonne par colonne.

Exercices corrigés de la Partie II

Passez en mode script : chaque solution ci-dessous est un programme complet à écrire dans un fichier .py, à exécuter, puis à comparer avec la correction. Testez toujours vos programmes avec les jeux de données fournis avant de lire la solution.

Ex. 6 Convertisseur Celsius vers Fahrenheit BASIQUE

Écrivez un programme qui lit une température en degrés Celsius et affiche son équivalent en degrés Fahrenheit, arrondi à une décimale. Formule : F = C × 9/5 + 32. Jeu de test : 25 °C donnent 77.0 °F, et 0 °C donne 32.0 °F.

Solution

Stratégie : saisie, conversion en float, formule, affichage en f-string avec le format :.1f pour une décimale. Ce programme est le portrait miniature du schéma « saisir, traiter, afficher ».

Solution de l'exercice 6

```
celsius = float(input("Temperature en degres Celsius : "))
fahrenheit = celsius * 9 / 5 + 32
print(f"{celsius} degres C correspondent a {fahrenheit:.1f} degres F")
```

Pour 25, l'exécution produit : 25.0 degrés C correspondent à 77.0 degrés F. Remarquez que la saisie 25 devient 25.0 après conversion : float() accepte aussi bien 25 que 25.0. Si vous aviez utilisé int(input(...)), la saisie de 25.5 aurait levé une ValueError, choisissez donc le type de conversion selon les données attendues.

Ex. 7 Pair ou impair, version Python BASIQUE

Écrivez un programme qui lit un entier et affiche « Pair » ou « Impair » à l'aide de l'opérateur modulo %. Enchaînez avec une variante qui affiche le verdict pour tous les nombres de 1 à 20 grâce à une boucle for.

Solution

L'opérateur MOD du pseudo-code s'écrit % en Python. Un nombre est pair quand son reste par 2 vaut 0. La variante en boucle combine le for du chapitre 8 avec le if du chapitre 7 : deux notions en un programme de six lignes.

Solution de l'exercice 7

```python
# Version 1 : un seul nombre
n = int(input("Entrez un entier : "))
if n % 2 == 0:
    print(f"{n} est pair")
else:
    print(f"{n} est impair")
# Version 2 : de 1 a 20
for n in range(1, 21):
    if n % 2 == 0:
        print(f"{n} est pair")
    else:
        print(f"{n} est impair")
```

La version 2 affiche alternativement pair et impair de 1 à 20. Notez le test n % 2 == 0 avec un double signe égal : c'est une comparaison, pas une affectation. Une écriture élégante de la version 2 exploite l'opérateur ternaire : print(f"{n} est {'pair' if n % 2 == 0 else 'impair'}") : la boucle ne contient plus qu'une ligne.

Ex. 8 Calculatrice à menu INTERMÉDIAIRE

Écrivez une calculatrice qui lit deux nombres et un opérateur (+, -, *, /) puis affiche le résultat. Protégez deux cas d'erreur : un opérateur inconnu et une division par zéro, qui doivent afficher un message clair au lieu de planter le programme.

Solution

Stratégie : une cascade if / elif / elif / else trie les quatre opérateurs ; le cas de la division est traité par une condition de garde testant le dénominateur avant de diviser. Le else final rattrape tout opérateur hors liste.

Solution de l'exercice 8

```python
a = float(input("Premier nombre : "))
operateur = input("Operateur (+, -, *, /) : ")
```

```
b = float(input("Deuxieme nombre : "))
if operateur == "+":
    print(f"Resultat : {a + b}")
elif operateur == "-":
    print(f"Resultat : {a - b}")
elif operateur == "*":
    print(f"Resultat : {a * b}")
elif operateur == "/":
    if b == 0:
        print("Erreur : division par zero impossible.")
    else:
        print(f"Resultat : {a / b}")
else:
    print(f"Operateur '{operateur}' inconnu.")
```

Observez l'imbrication justifiée : le test if b == 0 n'a de sens que dans la branche de la division, il vit donc à l'intérieur de cette branche. L'ordre des branches est indifférent ici car les cas sont exclusifs, contrairement à la cascade des mentions où l'ordre était déterminant. Exécution pour 8, / et 0 : « Erreur : division par zéro impossible » : le programme informe au lieu de planter, c'est déjà de la programmation défensive.

Ex. 9 Moyenne de notes avec sentinelle

INTERMÉDIAIRE

Écrivez un programme qui lit des notes au clavier jusqu'à ce que l'utilisateur saisisse -1 (la sentinelle), puis affiche le nombre de notes saisies, la moyenne et la mention obtenue. Protégez le cas où aucune note n'est saisie avant la sentinelle. Jeu de test : 12, 16, 14, -1 donnent 3 notes et une moyenne de 14.0.

Solution

Stratégie : boucle while pilotée par la sentinelle, cumul de la somme et du compteur, puis calcul de la moyenne sous condition de garde (compteur non nul) et cascade elif pour la mention. C'est la synthèse des chapitres 6 à 8.

Solution de l'exercice 9

```
somme = 0.0
compteur = 0
note = float(input("Note (-1 pour terminer) : "))
while note != -1:
    somme += note
    compteur += 1
    note = float(input("Note (-1 pour terminer) : "))
if compteur == 0:
    print("Aucune note saisie.")
else:
    moyenne = somme / compteur
    print(f"{compteur} notes, moyenne = {moyenne:.2f}")
    if moyenne >= 16:
        print("Mention Excellent")
    elif moyenne >= 14:
        print("Mention Tres bien")
    elif moyenne >= 10:
        print("Passable")
    else:
        print("Insuffisant")
```

Trace pour 12, 16, 14, -1 : trois passages dans la boucle portent la somme à 42.0 et le compteur à 3 ; la saisie de -1 sort de la boucle. La moyenne vaut 42.0 / 3 = 14.0, affichée « 14.00 » grâce au format :.2f, puis la mention « Très bien » est déclenchée par le deuxième elif. La garde if compteur == 0 est essentielle : sans elle, la division par zéro ferait planter le programme dès que l'utilisateur tape -1 immédiatement.

Ex. 10 FizzBuzz : le grand classique des entretiens DÉFI

Affichez les entiers de 1 à 100, un par ligne, avec trois exceptions : les multiples de 3 sont remplacés par « Fizz », les multiples de 5 par « Buzz », et les multiples de 15 par « FizzBuzz ». Ce problème, réputé pour filtrer les candidats en entretien d'embauche, teste la combinaison boucle + conditions + modulo + ordre des tests.

Solution

La difficulté n'est pas technique mais stratégique : le test du multiple de 15 doit venir **en premier**. Si l'on teste d'abord le multiple de 3, le nombre 15 s'arrête à « Fizz » et n'atteint jamais la branche du multiple de 15. C'est la même leçon que la cascade des mentions : du plus sélectif au plus général.

Solution de l'exercice 10

```
for n in range(1, 101):
    if n % 15 == 0:                       # multiple de 3 ET de 5 : test prioritaire
        print("FizzBuzz")
    elif n % 3 == 0:
        print("Fizz")
    elif n % 5 == 0:
        print("Buzz")
    else:
        print(n)
```

Extrait de l'exécution : 1, 2, Fizz, 4, Buzz, Fizz, 7, 8, Fizz, Buzz, 11, Fizz, 13, 14, FizzBuzz... Une variante élégante évite les répétitions en construisant le message par concaténation : message = '' ; si n est multiple de 3, message += 'Fizz' ; s'il est multiple de 5, message += 'Buzz' ; puis afficher message s'il n'est pas vide, sinon n. Cette variante élimine le cas multiple de 15 : les deux chaînes se collent naturellement. Retenez surtout la leçon d'ordonnancement des tests : elle vaut bien plus que le puzzle lui-même.

PARTIE III

Structures de données et fonctions

Un programme professionnel ne manipule pas seulement quelques variables isolées : il gère des collections de données (des listes de produits, des annuaires de clients, des ensembles de tags) et les organise en fonctions réutilisables. Cette partie vous dote des structures fondamentales de Python (listes et tuples, dictionnaires, ensembles), des compréhensions de listes, de la technique des fonctions jusqu'aux closures et décorateurs, puis de la manipulation de fichiers qui rend vos programmes persistants.

CHAPITRE 9

Les listes et les tuples

La liste est LA structure de données de Python : ordonnée, modifiable, accueillante pour tous les types. Elle réalise le tableau du chapitre 4 sans taille déclarée à l'avance, capable de grandir et de rétrécir au gré du programme. Apprenez-la à fond : elle apparaît dans presque chaque ligne de code Python professionnel.

9.1 Créer une liste et accéder à ses éléments

Une liste s'écrit entre crochets, les éléments séparés par des virgules. Chaque élément est atteignable par son **index**, écrit entre crochets, et ici vient la grande nouveauté : les index commencent à **0**, comme nous l'avions annoncé au chapitre 4. Le premier élément est liste[0], le deuxième liste[1], et le dernier liste[len(liste) - 1], où len() renvoie la longueur. Les index négatifs comptent depuis la fin : liste[-1] désigne le dernier élément, liste[-2] l'avant-dernier. Cette double numérotation est une commodité adorée des développeurs.

Code 9.1 : Création, accès et index négatifs

```python
notes = [12.5, 15, 9.75, 18, 11]
print(notes[0])        # 12.5  -> premier element (index 0)
print(notes[2])        # 9.75  -> troisieme element
print(notes[-1])       # 18    -> dernier element
print(len(notes))      # 5     -> nombre d'elements
notes[2] = 10.25       # modification d'un element par son index
print(notes)           # [12.5, 15, 10.25, 18, 11]
# Attention : index hors bornes = erreur IndexError
# print(notes[5])      -> IndexError: list index out of range
```

ATTENTION

La confusion entre position humaine et index machine est la source d'erreurs numéro un : le « cinquième » élément porte l'index 4. Mémorisez le pont entre les deux mondes : la borne haute exclue de range() (chapitre 8) et l'indexation à partir de 0 sont conçues pour s'emboîter : for i in range(len(notes)) fait visiter exactement les index valides 0, 1, 2, 3, 4.

9.2 Les méthodes essentielles des listes

Une méthode est une fonction attachée à un objet, appelée avec la syntaxe pointée objet.methode(arguments). Les listes embarquent une riche boîte à outils : ajout, suppression, tri, recherche. Certaines méthodes modifient la liste sur place (append, sort), d'autres renvoient une valeur (count, index). La différence n'est pas cosmétique : append() ne renvoie rien d'utile, écrire liste = liste.append(x) est un bug qui remplace la liste par None.

Appel	Effet	Exemple sur [12.5, 15, 9.75]
liste.append(x)	Ajoute x à la fin	append(17) -> [12.5, 15, 9.75, 17]
liste.insert(i, x)	Insère x à l'index i	insert(1, 20) -> [12.5, 20, 15, 9.75]
liste.remove(x)	Retire la première occurrence de x	remove(15) -> [12.5, 9.75]
liste.pop(i)	Retire et renvoie l'élément d'index i	pop(0) -> renvoie 12.5
liste.sort()	Trie sur place (croissant)	sort() -> [9.75, 12.5, 15]
liste.reverse()	Inverse l'ordre sur place	reverse() -> [9.75, 15, 12.5]
len(liste)	Nombre d'éléments (fonction)	len(...) -> 3
x in liste	Appartenance (opérateur)	15 in [...] -> True

Tableau 9.1 : Méthodes et fonctions indispensables des listes

9.3 Parcourir et trancher

La boucle for de Python brille ici : for element in liste visite chaque élément directement, sans index manuel. Quand l'index lui-même est utile (numéroter un affichage, modifier sur place), la fonction enumerate(liste) fournit à la fois l'index et l'élément, le duo gagnant des professionnels. Les **tranches** (slices), écrites liste[a:b], extraient une sous-liste de l'index a inclus à b exclu : même convention de borne haute exclue, encore elle. Et dès le prochain chapitre, la compréhension de listes vous donnera un raccourci encore plus puissant pour construire des listes neuves.

Code 9.2 : Parcours avec enumerate et tranches

```python
villes = ["Lome", "Cotonou", "Accra", "Dakar", "Abidjan"]
for i, ville in enumerate(villes, start=1):   # index + element
    print(f"{i}. {ville}")
print(villes[1:4])     # ['Cotonou', 'Accra', 'Dakar']  (1 inclus, 4 exclu)
print(villes[:2])      # ['Lome', 'Cotonou']  (debut implicite)
print(villes[2:])      # ['Accra', 'Dakar', 'Abidjan'] (fin implicite)
print(villes[:])       # copie integrale de la liste
# Statistiques avec les fonctions integrees
notes = [12.5, 15, 9.75, 18, 11]
```

```
print(min(notes), max(notes), sum(notes) / len(notes))   # 9.75 18 13.25
```

9.4 Les tuples : des listes scellées

Le tuple est la version figée de la liste : il s'écrit entre parenthèses et n'accepte aucune modification après création : ni ajout, ni suppression, ni réaffectation d'élément. Cette immuabilité n'est pas une punition mais une garantie : un tuple ne peut pas être corrompu par accident. On l'utilise pour les données de nature fixe (des coordonnées, une date, les dimensions d'un rectangle) et pour renvoyer plusieurs résultats depuis une fonction, comme nous le verrons au chapitre 12.

Code 9.3 : Les tuples sont immuables

```
point = (3.5, 7.2)          # tuple de deux coordonnees
print(point[0], point[1])  # 3.5 7.2   (acces en lecture autorise)
# point[0] = 5.0             # TypeError ! un tuple ne se modifie pas
x, y = point                 # unpacking : x=3.5, y=7.2
def min_max(valeurs):
    return min(valeurs), max(valeurs)   # renvoie un tuple
bas, haut = min_max([4, 9, 1, 7])
print(bas, haut)             # 1 9
```

> **À RETENIR**
>
> Une liste ordonne ses éléments entre crochets, indexés à partir de 0 et accessibles en négatif depuis la fin. append, insert, remove, pop, sort modifient la liste sur place ; len, min, max, sum et l'opérateur in sont des fonctions et opérateurs externes. enumerate fournit index et élément, les tranches [a:b] extraient avec borne haute exclue. Le tuple est la liste immuable, parfaite pour les données fixes et les retours multiples.

CHAPITRE 10

La compréhension de listes

Les chapitres précédents vous ont fait écrire le même motif des dizaines de fois : créer une liste vide, la remplir dans une boucle for, éventuellement filtrer avec un if. Python a condensé ce motif en une syntaxe de prédilection, la **compréhension de listes** : une expression entre crochets qui construit la liste complète en une seule ligne lisible. Née de la notation ensembliste des mathématiciens (« l'ensemble des n au carré pour n allant de 1 à 5 »), elle est devenue la signature stylistique du code Python professionnel.

10.1 De la boucle à la compréhension

Comparons les deux écritures sur un exemple complet. Objectif : partir de nombres quelconques et construire la liste des positifs, chacun doublé. La version boucle occupe quatre lignes et mobilise une variable auxiliaire ; la compréhension exprime exactement la même idée, mais dans l'ordre naturel de la pensée : « je veux n fois 2, pour chaque n de la liste, si n est positif ».

Code 10.1 : Le même traitement, boucle puis compréhension

```
nombres = [3, -7, 12, -5, 8, -1]
# Version 1 : la boucle classique des chapitres precedents
positifs_doubles = []
for n in nombres:
    if n > 0:
        positifs_doubles.append(n * 2)
print(positifs_doubles)         # [6, 24, 16]
# Version 2 : la comprehension de listes, en une ligne
positifs_doubles = [n * 2 for n in nombres if n > 0]
print(positifs_doubles)         # [6, 24, 16]
```

Les deux versions produisent exactement la même liste et s'exécutent à la même vitesse. La compréhension se lit de gauche à droite : d'abord l'expression qui fabrique chaque élément (n * 2), ensuite la clause for qui parcourt la source (for n in nombres), enfin la clause if facultative qui filtre (if n > 0). Une fois ces trois zones identifiées, toute compréhension devient transparente.

10.2 La syntaxe générale et ses trois usages

Le gabarit complet est : [expression for variable in iterable if condition]. La condition est facultative ; l'expression, elle, est obligatoire. Trois usages couvrent la quasi-totalité des besoins : la **transformation** (exprimer chaque élément), le **filtre** (ne garder qu'une partie des éléments) et la **sélection conditionnelle**, qui utilise l'expression ternaire du chapitre 7 pour choisir la valeur produite, et non plus pour écarter des éléments.

Code 10.2 : Transformer, filtrer, choisir

```
cubes = [n ** 3 for n in range(1, 6)]
print(cubes)          # [1, 8, 27, 64, 125]     (transformation)
notes = [12.5, 8, 15, 9.5, 17]
admises = [n for n in notes if n >= 10]
print(admises)        # [12.5, 15, 17]          (filtrage)
verdicts = ["admis" if n >= 10 else "ajourne" for n in notes]
print(verdicts)       # un verdict par note     (choix)
```

ATTENTION

Ne confondez pas les deux positions du if. Écrit à la fin (for n in notes if n >= 10), il **filtre** : seuls les éléments retenus produisent un résultat. Écrit dans l'expression ("admis" if n >= 10 else "ajourne"), c'est un ternaire qui **choisit** la valeur produite pour chaque élément : aucun n'est écarté, tous sont transformés. Une compréhension peut combiner les deux, mais c'est déjà le signe qu'une boucle classique serait plus lisible.

10.3 Compréhensions en situation réelle

Voyez la compréhension à l'œuvre sur les objets de tous les jours du programmeur : nettoyer des prénoms, extraire des valeurs, apparier des données en paires. Le dernier exemple montre que la source peut être une chaîne : la clause for visite ses caractères un à un, exactement comme la boucle du chapitre 8.

Code 10.3 : Nettoyer, extraire, apparier

```
prenoms = ["awa", "koffi", "roland"]
propres = [p.capitalize() for p in prenoms]
print(propres)          # ['Awa', 'Koffi', 'Roland']
longs = [p for p in prenoms if len(p) > 4]
print(longs)            # ['koffi', 'roland']
paires = [(n, n * n) for n in range(1, 4)]
print(paires)           # [(1, 1), (2, 4), (3, 9)]
# Sur une chaine : garder les consonnes d'un mot
voyelles = "aeiouyAEIOUY"
consonnes = [c for c in "Algorithmique" if c not in voyelles]
print("".join(consonnes))    # lgrthmq
```

10.4 Les compréhensions imbriquées

Une compréhension peut contenir plusieurs clauses for : elles s'enchaînent de gauche à droite, exactement comme des boucles imbriquées, la seconde étant refaite entièrement à chaque tour de la première. Le premier usage classique est l'**aplatissement** d'une liste de listes ; le second est le produit croisé entre deux collections, qui fabrique toutes les combinaisons possibles.

Code 10.4 : Aplatir une matrice, produire des combinaisons

```
matrice = [[1, 2, 3], [4, 5, 6]]
aplatie = [valeur for ligne in matrice for valeur in ligne]
print(aplatie)          # [1, 2, 3, 4, 5, 6]
couleurs = ["rouge", "vert"]
tailles = ["S", "M"]
variantes = [(c, t) for c in couleurs for t in tailles]
print(variantes)
# [('rouge', 'S'), ('rouge', 'M'), ('vert', 'S'), ('vert', 'M')]
```

10.5 Compréhensions de dictionnaires, d'ensembles et générateurs

L'idée s'étend aux autres structures : des accolades produisent une compréhension de **dictionnaire** (avec clé: valeur) ou d'**ensemble** (sans deux-points), et des parenthèses produisent une **expression génératrice**, sœur paresseuse de la compréhension : elle ne construit aucune liste et livre les valeurs une à une, ce qui est parfait pour les fonctions comme sum() ou max().

Code 10.5 : Les trois sœurs : dict, set et générateur

```
notes = {"Awa": 14, "Koffi": 8, "Roland": 17}
# Comprehension de dictionnaire : ne garder que les admis
admises = {nom: note for nom, note in notes.items() if note >= 10}
```

```
print(admises)          # {'Awa': 14, 'Roland': 17}
# Comprehension d'ensemble : les voyelles utilisees, sans doublon
voyelles = {c for c in "programmation orientee" if c in "aeiouy"}
print(voyelles)         # {'a', 'e', 'i', 'o'}  (ordre libre)
# Expression generatrice : aucune liste intermediaire
total = sum(n ** 2 for n in range(1, 5))
print(total)            # 30   (1 + 4 + 9 + 16)
```

Vous avez déjà croisé cette dernière forme sans la nommer : la méthode fortune() du mini-projet final écrit sum(c.solde for c in self.comptes). L'expression génératrice évite de bâtir une liste complète quand seuls le total ou l'extremum sont recherchés : sur de grandes données, c'est une économie de mémoire réelle.

ASTUCE

La compréhension n'est pas une obligation : c'est un outil de lisibilité. Dès que la logique demande plus d'une clause for, un if complexe ou un effet de bord (print, compteur), revenez à la boucle classique : un code que vos camarades comprennent en dix secondes vaut mieux qu'une ligne brillante de cent caractères. Et rappelez-vous qu'une compréhension construit une **nouvelle** liste : elle ne modifie jamais la liste source.

À RETENIR

[expression for x in iterable if condition] construit une nouvelle liste : expression fabrique chaque élément, for parcourt, if filtre (facultatif). Le ternaire placé dans l'expression choisit la valeur au lieu de filtrer. Plusieurs clauses for s'imbriquent comme des boucles. Les accolades donnent des compréhensions de dictionnaires et d'ensembles, les parenthèses des expressions génératrices économes en mémoire.

CHAPITRE 11

Dictionnaires et ensembles

La liste range ses données par position ; mais que faire quand la donnée utile est un **nom** ? Retrouver le numéro de M. HOUNKPE dans un annuaire, le prix de l'article « riz », la note de l'étudiant « Awa » : voilà le travail du dictionnaire. Le dictionnaire associe des **clés** uniques à des **valeurs**, comme un mot associe une définition. L'ensemble, lui, range des valeurs uniques sans ordre ni doublon : la structure idéale pour les tests d'appartenance et le dédoublonnage.

11.1 Créer et manipuler un dictionnaire

Un dictionnaire s'écrit entre accolades, chaque entrée de la forme clé: valeur. Les clés sont généralement des chaînes ou des nombres ; les valeurs peuvent être de n'importe quel type (y compris des listes ou d'autres dictionnaires). L'accès se fait par la clé entre crochets ; la réaffectation crée la clé si elle n'existe pas : c'est ainsi qu'on ajoute des entrées. La méthode get() propose un accès souple qui renvoie une valeur par défaut au lieu d'une erreur lorsque la clé est absente.

Code 11.1 : Fiche étudiante dans un dictionnaire

```
etudiant = {
    "nom": "Awa Koffi",
    "filiere": "Informatique",
    "notes": [14, 16.5, 11],
    "majeur": True,
}
print(etudiant["nom"])               # Awa Koffi
etudiant["age"] = 21                 # ajout d'une nouvelle entree
etudiant["filiere"] = "Data Science" # modification d'une entree existante
print(etudiant.get("telephone", "non renseigne"))   # non renseigne
# Parcours complet : cles, valeurs, ou les deux
for cle in etudiant:                 # par defaut : les cles
    print(cle, "->", etudiant[cle])
for cle, valeur in etudiant.items(): # items() fournit les paires
    print(f"{cle} : {valeur}")
```

11.2 Les méthodes du dictionnaire

Appel	Effet	Retour
d[k]	Accès par clé (erreur si absente)	la valeur
d.get(k, defaut)	Accès souple avec valeur par défaut	la valeur ou defaut
d[k] = v	Ajout ou modification	(aucun)
del d[k]	Suppression d'une entrée	(aucun)
d.pop(k)	Suppression et récupération	la valeur retirée
k in d	Test d'existence de la clé	True / False
d.keys()	Vue des clés	itérable des clés
d.values()	Vue des valeurs	itérable des valeurs
d.items()	Vue des paires (clé, valeur)	itérable de tuples

Tableau 11.1 : Méthodes essentielles des dictionnaires

ATTENTION

Deux tests ne doivent pas être confondus : « k in d » teste l'existence d'une **clé**, pas d'une valeur. Pour chercher une valeur, il faut passer par d.values() : "Awa Koffi" in etudiant vaut False, mais "Awa Koffi" in etudiant.values() vaut True. De même, accéder à d[k] sur une clé absente lève une KeyError : préférez d.get(k) quand l'absence est possible.

11.3 Les ensembles : l'unicité garantie

Un ensemble (set) s'écrit entre accolades comme un dictionnaire sans valeurs : {"riz", "huile", "sucre"}. Il refuse les doublons (ajouter un élément déjà présent ne change rien) et n'offre ni ordre ni indexation. Ses forces : le dédoublonnage instantané d'une liste via set(liste), et les opérations mathématiques d'ensembles : union |, intersection &, différence -. Le test d'appartenance x in ensemble est également très rapide, même sur de grandes collections.

Code 11.2 : Dédoublonnage et opérations ensemblistes

```python
inscrits = ["awa", "koffi", "roland", "awa", "koffi"]
uniques = set(inscrits)
print(uniques)                  # {'roland', 'koffi', 'awa'}  (doublons fondus)
groupe_a = {"awa", "koffi", "djibril"}
groupe_b = {"koffi", "roland"}
print(groupe_a | groupe_b)      # union : tous les membres
print(groupe_a & groupe_b)      # intersection : {'koffi'}
print(groupe_a - groupe_b)      # difference : {'awa', 'djibril'}
```

À RETENIR

Le dictionnaire mappe des clés uniques vers des valeurs : accès par d[k], ajout par affectation, accès souple par get(), parcours par keys(), values() et items(). L'ensemble range des valeurs uniques sans ordre : set(liste) dédoublonne, | & - opèrent comme en mathématiques. Choisir la bonne structure : position -> liste ; étiquette nommée -> dictionnaire ; unicité -> ensemble.

CHAPITRE 12

Les fonctions en Python

Le chapitre 4 vous a fait écrire des fonctions en pseudo-code ; Python les réalise avec le mot-clé def. Le bénéfice est le même (écrire une fois, réutiliser partout), mais Python ajoute une gamme de mécanismes élégants : paramètres par défaut, arguments nommés, retour multiple, fonctions anonymes et paramètres illimités. La fonction est aussi l'unité de test et de maintenance : un code bien découpé en fonctions se corrige pièce par pièce.

12.1 Définir, appeler, retourner

Une fonction se définit par def nom(parametres): suivi du corps indenté. L'instruction return renvoie le résultat et sort immédiatement de la fonction. Une fonction sans return renvoie None, le « rien » de Python. La même fonction peut renvoyer des types différents selon les cas, mais la bonne pratique est de garder un type de retour cohérent : c'est ce qui rend une fonction prévisible.

Code 12.1 : Une fonction documentée

```python
def moyenne_ponderee(n1, n2, coef1=2, coef2=3):
    """Calcule la moyenne ponderee de deux notes.
    coef1 et coef2 ont des valeurs par defaut : ils sont optionnels.
    """
    return (n1 * coef1 + n2 * coef2) / (coef1 + coef2)
# Trois appels legaux
print(moyenne_ponderee(12.5, 15))              # 14.0  (defauts 2 et 3)
print(moyenne_ponderee(12.5, 15, 1, 1))        # 13.75 (par position)
print(moyenne_ponderee(12.5, 15, coef2=1))     # 13.75 (argument nomme)
```

12.2 Paramètres par défaut et arguments nommés

Un paramètre peut recevoir une valeur par défaut : il devient optionnel lors de l'appel. À l'appel, les arguments peuvent être passés **par position** (l'ordre compte) ou **par nom**, comme moyenne_ponderee(12.5, 15, coef2=1) : l'ordre devient alors libre et le code s'auto-documente. La règle syntaxique : les paramètres sans défaut viennent toujours avant ceux avec défaut. Cette technique réduit les erreurs d'ordre et permet de construire des fonctions à la fois simples d'usage et flexibles.

12.3 La portée des variables : local et global

Une variable créée à l'intérieur d'une fonction est **locale** : elle naît à l'appel et disparaît au retour, invisible depuis l'extérieur. Une variable créée au niveau du module est **globale** : visible partout, mais une fonction qui veut la réaffecter doit la déclarer global, une pratique déconseillée. La bonne architecture fait passer toutes les données par les paramètres et ramène les résultats par return : chaque fonction devient alors une boîte noire testable indépendamment, sans effet de bord caché.

Code 12.2 : Portée locale et globale

```
compteur_global = 0            # variable globale (niveau module)
def incrementer():
    local = 1                  # variable locale a la fonction
    global compteur_global     # autorise la REAFFECTATION de la globale
    compteur_global += 1
    return local
incrementer()
print(compteur_global)         # 1  (la globale a change)
# print(local)                 # NameError : local n'existe plus ici
```

ATTENTION

Le mot-clé global est un aveu de conception fragile : une fonction qui modifie les variables du module devient impossible à raisonner isolément. Exception tolérée : les constantes partagées en lecture (TAUX_TVA = 0.18). Dans tous les autres cas, préférez les paramètres d'entrée et le return, même lorsqu'il faut passer cinq paramètres : cinq paramètres visibles valent mieux qu'un effet de bord caché.

12.4 Retour multiple, *args et **kwargs

Le return peut renvoyer plusieurs valeurs en les séparant par des virgules : Python les emballe dans un tuple (chapitre 9) que l'appelant déballe en variables distinctes. Les paramètres spéciaux *args et **kwargs, eux, accueillent un nombre libre d'arguments : *args ramasse les arguments positionnels excédentaires dans un tuple, **kwargs ramasse les arguments nommés excédentaires dans un dictionnaire. Vous les croiserez partout dans les bibliothèques professionnelles : ils permettent d'écrire des fonctions passe-partout qui transmettent tous leurs arguments à une autre fonction, comme le fera le décorateur du chapitre 13.

Code 12.3 : Retour multiple et tri par critère

```python
def bilan(notes):
    """Renvoie (moyenne, maximum, minimum) d'une liste."""
    return sum(notes) / len(notes), max(notes), min(notes)
moy, haut, bas = bilan([14, 16.5, 11, 18])
print(f"Moyenne {moy:.2f} | Max {haut} | Min {bas}")
noms = ["roland", "awa", "koffi"]
noms_tries = sorted(noms, key=lambda n: len(n))   # tri par longueur
print(noms_tries)          # ['awa', 'koffi', 'roland']
```

Code 12.4 : Une fonction au nombre d'arguments libre

```python
def rapport(titre, *valeurs, **precisions):
    """Accepte n'importe quels arguments supplementaires."""
    print(f"== {titre} ==")
    print(f"Valeurs recues : {valeurs}")           # tuple
    print(f"Precisions : {precisions}")            # dictionnaire
rapport("Classe A", 14, 16.5, 11, effectif=32, salle="B12")
# == Classe A ==
# Valeurs recues : (14, 16.5, 11)
# Precisions : {'effectif': 32, 'salle': 'B12'}
```

12.5 map() et filter() : les vétérans fonctionnels

Avant que les compréhensions du chapitre 10 ne deviennent le style dominant, on transformait les listes avec **map(fonction, sequence)** et on les filtrait avec **filter(condition, sequence)**. Ces vétérans restent vivants dans le code historique et dans certaines équipes : map applique sa fonction à chaque élément, filter ne garde que ceux où la condition est vraie. Les deux renvoient des itérateurs paresseux, comme les expressions génératrices : entourez-les de list() pour voir leur contenu. La recommandation actuelle est de leur préférer les compréhensions, plus lisibles ; gardez map et filter en lecture passive pour comprendre le code des autres.

Code 12.5 : map et filter face aux compréhensions

```python
noms = ["awa", "koffi", "roland"]
# Style veteran : map + lambda
propres = list(map(lambda n: n.capitalize(), noms))
print(propres)          # ['Awa', 'Koffi', 'Roland']
# Style veteran : filter + lambda
longs = list(filter(lambda n: len(n) > 4, noms))
print(longs)            # ['koffi', 'roland']
# Style moderne recommande : comprehensions (chapitre 10)
propres = [n.capitalize() for n in noms]
longs = [n for n in noms if len(n) > 4]
```

À RETENIR

def nom(parametres): définit une fonction ; return renvoie le résultat (sans return, la fonction renvoie None). Les paramètres par défaut rendent des arguments optionnels, les arguments nommés clarifient les appels, le retour multiple s'appuie sur les tuples et *args / **kwargs accueillent un nombre libre d'arguments. Les variables locales vivent et meurent dans la fonction : faites transiter les données par paramètres et valeurs de retour, et bannissez global.

CHAPITRE 13

Portée approfondie, closures et décorateurs

Le chapitre 12 vous a appris qu'une variable locale naît à l'appel d'une fonction et disparaît au retour. Ce chapitre pousse la porte du grenier : que se passe-t-il quand une fonction se définit **à l'intérieur** d'une autre ? Python répond par trois mécanismes d'une élégance redoutable : la règle de résolution des noms, les **closures** et les **décorateurs**. Au bout du voyage, vous comprendrez ce qui se cache réellement derrière le @property du chapitre 16.

13.1 Les fonctions imbriquées et la règle LEGB

Python autorise la définition d'une fonction dans le corps d'une autre. La fonction interne n'existe que pour sa conteneur : l'extérieur ne peut pas l'appeler directement. Elle hérite en revanche d'un pouvoir précieux : elle peut **lire** les variables de la fonction qui l'entoure. C'est la règle LEGB : pour résoudre un nom, Python regarde d'abord la portée **L**ocale (la fonction interne), puis la portée **E**nglobante (la fonction externe), puis la portée **G**lobale (le module), enfin les **B**uilt-ins (les fonctions intégrées comme print).

Code 13.1 : Une fonction qui lit la portée englobante

```python
def externe():
    message = "je viens de la portee englobante"    # portee E
    def interne():                                   # portee L
        print(message)     # lecture : LEGB remonte de L vers E
    interne()
externe()       # je viens de la portee englobante
# interne()     # NameError : interne n'existe pas en dehors de externe
```

13.2 Le mot-clé nonlocal

Lire la variable englobante est libre ; la **réaffecter** exige une déclaration, exactement comme global l'exigeait au niveau module. Le mot-clé **nonlocal** proclame : « ce nom désigne la variable de la fonction englobante, pas une nouvelle variable locale ». C'est la clé qui permet à une fonction interne de tenir un état durable.

Code 13.2 : nonlocal autorise la modification

```python
def fabrique_compteur():
    total = 0                        # variable englobante
    def incrementer():
        nonlocal total               # total = celui de la fabrique
        total += 1
        return total
    return incrementer
compter = fabrique_compteur()
print(compter())     # 1
print(compter())     # 2   : total a SURVECUT au retour de la fabrique !
print(compter())     # 3
```

13.3 Les closures : des fonctions qui se souviennent

Regardez le code 13.2 avec attention : la fabrique est **terminée** depuis le premier appel, pourtant total continue d'exister et de progresser. Ce miracle s'appelle une **closure** (fermeture) : quand une fonction interne capture des variables de sa portée englobante, Python les range dans un petit bagage attaché à la fonction elle-même. La fonction voyage avec ses variables : elles vivent tant qu'elle vit. On dit que la fonction « ferme » sur ses variables capturées.

Code 13.3 : Fabrique de multiplicateurs

```
def fabrique_multiplicateur(coefficient):
    def multiplier(x):
        return x * coefficient       # coefficient est capture
    return multiplier
double = fabrique_multiplicateur(2)   # une closure, coefficient vaut 2
triple = fabrique_multiplicateur(3)   # une autre, coefficient vaut 3
print(double(21))       # 42
print(triple(14))       # 42
print(double.__closure__[0].cell_contents)   # 2 : le bagage capture
```

Chaque appel à fabrique_multiplicateur crée une closure indépendante : double et triple coexistent sans se gêner, chacun avec son coefficient figé au moment de la création. Cette technique de **fabrique de fonctions** permet de préparer toute une famille de fonctions paramétrées sans aucune variable globale. Inspectez double.__closure__ si la curiosité vous prend : Python y conserve littéralement les valeurs capturées, dans des objets appelés cellules.

ATTENTION

Piège célèbre de la late binding : une closure ne capture pas la **valeur** d'une variable, elle capture la **variable** elle-même. Si la variable change après la création de la closure, la closure voit la nouvelle valeur. Évitez donc de construire des closures dans une boucle en captant le compteur de la boucle : toutes verraient sa valeur finale. Capturez une copie via un paramètre par défaut (def f(i=i):) ou construisez vos closures dans une fabrique, comme au code 13.3.

13.4 Les décorateurs : la closure devenue syntaxe

Un **décorateur** est une fonction qui reçoit une fonction en argument et renvoie une fonction enrichie, construite par closure. La syntaxe @nom au-dessus d'un def est un simple raccourci : @encadre au-dessus de def bonjour()

vaut exactement bonjour = encadre(bonjour). Les décorateurs ajoutent à une fonction des services transversaux (journalisation, comptage, contrôle d'accès) sans toucher une ligne de son corps. Vous en connaissez déjà trois : @property, @classmethod et @staticmethod du chapitre 16 !

Code 13.4 : Écrire son premier décorateur

```
def compte_appels(func):
    appels = 0                        # l'etat vit dans la closure
    def enveloppee(*args, **kwargs):
        nonlocal appels
        appels += 1
        print(f"[Appel no {appels} de {func.__name__}]")
        return func(*args, **kwargs)  # delegation a la fonction d'origine
    return enveloppee

@compte_appels                        # saluer = compte_appels(saluer)
def saluer(nom):
    return f"Bonjour {nom} !"

print(saluer("Awa"))     # [Appel no 1 de saluer]  puis Bonjour Awa !
print(saluer("Koffi"))   # [Appel no 2 de saluer]  puis Bonjour Koffi !
```

Suivez la mécanique : @compte_appels enveloppe saluer dans la closure enveloppee, qui mémorise à la fois le compteur appels (via nonlocal) et la fonction originale func. À chaque appel, elle journalise, incrémente, puis délègue. La fonction saluer, elle, n'a pas changé d'une ligne : la fonctionnalité de comptage s'est ajoutée par-dessus, comme un gant. C'est ce principe qui anime @property : le getter que vous écrivez est enveloppé dans un mécanisme qui le branche sur l'accès aux attributs.

Code 13.5 : Un décorateur d'encadrement, pas à pas

```
def encadre(func):
    def enveloppee():
        print("=" * 28)
        func()
        print("=" * 28)
    return enveloppee

@encadre
def bonjour():
    print("Bonjour la classe !")

bonjour()
# ============================
# Bonjour la classe !
# ============================
# (les deux lignes d'egal viennent du decorateur)
```

ASTUCE

Quand vous écrivez un décorateur sérieux, la fonction enveloppee masque l'identité de la fonction d'origine : saluer.__name__ vaudrait enveloppee ! Le module standard functools fournit le décorateur @functools.wraps(func) à poser sur enveloppee pour recopier le nom, la documentation et les autres métadonnées. C'est une marque de polish professionnel, indispensable dès que vos outils (débogueur, documentation automatique) lisent ces informations.

À RETENIR

Une fonction définie dans une autre lit librement ses variables (règle LEGB : Local, Englobante, Globale, Built-ins) ; nonlocal autorise leur modification. Une closure est une fonction interne qui emporte les variables capturées dans son bagage : elles survivent à l'appel de la fabrique. Un décorateur @nom remplace la fonction par une version enrichie via closure : c'est la syntaxe derrière @property, @classmethod et @staticmethod.

CHAPITRE 14

Chaînes de caractères et fichiers

Le texte est le format universel des échanges : noms, messages, adresses, contenus de fichiers. Ce chapitre complète votre maîtrise des chaînes, objets riches en méthodes de nettoyage, de découpe et de recherche, puis vous fait franchir un seuil décisif : faire survivre vos données après la fermeture du programme grâce aux fichiers.

14.1 Les méthodes des chaînes

Une chaîne Python est une séquence immuable : on peut la lire caractère par caractère (mot[0], mot[-1], tranches), mais jamais modifier une lettre sur place. Toutes les méthodes de transformation renvoient donc une **nouvelle chaîne**, laissant l'originale intacte. Il faut donc récupérer le résultat : mot = mot.upper() et non mot.upper() seul.

Appel	Effet	Exemple
s.upper() / s.lower()	Majuscules / minuscules	"PyThOn".upper() -> "PYTHON"
s.strip()	Retire espaces de bord	" hi ".strip() -> "hi"
s.replace(a, b)	Remplace a par b	"12-05".replace("-", "/") -> "12/05"
s.split(sep)	Découpe en liste	'a;b;c'.split(';') -> ['a','b','c']
sep.join(liste)	Recolle une liste	'-'.join(['x','y']) -> 'x-y'
s.find(sous)	Index de la 1re occurrence (-1 si absente)	"info".find("f") -> 2
s.startswith(p)	Test de préfixe	"python".startswith("py") -> True
s.isdigit()	Test chiffres uniquement	'2026'.isdigit() -> True

Tableau 14.1 : Méthodes de chaînes à connaître par cœur

14.2 Formater les nombres dans les f-strings

Les f-strings du chapitre 6 cachent un mini-langage de formatage après les deux-points. Les plus utiles pour un développeur : :.2f pour arrondir à deux décimales, le séparateur de milliers :, pour les grands montants, et le remplissage par des zéros :05d pour les codes numériques. Ce sont ces détails qui distinguent un affichage d'amateur d'un état professionnel.

Code 14.1 : Formatage professionnel des nombres

```python
montant = 4517862.4271
code = 42
print(f"Montant : {montant:,.2f} FCFA")   # 4,517,862.43 FCFA
print(f"Code d'inscription : {code:05d}") # 00042
print(f"Pourcentage : {0.8573:.1%}")      # 85.7%
```

14.3 Lire et écrire des fichiers

La fonction open(chemin, mode) ouvre un fichier selon un mode : "r" pour lire (défaut), "w" pour écrire en écrasant, "a" pour ajouter à la fin. La bonne pratique impose la forme **with open(...) as f** : elle ferme automatiquement le fichier même en cas d'erreur (sans elle, un fichier oublié ouvert reste verrouillé et consomme des ressources). Les chaînes lues ou écrites se terminent par \n : la méthode strip() nettoie ces fins de ligne.

Code 14.2 : Écrire puis relire un fichier de notes

```python
# Ecriture (mode "w" : ecrase le contenu precedent)
with open("notes.txt", "w", encoding="utf-8") as f:
    f.write("Awa;14\n")
    f.write("Koffi;16.5\n")
    f.write("Roland;12\n")
# Lecture ligne a ligne
with open("notes.txt", "r", encoding="utf-8") as f:
    total = 0
    nombre = 0
    for ligne in f:
        ligne = ligne.strip()                    # retire le \n de fin
        nom, note = ligne.split(";")             # decoupe en deux morceaux
        print(f"{nom} a obtenu {note}")
        total += float(note)
        nombre += 1
    print(f"Moyenne de la classe : {total / nombre:.2f}")
```

ATTENTION

Le mode "w" écrase instantanément le contenu existant : si vous ouvrez notes.txt en mode "w" croyant « ajouter », tout l'historique est détruit avant même la première écriture. Pour prolonger un fichier, utilisez le mode "a" (append). Deuxième piège : oubliez encoding="utf-8" et vos accents peuvent devenir des caractères bizarres selon la machine qui relit le fichier.

À RETENIR

Les chaînes sont immuables : leurs méthodes renvoient de nouvelles chaînes qu'il faut récupérer. split() découpe, join() recolle, strip() nettoie, find() cherche. Les f-strings formatent :.2f, :, et :05d. Les fichiers s'ouvrent toujours avec with open(chemin, mode, encoding="utf-8") : "w" crée ou écrase, "a" ajoute, "r" lit ligne à ligne.

Exercices corrigés de la Partie III

Ces exercices mobilisent l'ensemble de la partie : listes, dictionnaires, fonctions, compréhensions et closures. Écrivez d'abord vos propres fonctions, testez-les avec les jeux de données, puis comparez avec les corrections commentées.

Ex. 11 Gestion d'une liste de courses

BASIQUE

Créez un programme avec une liste initiale ["riz", "huile"]. Ajoutez "sucre" à la fin, insérez "pain" en tête de liste, retirez "huile", puis affichez la liste triée alphabétiquement et sa longueur. Résultat attendu final : ['pain', 'riz', 'sucre'] et 3.

Solution

Stratégie : enchaîner les méthodes du tableau 9.1. Le point de vigilance est que sort() trie sur place et renvoie None : il ne faut donc PAS écrire courses = courses.sort().

Solution de l'exercice 11

```python
courses = ["riz", "huile"]
courses.append("sucre")        # ajout a la fin
courses.insert(0, "pain")      # insertion en tete (index 0)
courses.remove("huile")        # retrait par valeur
courses.sort()                 # tri alphabetique sur place
print(courses)                 # ['pain', 'riz', 'sucre']
print(f"{len(courses)} articles")   # 3 articles
```

Trace pas à pas : après append, la liste vaut ['riz', 'huile', 'sucre'] ; après insert(0, 'pain'), ['pain', 'riz', 'huile', 'sucre'] ; remove retire la première occurrence de 'huile' ; le tri final donne ['pain', 'riz', 'sucre']. Si un retrait peut concerner un article absent, la bonne pratique consiste à tester avant : if article in courses: courses.remove(article).

Ex. 12 Compteur de voyelles BASIQUE

Écrivez une fonction compter_voyelles(phrase) qui renvoie le nombre de voyelles (a, e, i, o, u, y, majuscules comprises) d'une phrase. Exemple : compter_voyelles("Bonjour le Monde") doit renvoyer 6. Indice : l'opérateur in fonctionne aussi sur les chaînes.

Solution

Stratégie : normaliser la phrase en minuscules pour neutraliser les majuscules, puis parcourir chaque caractère et compter ceux qui appartiennent à la chaîne de référence 'aeiouy'. Le test caractere in voyelles exprime l'appartenance en une seule lecture.

Solution de l'exercice 12

```python
def compter_voyelles(phrase):
    """Renvoie le nombre de voyelles de la phrase donnee."""
    voyelles = "aeiouy"
    total = 0
    for caractere in phrase.lower():     # normalisation en minuscules
        if caractere in voyelles:
            total += 1
    return total
print(compter_voyelles("Bonjour le Monde"))    # 6
print(compter_voyelles("XYZ"))                 # 1 (le y compte)
```

Vérification du premier appel : B-o-n-j-o-u-r contient o, o, u ; l-e contient e ; M-o-n-d-e contient o, e. Total : 3 + 1 + 2 = 6. La version compacte avec un sum généreux, sum(1 for c in phrase.lower() if c in voyelles), produit le même résultat en une ligne, mais la version en boucle reste la plus lisible pour l'apprentissage. La fonction étant paramétrée, elle se réutilise sur n'importe quel texte sans retoucher une ligne.

Ex. 13 Carnet de notes par étudiant INTERMÉDIAIRE

À partir du dictionnaire notes = {"Awa": [14, 16.5, 11], "Koffi": [9, 12, 15], "Roland": [18, 17, 19]}, écrivez un programme qui affiche, pour chaque étudiant, sa moyenne, et qui détermine ensuite le major de la classe (la meilleure moyenne) avec son nom. Résultats attendus : moyennes 13.83, 12.00, 18.00 et major Roland.

Solution

Stratégie : parcours du dictionnaire avec items() pour calculer chaque moyenne à la volée, puis recherche du maximum par candidat courant, la technique de l'algorithme 4.1, transposée des listes aux clés d'un dictionnaire.

Solution de l'exercice 13

```python
notes = {"Awa": [14, 16.5, 11], "Koffi": [9, 12, 15], "Roland": [18, 17, 19]}
# Etape 1 : moyenne de chacun
```

```
for nom, liste_notes in notes.items():
    moyenne = sum(liste_notes) / len(liste_notes)
    print(f"{nom} : {moyenne:.2f}")
# Etape 2 : recherche du major par candidat courant
major_nom = None
major_moyenne = -1
for nom, liste_notes in notes.items():
    moyenne = sum(liste_notes) / len(liste_notes)
    if moyenne > major_moyenne:
        major_moyenne = moyenne
        major_nom = nom
print(f"Major de la classe : {major_nom} avec {major_moyenne:.2f}")
```

Le major démarre à -1 : toute moyenne réelle le dépasse, donc le premier étudiant devient automatiquement le candidat. Solution alternative plus expérimentée : construire un dictionnaire des moyennes par compréhension {nom: sum(n) / len(n) for nom, n in notes.items()} puis appliquer max() avec key=moyennes.get, deux lignes au lieu de huit. Les deux chemins sont corrects : le premier montre la mécanique, le second montre la maturité.

Ex. 14 Fonction de statistiques avec validation

INTERMÉDIAIRE

Écrivez une fonction statistiques(valeurs) qui renvoie un tuple (minimum, maximum, moyenne). Si la liste est vide, la fonction doit renvoyer None au lieu de planter. Testez avec [4, 9, 1, 7] (attendu (1, 9, 5.25)) puis avec [] (attendu None).

Solution

Stratégie : garde en tête (cas liste vide), calcul via les fonctions intégrées min, max et sum/len, retour multiple en tuple que le testeur déballe. C'est l'exercice de la boîte noire parfaite : entrée, validation, sortie.

Solution de l'exercice 14

```
def statistiques(valeurs):
    """Renvoie (minimum, maximum, moyenne), ou None si liste vide."""
    if len(valeurs) == 0:
        return None
    mini = min(valeurs)
    maxi = max(valeurs)
    moyenne = sum(valeurs) / len(valeurs)
    return mini, maxi, moyenne
resultat = statistiques([4, 9, 1, 7])
print(resultat)                 # (1, 9, 5.25)
print(statistiques([]))         # None
# Le tuple retourne se deballe en variables :
# mini, maxi, moy = statistiques([4, 9, 1, 7])
```

Sans la garde, statistiques([]) lèverait ZeroDivisionError sur sum([]) / 0 : min([]) et max([]) lèvent d'ailleurs déjà ValueError sur liste vide. La validation en tête de fonction est un réflexe professionnel : on traite d'abord les cas limites, puis le cas général. Ce patron, if cas_degrade: return valeur_par_defaut, s'appelle la clause de garde (early return) et évite d'indenter tout le corps du calcul dans un else.

Ex. 15 Compréhensions et fabrique de multiplicateurs

INTERMÉDIAIRE

Partie A : à partir de la liste prix = [2500, 12000, 8000, 15000, 950], construisez en une ligne chacune : la liste des prix TTC (TVA de 18 %, arrondis à deux décimales avec round), la liste des seuls prix supérieurs ou égaux à 10000, et la liste des textes « 2500 F » prêts à afficher. Partie B : écrivez une fonction fabrique_multiplicateur(coefficient) qui renvoie une closure multipliant son argument par coefficient. Créez double et moitie, puis vérifiez : double(21) vaut 42 et moitie(9) vaut 4.5.

Solution

Stratégie : la partie A s'écrit entièrement par compréhensions, une par objectif, chacune en une ligne lisible ; la partie B est la closure du chapitre 13 appliquée telle quelle : la fabrique capture coefficient, et chaque fonction créée conserve sa propre copie capturée.

Solution de l'exercice 15 : compréhensions

```python
prix = [2500, 12000, 8000, 15000, 950]
ttc = [round(p * 1.18, 2) for p in prix]
print(ttc)           # [2950.0, 14160.0, 9440.0, 17700.0, 1121.0]
grands = [p for p in prix if p >= 10000]
print(grands)        # [12000, 15000]
etiquettes = [f"{p} F" for p in prix]
print(etiquettes)    # ['2500 F', '12000 F', '8000 F', '15000 F', '950 F']
```

Solution de l'exercice 15 : fabrique de multiplicateurs

```python
def fabrique_multiplicateur(coefficient):
    def multiplier(x):
        return x * coefficient       # coefficient est capture ici
    return multiplier
double = fabrique_multiplicateur(2)
moitie = fabrique_multiplicateur(0.5)
print(double(21))      # 42
print(moitie(9))       # 4.5
```

Vérification de la partie A : 2500 × 1.18 = 2950.0, 12000 × 1.18 = 14160.0 et 950 × 1.18 = 1121.0. Pour la partie B, double et moitie coexistent sans se gêner : chaque closure a figé son propre coefficient au moment de la création. Cette technique de fabrique de fonctions permet de préparer une famille de fonctions paramétrées sans jamais toucher à une variable globale, et prépare directement le décorateur du chapitre 13.

Ex. 16 Mini-gestionnaire de bibliothèque DÉFI

Construisez un gestionnaire de bibliothèque reposant sur un dictionnaire d'état : biblio = {"Python facile": True, "Algorithmique": False}, où la valeur indique la disponibilité (True = en rayon). Écrivez quatre fonctions : afficher_biblio(biblio), emprunter(biblio, titre), rendre(biblio, titre) et disponible(biblio, titre) qui renvoie True/False. Chaque action impossible (titre inconnu, livre déjà emprunté, livre déjà présent) doit afficher un message clair. Terminez par une boucle de menu qui propose : 1) Afficher, 2) Emprunter, 3) Rendre, 0) Quitter.

Solution

Stratégie : le dictionnaire porte l'état, les fonctions le transforment, le menu l'observe. Chaque fonction commence par ses clauses de garde (titre inconnu, état incohérent), puis applique la modification. La boucle de menu combine while, input et if/elif : la synthèse totale des parties II et III.

Solution de l'exercice 16 : fonctions

```python
biblio = {"Python facile": True, "Algorithmique": False}
def disponible(biblio, titre):
    """Renvoie True si le titre est present ET disponible."""
    return biblio.get(titre, False)
def afficher_biblio(biblio):
    for titre, dispo in biblio.items():
        etat = "en rayon" if dispo else "emprunte"
        print(f" - {titre} ({etat})")
def emprunter(biblio, titre):
    if titre not in biblio:
        print(f"Le titre '{titre}' n'existe pas dans la bibliotheque.")
    elif not biblio[titre]:
        print(f"'{titre}' est deja emprunte.")
    else:
        biblio[titre] = False
        print(f"'{titre}' emprunte avec succes.")
def rendre(biblio, titre):
    if titre not in biblio:
        print(f"Le titre '{titre}' n'existe pas dans la bibliotheque.")
    elif biblio[titre]:
        print(f"'{titre}' n'a pas ete emprunte.")
    else:
        biblio[titre] = True
        print(f"'{titre}' rendu, merci !")
```

Solution de l'exercice 16 : boucle de menu

```python
while True:
    print("\n1) Afficher  2) Emprunter  3) Rendre  0) Quitter")
    choix = input("Votre choix : ")
    if choix == "0":
        print("Au revoir !")
        break
    elif choix == "1":
        afficher_biblio(biblio)
    elif choix == "2":
        titre = input("Titre a emprunter : ")
        emprunter(biblio, titre)
    elif choix == "3":
        titre = input("Titre a rendre : ")
        rendre(biblio, titre)
    else:
        print("Choix invalide, recommencez.")
```

Exemple de session : emprunter « Algorithmique » affiche « déjà emprunté » ; rendre « Python facile » puis le réemprunter fonctionne ; un titre fantôme est poliment refusé par la clause titre not in biblio. Remarquez la structure en couches : les fonctions ne contiennent aucun input de menu, elles reçoivent leurs données en paramètres : c'est ce découplage qui permet de les réutiliser plus tard dans une interface graphique sans rien réécrire. Ce programme est votre premier vrai logiciel : état persistant en mémoire, opérations contrôlées, interface de commande.

PARTIE IV

La programmation orientée objet

Jusqu'ici, données et traitements voyageaient séparément : des listes d'un côté, des fonctions de l'autre. La programmation orientée objet (POO) les réunifie : on définit des **classes** qui modélisent des objets du monde réel, chacun portant ses données (attributs) et ses comportements (méthodes). Cette partie vous mène des premières classes aux propriétés protégées, de l'héritage aux méthodes spéciales, jusqu'à un mini-projet bancaire complet.

CHAPITRE 15

Classes et objets : la première rencontre

Une **classe** est un plan de construction : elle décrit ce que des objets d'un même type possèdent et savent faire. Un **objet**, on dit aussi une **instance**, est une réalisation concrète de ce plan, comme une voiture construite d'après le dessin d'un ingénieur. Les données qu'un objet porte s'appellent ses **attributs** (nom, solde, couleur) ; les actions qu'il sait accomplir s'appellent ses **méthodes** (se présenter, déposer, avancer). La POO, c'est l'art de découper un programme en objets qui collaborent, chacun responsable de ses propres données.

15.1 Écrire sa première classe

Une classe se déclare avec le mot-clé class, suivi du nom en **CamelCase** (première lettre de chaque mot en majuscule) et de deux points. La méthode spéciale **__init__** est le constructeur : Python l'appelle automatiquement à chaque création d'objet, et c'est elle qui installe les attributs initiaux. Son premier paramètre, **self**, désigne l'objet en cours de construction : c'est lui qui relie les données à l'objet particulier dont on parle.

Code 15.1 : La classe Personne

```python
class Personne:
    """Représente une personne avec son nom et son âge."""
    def __init__(self, nom, age):
        self.nom = nom          # attribut : l'objet retient son nom
        self.age = age          # attribut : l'objet retient son age
    def se_presenter(self):
        """Affiche une presentation de la personne."""
        print(f"Je m'appelle {self.nom}, j'ai {self.age} ans.")
# Creation de deux objets independants
p1 = Personne("Roland", 27)     # __init__ est appele automatiquement
p2 = Personne("Ounimborbitibou", 34)
p1.se_presenter()               # Je m'appelle Roland, j'ai 27 ans.
p2.se_presenter()               # Je m'appelle Ounimborbitibou, j'ai 34 ans.
print(p1.nom)                   # acces direct a un attribut : Roland
```

Lisez attentivement la ligne de création : Personne("Roland", 27) ne passe **aucun** argument pour self, Python le fournit lui-même en y plaçant le nouvel objet. En revanche, à l'intérieur de la classe, toute référence à un attribut passe obligatoirement par self.nom, self.age : sans le préfixe, Python chercherait une variable locale ordinaire et lèverait une NameError. Cette gymnastique du self est la première habitude à ancrer : self désigne TOUJOURS l'objet sur lequel on travaille.

15.2 Une classe qui porte des données : CompteBancaire

Le vrai pouvoir de la POO apparaît quand un objet protège des données qui lui appartiennent. Un compte bancaire détient un titulaire et un solde, et propose des opérations contrôlées : déposer, retirer, consulter. Plutôt que de laisser n'importe quel code écrire compte.solde = -5000, la classe expose des méthodes qui appliquent les règles métier : un retrait ne peut pas dépasser le solde disponible.

Code 15.2 : La classe CompteBancaire (version initiale)

```python
class CompteBancaire:
    """Compte bancaire simple avec operations controlees."""
    def __init__(self, titulaire, solde_initial=0):
        self.titulaire = titulaire
        self.solde = solde_initial
    def deposer(self, montant):
        if montant <= 0:
            print("Le montant doit etre positif.")
        else:
            self.solde += montant
            print(f"{montant} F depose. Solde : {self.solde} F")
    def retirer(self, montant):
        if montant <= 0:
            print("Le montant doit etre positif.")
        elif montant > self.solde:
            print("Fonds insuffisants pour ce retrait.")
        else:
            self.solde -= montant
            print(f"{montant} F retires. Solde : {self.solde} F")
    def afficher(self):
        print(f"Compte de {self.titulaire} : {self.solde} F")
# Utilisation
compte = CompteBancaire("Awa", 25000)
compte.deposer(10000)      # 10000 F depose. Solde : 35000 F
compte.retirer(40000)      # Fonds insuffisants pour ce retrait.
compte.retirer(15000)      # 15000 F retires. Solde : 20000 F
compte.afficher()          # Compte de Awa : 20000 F
```

ATTENTION

Deux erreurs classiques au premier contact avec les classes. Oublier self dans la signature (def deposer(montant):) provoque une TypeError à l'appel, car Python transmet l'objet en premier argument quoi qu'il arrive. Et écrire solde au lieu de self.solde dans le corps crée une variable locale qui disparaît aussitôt : le vrai solde de l'objet n'a pas bougé. Vérifiez systématiquement la présence de self, en signature ET devant chaque attribut.

15.3 Plusieurs objets, une seule classe

Créons maintenant deux comptes : compte1 = CompteBancaire("Awa") et compte2 = CompteBancaire("Roland", 100000). Chaque objet possède son propre coffre : déposer sur compte1 ne change rien à compte2. C'est là toute la valeur du self : une seule écriture de méthode, mais autant de données que d'objets. On dit que les attributs sont des variables d'instance : chaque instance en détient un exemplaire indépendant. Ce mécanisme remplace élégamment les variables globales et leurs effets de bord du chapitre 12.

15.4 Attributs de classe et attributs d'instance

Un attribut défini dans __init__ avec self.nom = ... appartient à l'instance : chaque objet en possède son propre exemplaire. Mais Python permet aussi de définir un attribut directement dans le corps de la classe, hors de toute méthode : c'est un **attribut de classe**, partagé par toutes les instances. Il sert aux constantes et aux données communes : un taux d'intérêt unique, un compteur d'objets créés. La lecture self.taux_interet remonte d'abord vers la classe si l'instance ne possède pas son propre attribut de ce nom : attention, une affectation compte1.taux_interet = ... créerait un attribut d'instance qui masque celui de la classe, pour ce seul objet.

Code 15.3 : Un attribut de classe partagé par tous les comptes

```python
class CompteBancaire:
    taux_interet = 2.5             # attribut de CLASSE : partage par tous
    def __init__(self, titulaire, solde):
        self.titulaire = titulaire   # attribut d'INSTANCE : propre a chacun
        self.solde = solde
c1 = CompteBancaire("Awa", 10000)
c2 = CompteBancaire("Roland", 50000)
print(c1.taux_interet, c2.taux_interet)   # 2.5 2.5  (valeur partagee)
CompteBancaire.taux_interet = 3.0         # changement centralise
print(c1.taux_interet, c2.taux_interet)   # 3.0 3.0  (tout le monde suit)
c1.taux_interet = 9.9                     # cree un attribut d'INSTANCE !
print(c1.taux_interet, c2.taux_interet)   # 9.9 3.0  (c1 s'est decroche)
```

15.5 Tout est objet

Un secret bien gardé : vous pratiquez la POO depuis le chapitre 5 sans le savoir. En Python, **toute valeur est un objet** : les entiers, les chaînes, les listes, les dictionnaires, même les fonctions. Chacun est une instance d'une classe : type(42) répond , type("salut") répond , et les méthodes que vous appelez depuis toujours, notes.append(x) ou texte.upper(), sont des méthodes de ces classes. La syntaxe pointée est universelle :

objet.attribut lit une donnée, objet.methode(...) déclenche un comportement, quel que soit le type.

Code 15.4 : Toutes les valeurs sont des objets

```
print(type(42))              # <class 'int'>
print(type("salut"))         # <class 'str'>
print(type([1, 2]))          # <class 'list'>
print(type(len))             # <class 'builtin_function_or_method'>
# Un objet porte ses methodes : la preuve par la classe str
print("abc".upper())         # ABC : upper est une methode de str
print([3, 1].sort())         # None : sort trie sur place (chapitre 9)
# type() fonctionne aussi sur vos propres classes
p = Personne("Awa", 21)
print(type(p))               # <class '__main__.Personne'>
```

> **À RETENIR**
>
> class NomClasse définit un plan ; l'appel NomClasse(arguments) crée un objet et déclenche automatiquement __init__, qui installe les attributs initiaux. self désigne l'objet courant : présent en tête de chaque méthode, il préfixe chaque attribut (self.solde). Les attributs d'instance sont propres à chaque objet ; les attributs de classe sont partagés par tous. Et toute valeur Python est déjà un objet : vous pratiquiez la POO sans le savoir.

CHAPITRE 16

Encapsulation et propriétés

Notre classe CompteBancaire affiche un talon d'Achille : n'importe quelle ligne de code peut écrire compte.solde = -999999 et corrompre les règles métier. L'**encapsulation** est la technique qui consiste à cacher les données brutes derrière une interface contrôlée de méthodes et de propriétés. C'est un contrat de confiance : la classe promet que son état interne reste toujours cohérent, quel que soit le code qui l'utilise.

16.1 Signaler le privé : conventions _ et __

Python n'a pas de mot-clé private ; il s'appuie sur deux conventions universelles. Un attribut préfixé d'un seul tiret bas (_solde) signifie « interne : ne touchez pas depuis l'extérieur », un avertissement respecté par tout développeur sérieux. Le double tiret bas (__solde) déclenche la **modification de nom** (name mangling) : l'attribut est renommé en _CompteBancaire__solde, rendant l'accès extérieur accidentel pratiquement impossible. La convention d'usage : un tiret pour la vie privée polie, deux pour les cas où une erreur d'accès serait vraiment grave.

Code 16.1 : Passer aux attributs protégés

```
class CompteBancaire:
    def __init__(self, titulaire, solde_initial=0):
        self.titulaire = titulaire
        self._solde = solde_initial      # _ : attribut interne
    def deposer(self, montant):
        if montant > 0:
            self._solde += montant
    def retirer(self, montant):
        if 0 < montant <= self._solde:
            self._solde -= montant
    def afficher(self):
        print(f"Compte de {self.titulaire} : {self._solde} F")
compte = CompteBancaire("Awa", 20000)
compte.deposer(5000)
compte.afficher()          # Compte de Awa : 25000 F
# compte._solde = -999      # techniquement possible, mais trahit le contrat
# print(compte.__solde)     # AttributeError : le nom a ete transforme
```

16.2 @property : l'interface élégante

Cacher les données soulève une question pratique : comment permettre une **lecture** légitime du solde sans exposer l'écriture brute ? La réponse de Python est le décorateur **@property** : il transforme une méthode en attribut de lecture virtuel. Le code appelant écrit compte.solde comme un attribut ordinaire, mais c'est en réalité la méthode getter qui répond. On peut lui adjoindre un contrôleur d'écriture @solde.setter qui valide chaque tentative de modification. L'élégance du résultat : la syntaxe d'accès reste simple, la protection devient totale.

Code 16.2 : Solde protégé par @property

```
class CompteBancaire:
    def __init__(self, titulaire, solde_initial=0):
        self.titulaire = titulaire
        self._solde = solde_initial
    @property
    def solde(self):
        """Lecture du solde : compte.solde fonctionne comme un attribut."""
        return self._solde
    @solde.setter
    def solde(self, nouveau):
        """Ecriture controlee : les regles metier sont appliquees."""
        if nouveau < 0:
            raise ValueError("Le solde ne peut pas etre negatif.")
        self._solde = nouveau
    def deposer(self, montant):
        if montant > 0:
            self._solde += montant
    def retirer(self, montant):
        if 0 < montant <= self._solde:
            self._solde -= montant
        else:
            raise ValueError("Retrait refuse : fonds insuffisants.")
compte = CompteBancaire("Roland", 10000)
print(compte.solde)              # 10000  -> passe par le getter
compte.solde = 12000             # passe par le setter (valeur valide)
```

```
# compte.solde = -5              # ValueError : refuse par le setter
# compte.retirer(99999)          # ValueError : fonds insuffisants
```

Comparez les deux univers : au chapitre 15, compte.solde = -999999 réussissait sans broncher ; ici, la même tentative lève une ValueError explicite. Le setter est le gardien du **contrat d'invariant** (le solde d'un compte n'est jamais négatif) et cet invariant tient quel que soit le nombre de programmeurs qui utilisent la classe. Remarquez aussi raise : l'opposé du retrait silencieux, il signale bruyamment l'erreur pour que le code appelant puisse réagir ; le chapitre 20 approfondira la gestion de ces exceptions avec try / except.

16.3 Méthodes de classe et méthodes statiques

Deux variantes de méthodes complètent la boîte à outils. La méthode de classe (@classmethod) reçoit la **classe** elle-même en premier paramètre, conventionnellement nommé cls : elle sert typiquement à fabriquer des objets par des chemins alternatifs, par exemple CompteBancaire.creer_depuis_chaine("Awa;25000"). La méthode statique (@staticmethod) ne reçoit ni self ni cls : c'est une fonction utilitaire rangée dans la classe par commodité thématique, comme un calcul de conversion de devise. Utilisées avec parcimonie, ces deux décorateurs clarifient l'intention du code.

Code 16.3 : Une fabrique alternative et un utilitaire

```
class CompteBancaire:
    def __init__(self, titulaire, solde):
        self.titulaire = titulaire
        self.solde = solde
    @classmethod
    def creer_depuis_chaine(cls, enregistrement):
        """Fabrique alternative : 'Awa;25000' devient un objet."""
        titulaire, solde = enregistrement.split(";")
        return cls(titulaire, float(solde))   # cls() appelle __init__
    @staticmethod
    def devise(montant):
        """Utilitaire de presentation, independant de l'objet."""
        return f"{montant:,.0f} FCFA".replace(",", " ")
c = CompteBancaire.creer_depuis_chaine("Awa;25000")
print(c.titulaire, c.solde)             # Awa 25000.0
print(CompteBancaire.devise(c.solde))   # 25 000 FCFA
```

Deux lectures du code 16.3. Le classmethod construit : reconnu par cls en premier paramètre, il appelle cls(titulaire, float(solde)) comme on appellerait la classe elle-même, et si une fille de CompteBancaire l'utilise, c'est la fille qui sera construite : ce comportement polymorphe est la vraie raison d'exister de cls. Le staticmethod, lui, ne connaît ni l'objet ni la classe : il ne fait que ranger un utilitaire à sa place logique, dans la classe qui lui donne un sens.

ASTUCE

Règle d'or d'architecture : rendez les attributs privés (_nom) dès qu'une règle métier les concerne, et exposez @property pour la lecture. Le coût est de cinq lignes ; le bénéfice est immense : vous pourrez un jour changer la représentation interne (par exemple stocker les soldes en centimes entiers) sans casser le moindre code appelant, car l'interface compte.solde restera identique.

À RETENIR

L'encapsulation cache les données brutes (_solde) derrière une interface contrôlée. @property transforme une méthode en lecture d'attribut ; @attribut.setter valide chaque écriture et peut lever ValueError. Les méthodes de classe reçoivent cls pour fabriquer des objets autrement, les méthodes statiques sont des utilitaires neutres. L'objectif : des invariants garantis (un solde négatif est désormais impossible).

CHAPITRE 17

Héritage et polymorphisme

Regardez une flotte de véhicules : voiture, moto, camion. Tous possèdent une marque, une vitesse, savent rouler et se décrire, mais chacun redéfinit certains comportements à sa manière. L'**héritage** formalise cette parenté : une classe fille hérite des attributs et méthodes de sa classe mère, les enrichit et les spécialise. Le **polymorphisme** en est la conséquence magique : le même appel de méthode produit des comportements différents selon l'objet qui le reçoit.

17.1 Créer une classe fille

La syntaxe d'héritage tient dans les parenthèses du nom de classe : class Voiture(Vehicule) déclare Voiture comme fille de Vehicule. La fille bénéficie d'office de tout le contenu de la mère. Dans son constructeur, elle étend celui de la mère en l'appelant explicitement via **super().__init__(...)**, où super() désigne la classe mère, puis installe ses attributs propres. Cette étape super() est la seule façon correcte d'initialiser la partie héritée de l'objet.

Code 17.1 : Héritage entre Vehicule, Voiture et Moto

```python
class Vehicule:
    """Classe mere : ce qui est commun a tous les vehicules."""
    def __init__(self, marque, vitesse=0):
        self.marque = marque
        self.vitesse = vitesse
    def rouler(self, duree):
        self.vitesse += duree * 10
        print(f"{self.marque} roule a {self.vitesse} km/h.")
    def se_decrire(self):
        return f"Vehicule de marque {self.marque}"
class Voiture(Vehicule):
    """Fille specialisee : ajoute le nombre de places."""
    def __init__(self, marque, places, vitesse=0):
        super().__init__(marque, vitesse)   # la partie heritee d'abord
        self.places = places                # puis les attributs propres
    def klaxonner(self):
        print(f"{self.marque} : Pouet pouet !")
class Moto(Vehicule):
    """Autre fille : ajoute la cylindree."""
    def __init__(self, marque, cylindree, vitesse=0):
        super().__init__(marque, vitesse)
        self.cylindree = cylindree
# Une seule classe mere, trois comportements partages
v = Voiture("Toyota", 5)
m = Moto("Suzuki", 650)
v.rouler(2)              # Toyota roule a 20 km/h.   (methode heritee)
m.rouler(1)              # Suzuki roule a 10 km/h.   (methode heritee)
v.klaxonner()            # Toyota : Pouet pouet !    (methode propre)
print(isinstance(m, Vehicule))   # True : une Moto EST un Vehicule
```

La fonction isinstance(objet, Classe) vérifie la parenté : une Moto est un Vehicule, mais un Vehicule n'est pas forcément une Moto. Ce test est le couteau suisse du code polymorphe : il permet de réagir différemment selon la vraie nature d'un objet reçu en paramètre, sans jamais casser si le type inattendu survient.

17.2 Redéfinir une méthode (override)

Une fille peut réécrire une méthode héritée en la redéfinissant sous le même nom : on parle de **surcharge** ou override. C'est ici que naît le polymorphisme : le même message se_decrire() envoyé à une flotte d'objets hétérogènes obtient de chacun une réponse adaptée à sa nature. Pour étendre plutôt que remplacer, la redéfinition peut appeler la version mère via super().se_decrire() et concaténer son propre ajout : la fille conserve alors l'héritage et l'enrichit.

Code 17.2 : Polymorphisme en action

```
class VoitureSport(Voiture):
    def se_decrire(self):                       # override
        base = super().se_decrire()             # reutilise la version mere
        return base + " (edition sport)"
flotte = [Voiture("Toyota", 5), Moto("Suzuki", 650), VoitureSport("Ferrari", 2)]
for engin in flotte:
    print(engin.se_decrire())
# Affichage :
#   Vehicule de marque Toyota
#   Vehicule de marque Suzuki
#   Vehicule de marque Ferrari (edition sport)
```

Contemplez la boucle : elle ne connaît ni Voiture ni Moto ni VoitureSport, elle appelle seulement se_decrire() et laisse chaque objet répondre à sa façon. Ajouter demain un Camion ne modifiera pas une ligne de cette boucle : c'est le principe d'**ouverture à l'extension, fermeture à la modification**, pierre angulaire des architectures logicielles professionnelles.

17.3 Héritage avec discernement : la composition

L'héritage exprime une relation « EST UN » : une Voiture EST UN Vehicule, un Etudiant EST UNE Personne. Quand la relation est « A UN » (une Voiture A UN moteur, une Ecole A DES eleves), la **composition** est le bon outil : stocker l'objet en attribut plutôt qu'en hériter, puis lui déléguer le travail. Abuser de l'héritage crée des hiérarchies rigides où chaque changement de la mère secoue toutes les filles. La sagesse du métier : préférez la composition, réservez l'héritage aux parentés réelles et stables.

Code 17.3 : La composition : une Voiture A UN Moteur

```
class Moteur:
    def __init__(self, puissance):
        self.puissance = puissance
    def demarrer(self):
        print(f"Moteur {self.puissance} ch en marche.")
class Voiture:
    def __init__(self, marque, puissance):
        self.marque = marque
        self.moteur = Moteur(puissance)    # la voiture A UN moteur
    def partir(self):
        print(f"{self.marque} prend la route.")
        self.moteur.demarrer()             # delegation au composant
v = Voiture("Toyota", 120)
v.partir()
# Toyota prend la route.
# Moteur 120 ch en marche.
```

17.4 L'héritage multiple et l'ordre MRO

Python autorise une classe à hériter de plusieurs mères : class Smartphone(Telephone, AppareilPhoto). L'objet réunit alors les attributs et méthodes des deux lignées. Pour savoir dans quel ordre Python cherche une méthode quand plusieurs mères la proposent, consultez l'attribut **__mro__** (Method Resolution Order) : Python remonte la chaîne de gauche à droite, la première classe listée qui sait répondre l'emporte, et tout le monde retombe in fine sur object, l'ancêtre universel de toutes les classes. L'héritage multiple est puissant mais exigeant : réservez-le aux combinaisons naturelles, ou aux **mixins**, ces petites classes qui apportent un service unique (sérialisation, journalisation) et n'ont pas de vie propre.

Code 17.4 : Héritage multiple et ordre de résolution

```
class Telephone:
    def appeler(self):
        print("Je passe un appel.")
class AppareilPhoto:
    def photographier(self):
        print("Je prends une photo.")
class Smartphone(Telephone, AppareilPhoto):    # deux meres declarees
    def photographier(self):                   # specialisation locale
```

```
        print("Je prends une photo en 48 megapixels.")
s = Smartphone()
s.appeler()          # Je passe un appel.       (vient de Telephone)
s.photographier()    # Je prends une photo en 48 megapixels. (la fille)
print(Smartphone.__mro__)
# (Smartphone, Telephone, AppareilPhoto, object)
```

Lisez l'ordre MRO affiché : Smartphone d'abord, puis Telephone, puis AppareilPhoto, puis object. Si Telephone et AppareilPhoto proposaient toutes deux une méthode d'urgence, la version de Telephone gagnerait, car elle apparaît avant dans le MRO. super(), lui, ne désigne pas simplement « la mère » mais « la classe suivante dans le MRO » : c'est ce qui rend super() fiable même en présence d'héritage multiple, chaque maillon initialisant le suivant sans doublon.

ATTENTION

Oublier l'appel super().__init__(...) dans le constructeur d'une fille est le bug d'héritage le plus fréquent : les attributs de la mère ne sont alors jamais installés, et le premier accès lève une AttributeError mystérieuse. Discipline : dans tout __init__ d'une fille, la première ligne est super().__init__(...) avec les paramètres hérités.

À RETENIR

class Fille(Mere) hérite de tout le contenu de la mère ; super().__init__(...) initialise la partie héritée. Redéfinir une méthode spécialise son comportement ; super().methode() l'étend au lieu de la remplacer. Le polymorphisme permet à une même boucle de traiter des objets hétérogènes par leur interface commune, l'héritage multiple réunit plusieurs lignées et __mro__ fixe l'ordre de recherche. Relation EST UN : héritage ; relation A UN : composition.

CHAPITRE 18

Les méthodes spéciales (dunder)

Python nourrit ses classes de **méthodes spéciales**, reconnaissables à leurs doubles tirets bas, d'où leur surnom de méthodes *dunder* (double underscore). Vous en connaissez déjà une : __init__. Elles ne se callent jamais directement : Python les invoque tout seul quand vous affichez un objet, comparez deux objets, mesurez leur longueur. Les implémenter rend vos classes natives, indiscernables des types intégrés du langage.

18.1 Les méthodes dunder essentielles

Méthode	Déclenchée par	Rôle
__init__(self, ...)	Objet(...) à la création	Installe les attributs initiaux
__str__(self)	print(objet) ou str(objet)	Version lisible pour l'utilisateur
__repr__(self)	Saisie en console, débogueur	Version technique pour le développeur
__len__(self)	len(objet)	Définit ce que signifie « longueur »
__eq__(self, autre)	objet == autre	Définit l'égalité entre deux objets
__add__(self, autre)	objet + autre	Définit l'addition personnalisée

Tableau 18.1 : Les méthodes spéciales les plus utiles

Code 18.1 : Une classe Etudiant dotée de __str__ et __eq__

```python
class Etudiant:
    def __init__(self, nom, notes):
        self.nom = nom
        self.notes = notes
    def moyenne(self):
        return sum(self.notes) / len(self.notes)
    def __str__(self):
        return f"Etudiant({self.nom}, moyenne {self.moyenne():.2f})"
    def __repr__(self):
        return f"Etudiant({self.nom!r}, {self.notes})"
    def __eq__(self, autre):
        return self.nom == autre.nom and self.notes == autre.notes
a = Etudiant("Awa", [14, 16.5, 11])
b = Etudiant("Awa", [14, 16.5, 11])
print(a)                 # Etudiant(Awa, moyenne 13.83)   -> via __str__
print(a == b)            # True                           -> via __eq__
# Sans __eq__, a == b comparait les ADRESSES memoire : False !
```

Sans __eq__, l'égalité entre deux objets compare leurs adresses mémoire : deux fiches identiques mais distinctes seraient « différentes ». En définissant __eq__, vous décidez de la sémantique : deux étudiants sont égaux si nom et notes coïncident. C'est exactement ainsi que "abc" == "abc" fonctionne : la classe str implémente elle-même ces méthodes spéciales. Vous pratiquiez donc la POO sans le savoir, car les listes, les chaînes et les dictionnaires sont des classes.

18.2 La surcharge d'opérateurs

Les méthodes dunder ne se limitent pas à l'affichage et à l'égalité : chacune branche une construction syntaxique de Python. Définir __add__ donne un sens à l'opérateur +, __lt__ à l'opérateur <, __len__ à la fonction len(), __getitem__ aux crochets objet[index]. Cette pratique, la **surcharge d'opérateurs**, fait de vos objets des citoyens de première classe : on peut les additionner, les trier avec sorted(), les parcourir avec for, exactement comme les types intégrés.

Code 18.2 : Une classe Duree qui s'additionne et se compare

```python
class Duree:
    """Une duree en heures et minutes, comparable et additionnable."""
    def __init__(self, heures=0, minutes=0):
        self.minutes_totales = heures * 60 + minutes
    @property
    def heures(self):
        return self.minutes_totales // 60
    @property
    def minutes(self):
        return self.minutes_totales % 60
    def __add__(self, autre):
        return Duree(0, self.minutes_totales + autre.minutes_totales)
    def __eq__(self, autre):
        return self.minutes_totales == autre.minutes_totales
    def __lt__(self, autre):
        return self.minutes_totales < autre.minutes_totales
    def __str__(self):
        return f"{self.heures} h {self.minutes:02d}"
t1 = Duree(1, 50)          # 1 h 50
t2 = Duree(0, 40)          # 0 h 40
print(t1 + t2)             # 2 h 30     (__add__ a travaille)
print(t1 == Duree(0, 110)) # True       (__eq__)
print(t2 < t1)             # True       (__lt__)
print(sorted([t1, t2, Duree(2, 5)]))   # [0 h 40, 1 h 50, 2 h 05]
```

Trois points de design dans cette classe. Le constructeur normalise tout en minutes_totales : une seule source de vérité, les propriétés heures et minutes ne sont que des vues. __add__ renvoie un **nouvel** objet Duree au lieu de modifier l'un des opérandes : l'addition ne détruit pas ses termes, comme la somme de deux entiers. Et __lt__ suffit à sorted() pour ordonner toute une liste : le tri professionnel n'exige que la comparaison de deux éléments à la fois.

Code 18.3 : Une Playlist qui se parcourt comme une liste

```python
class Playlist:
    def __init__(self, nom):
        self.nom = nom
        self.morceaux = []
    def ajouter(self, titre):
        self.morceaux.append(titre)
    def __len__(self):
        return len(self.morceaux)
    def __getitem__(self, index):
        return self.morceaux[index]
    def __str__(self):
        return f"Playlist '{self.nom}' ({len(self)} morceaux)"
playlist = Playlist("Ambiance etude")
```

```
playlist.ajouter("Piano douce")
playlist.ajouter("Lo-fi classique")
print(len(playlist))        # 2             (__len__)
print(playlist[0])          # Piano douce   (__getitem__)
for titre in playlist:      # la boucle for passe par __getitem__
    print(f"> {titre}")
```

Notez la boucle for finale : Python n'exige pas une vraie liste, il demande simplement à l'objet de savoir répondre à playlist[0], playlist[1], etc. Dès que __getitem__ existe, la boucle for et le test d'appartenance fonctionnent sur votre classe. Remarquez aussi __str__ qui appelle len(self) : la classe réutilise sa propre méthode, comme toute bonne classe. Le Tome 2 présentera __iter__ et les générateurs, la mécanique encore plus élégante des itérateurs authentiques.

CHAPITRE 19

Mini-projet final : un système bancaire à trois classes

Le projet de fin de tome assemble tout : encapsulation (chapitre 16), composition (chapitre 17), attributs de classe (chapitre 15), méthodes spéciales (chapitre 18), dictionnaires et fonctions (partie III). L'architecture tient en trois rôles : **Compte** protège son solde avec @property ; **Client** porte l'identité et la liste de ses comptes (composition : un client A DES comptes) ; **Banque** administre un dictionnaire de clients et propose les opérations globales. Lisez chaque classe, puis le scénario d'utilisation qui fait vivre le système.

Code 19.1 : Le projet, classe par classe

```
class Compte:
    """Compte bancaire encapsule, identifie par un numero."""
    compteur = 0                         # attribut de CLASSE : partage par tous
    def __init__(self, solde_initial=0):
        Compte.compteur += 1
        self.numero = f"CPT-{Compte.compteur:04d}"
        self._solde = solde_initial
    @property
    def solde(self):
        return self._solde
    def deposer(self, montant):
        if montant > 0:
            self._solde += montant
            return True
        return False
    def retirer(self, montant):
        if 0 < montant <= self._solde:
            self._solde -= montant
            return True
        return False
    def __str__(self):
        return f"{self.numero} : {self._solde} F"
class Client:
    """Client de la banque : un nom et DES comptes (composition)."""
    def __init__(self, nom):
        self.nom = nom
        self.comptes = []
    def ouvrir_compte(self, solde_initial=0):
        compte = Compte(solde_initial)
        self.comptes.append(compte)
        return compte
    def fortune(self):
        return sum(c.solde for c in self.comptes)
class Banque:
    """Gere un annuaire de clients par nom."""
    def __init__(self, nom_banque):
        self.nom_banque = nom_banque
        self.clients = {}
    def inscrire(self, nom):
        if nom not in self.clients:
            self.clients[nom] = Client(nom)
        return self.clients[nom]
    def virement(self, source, destination, montant):
        if source.retirer(montant):
            destination.deposer(montant)
            return True
        return False
```

Code 19.2 : Le projet, scénario d'utilisation

```
# Mise en service
banque = Banque("Bourse du Fleuve")
awa = banque.inscrire("Awa")
roland = banque.inscrire("Roland")
c1 = awa.ouvrir_compte(50000)
c2 = roland.ouvrir_compte(120000)
# Vie du compte
c1.deposer(20000)
banque.virement(c1, c2, 30000)     # retrait ok puis depot : le virement passe
banque.virement(c2, c1, 999999)    # retrait refuse : le virement echoue
# Etat final
print(c1)                          # CPT-0001 : 40000 F    (via __str__)
print(c2)                          # CPT-0002 : 150000 F
print(f"Fortune de {awa.nom} : {awa.fortune()} F")
```

Détails à savourer. L'attribut de classe compteur (section 15.4) est partagé par tous les comptes : il fabrique des numéros uniques sans dictionnaire d'administration. Le virement est **atomique par construction** : si le retrait échoue, le dépôt n'a pas lieu, impossible de créer de l'argent ou d'en détruire. La méthode fortune() exploite une expression génératrice sum(... for ...), cousin des compréhensions (chapitre 10). Et chaque classe reste testable isolément : créez un Compte tout seul, il fonctionne, c'est la marque d'une bonne séparation des responsabilités.

Pistes pour dépasser le projet une fois la leçon acquise : ajouter une propriété decouvert autorisé avec plafond, une classe CompteEpargne fille qui capitalise des intérêts à l'appel d'une méthode passer_le_mois(), un historique des opérations par compte (une liste de textes remplie par deposer et retirer), ou un rapport __str__ pour Banque qui affiche clients et fortunes. Chaque extension est un exercice de design en soi : choisissez la composition ou l'héritage en posant d'abord la question EST UN ou A UN.

> **À RETENIR**
>
> Les méthodes dunder (__init__, __str__, __repr__, __eq__, __add__, __lt__, __len__, __getitem__) branchent vos classes sur la syntaxe native de Python : print, ==, +, sorted, len, for. Un attribut de classe est partagé par toutes les instances, contrairement aux attributs d'instance. Un mini-projet POO se structure en responsabilités : la donnée protégée (Compte), le porteur (Client, composition), l'administrateur (Banque, dictionnaire). Si chaque classe fonctionne seule, l'architecture est saine.

Exercices corrigés de la Partie IV

Dernière ligne droite : une série d'exercices de POO, du rectangle au système scolaire complet. Écrivez chaque classe dans un fichier séparé, testez-la dans la console, puis comparez aux corrections. Concentrez-vous sur la qualité de l'encapsulation autant que sur les résultats.

Ex. 17 La classe Rectangle BASIQUE

Écrivez une classe Rectangle avec deux attributs longueur et largeur, une méthode perimetre(), une méthode aire() et la méthode __str__ qui affiche par exemple « Rectangle 5 x 3 (aire 15) ». Testez avec un rectangle de 5 par 3 : périmètre attendu 16, aire 15.

Solution

Stratégie : le constructeur installe les deux attributs, deux méthodes calculent, __str__ embellit l'affichage. C'est l'exercice de mise en jambes : class, __init__, self, méthodes, dunder, rien de plus.

Solution de l'exercice 17

```python
class Rectangle:
    def __init__(self, longueur, largeur):
        self.longueur = longueur
        self.largeur = largeur
    def perimetre(self):
        return 2 * (self.longueur + self.largeur)
    def aire(self):
        return self.longueur * self.largeur
    def __str__(self):
        return f"Rectangle {self.longueur} x {self.largeur} (aire {self.aire()})"
r = Rectangle(5, 3)
print(r)                    # Rectangle 5 x 3 (aire 15)
print(r.perimetre())        # 16
```

Notez que __str__ réutilise self.aire() au lieu de recalculer self.longueur * self.largeur : une méthode appelle librement les autres méthodes de sa propre classe. Ce réflexe (une seule source de vérité par calcul) garantit que si la formule de l'aire change un jour, l'affichage suivra automatiquement.

Ex. 18 La classe Etudiant et sa mention BASIQUE

Écrivez une classe Etudiant munie d'un nom et d'une liste de notes. Ajoutez une méthode moyenne() et une méthode mention() qui renvoie « Excellent » si la moyenne est au moins 16, « Très bien » à partir de 14, « Passable » à partir de 10, sinon « Insuffisant ». Testez avec Awa [14, 16.5, 11] (moyenne 13.83, mention Passable) et Roland [18, 17, 19] (moyenne 18.0, mention Excellent).

Solution

Stratégie : la méthode mention() s'appuie sur moyenne() plutôt que de recalculer la somme, même réflexe de réutilisation qu'à l'exercice précédent. La cascade de seuils reprend la logique du chapitre 7, du plus sélectif au plus général.

Solution de l'exercice 18

```python
class Etudiant:
    def __init__(self, nom, notes):
        self.nom = nom
        self.notes = notes
    def moyenne(self):
        return sum(self.notes) / len(self.notes)
    def mention(self):
        m = self.moyenne()
        if m >= 16:
            return "Excellent"
        elif m >= 14:
            return "Tres bien"
        elif m >= 10:
            return "Passable"
        else:
            return "Insuffisant"
awa = Etudiant("Awa", [14, 16.5, 11])
roland = Etudiant("Roland", [18, 17, 19])
print(f"{awa.nom} : {awa.moyenne():.2f} -> {awa.mention()}")
print(f"{roland.nom} : {roland.moyenne():.2f} -> {roland.mention()}")
```

Affichage attendu : « Awa : 13.83 -> Passable » puis « Roland : 18.00 -> Excellent ». Vérifions la première : (14 + 16.5 + 11) / 3 = 41.5 / 3 = 13.83. La moyenne est calculée une fois et stockée dans la variable locale m : appeler self.moyenne() quatre fois dans la cascade fonctionnerait, mais gaspillerait trois calculs identiques : l'efficacité aussi est une politesse.

Ex. 19 Héritage de livres : papier et numérique INTERMÉDIAIRE

Écrivez une classe Livre (titre, auteur, prix) avec une méthode presenter() renvoyant par exemple « Python facile par Roland : 8500 F ». Créez ensuite une classe fille LivreNumerique qui ajoute taille_mo et redéfinit presenter() pour ajouter « (telechargeable, X Mo) ». Instanciez les deux classes dans une liste et faites-les toutes se présenter dans une boucle : c'est le polymorphisme du chapitre 17 en situation.

Solution

Stratégie : la fille appelle super().__init__(titre, auteur, prix) pour la partie héritée, installe taille_mo, puis redéfinit presenter() en réutilisant super().presenter() et en collant son suffixe. La boucle finale démontre le polymorphisme sans même connaître les types.

Solution de l'exercice 19

```python
class Livre:
    def __init__(self, titre, auteur, prix):
        self.titre = titre
        self.auteur = auteur
        self.prix = prix
    def presenter(self):
        return f"{self.titre} par {self.auteur} : {self.prix} F"
class LivreNumerique(Livre):
    def __init__(self, titre, auteur, prix, taille_mo):
        super().__init__(titre, auteur, prix)
        self.taille_mo = taille_mo
    def presenter(self):
        base = super().presenter()
        return f"{base} (telechargeable, {self.taille_mo} Mo)"
catalogue = [
    Livre("Python facile", "Roland", 8500),
    LivreNumerique("Algorithmique", "DJABON", 6000, 12.4),
]
for livre in catalogue:
    print(livre.presenter())
# Python facile par Roland : 8500 F
# Algorithmique par DJABON : 6000 F (telechargeable, 12.4 Mo)
```

L'extension est sans douleur : ajouter demain LivreAudio (durée, narrateur) n'exigera aucune modification de la boucle ni de la classe mère. Le super() en cascade (LivreNumerique appelle Livre) illustre la bonne chaîne d'initialisation ; si vous l'oubliez, le premier accès à self.prix lèvera AttributeError. Retenez le patron complet : hériter, étendre le constructeur avec super(), redéfinir les méthodes à spécialiser en réutilisant super().methode().

Ex. 20 Conversion thermique par propriétés

INTERMÉDIAIRE

Écrivez une classe Temperature dont l'attribut interne _celsius est exposé via @property. La classe offre en plus une propriété fahrenheit en lecture ET écriture, qui convertit automatiquement (F = C × 9/5 + 32 ; C = (F - 32) × 5/9). Le setter de _celsius doit refuser toute valeur sous le zéro absolu (-273.15 °C) en levant une ValueError. Testez : fixer celsius à 25 doit donner fahrenheit 77.0 ; fixer fahrenheit à 32 doit donner celsius 0.0.

Solution

Stratégie : la propriété maîtresse est celsius, validée à l'écriture ; fahrenheit est une propriété calculée : son getter convertit _celsius, son setter convertit puis affecte à self.celsius, profitant ainsi gratuitement de la validation du zéro absolu. Un seul contrôle, appliqué aux deux portes : c'est l'architecture de la validation unique.

Solution de l'exercice 20

```python
class Temperature:
    def __init__(self, celsius=0.0):
        self.celsius = celsius          # passe par le setter -> valide
    @property
    def celsius(self):
        return self._celsius
    @celsius.setter
    def celsius(self, valeur):
        if valeur < -273.15:
            raise ValueError("En dessous du zero absolu : impossible.")
        self._celsius = float(valeur)
    @property
    def fahrenheit(self):
        return self._celsius * 9 / 5 + 32
    @fahrenheit.setter
    def fahrenheit(self, valeur):
        self.celsius = (valeur - 32) * 5 / 9   # reutilise la validation
t = Temperature(25)
print(t.fahrenheit)      # 77.0
t.fahrenheit = 32        # ecriture via la propriete calculee
print(t.celsius)         # 0.0
# t.celsius = -300       # ValueError : en dessous du zero absolu
```

Suivez le trajet de t.fahrenheit = 32 : le setter calcule (32 - 32) × 5/9 = 0.0, puis affecte self.celsius = 0.0 : ce qui déclenche le setter de celsius et sa garde du zéro absolu. Résultat : impossible d'introduire une température physiquement absurde par l'une ou l'autre porte. Essayez t.celsius = -300 : ValueError immédiat. Cette redondance contrôlée (deux vues d'une même donnée, une seule règle) est le vrai cadeau de @property au concepteur de classes.

Ex. 21 La classe Argent avec opérateurs

INTERMÉDIAIRE

Écrivez une classe Argent qui porte un montant en FCFA. Installez __str__ pour afficher « 12 500 FCFA » (espaces de milliers), __add__ pour additionner deux Argent (mais refusez l'addition avec un nombre simple en levant une TypeError), __eq__ pour comparer deux montants et __lt__ pour permettre sorted() sur une liste d'Argent. Testez avec [Argent(9500), Argent(2500), Argent(12000)] : le tri doit donner 2500, 9500, 12000 et la somme totale 24000.

Solution

Stratégie : un seul attribut numérique porte toute la logique ; les quatre dunder délèguent à des comparaisons d'entiers. Le point d'architecture : vérifier isinstance(autre, Argent) dans __add__ pour refuser poliment les mélanges de types, c'est le contrat d'invariant du chapitre 16 appliqué aux opérateurs.

Solution de l'exercice 21

```python
class Argent:
    def __init__(self, montant):
        self.montant = montant
    def __str__(self):
        return f"{self.montant:,} FCFA".replace(",", " ")
    def __add__(self, autre):
        if not isinstance(autre, Argent):
            raise TypeError("On n'additionne que des Argent.")
        return Argent(self.montant + autre.montant)
    def __eq__(self, autre):
        return self.montant == autre.montant
    def __lt__(self, autre):
        return self.montant < autre.montant
portefeuille = [Argent(9500), Argent(2500), Argent(12000)]
for b in sorted(portefeuille):
    print(b)                        # 2 500 FCFA / 9 500 FCFA / 12 000 FCFA
total = sum(portefeuille, Argent(0))
print(total)                        # 24 000 FCFA
# total = Argent(100) + 50          # TypeError : refuse a juste titre
```

La garde isinstance dans __add__ illustre une décision de conception : mieux vaut une erreur immédiate et explicite qu'un résultat absurde (que vaudrait Argent(100) + 50 : 150 francs ? 150 objets ?). L'astuce de formatage : le spécificateur :, insère des virgules de milliers, puis replace() les échange contre des espaces. Et remarquez que sorted() n'a demandé aucun argument : il a découvert l'ordre tout seul grâce à __lt__, la méthode qu'il utilise en interne. L'appel sum(portefeuille, Argent(0)) part de zéro objet et additionne : la fonction sum marche aussi sur vos classes !

Ex. 22 Mini-système de gestion d'école

DÉFI

Construisez un système complet à partir de : une classe Personne (nom, age, méthode se_presenter()), deux filles Eleve (niveau, liste de notes, méthode moyenne()) et Professeur (matiere, salaire, méthode augmenter(pourcentage)), et une classe Ecole qui administre le tout : dictionnaire d'inscrits par nom, methodes inscrire_eleve(nom, age, niveau), recruter_prof(nom, age, matiere, salaire), ajouter_note(nom, note) pour un eleve, masse_salariale() qui totalise les salaires des professeurs, et bulletin(nom) qui affiche la fiche complète d'un élève (présentation, niveau, moyenne, mention de votre choix). Protégez chaque opération impossible par un message clair. Testez le scénario : inscrire Awa (17 ans, Terminale), recruter M. HOUNKPE (mathematiques, 350000 F), recruter M. DJABON (algorithmique, 400000 F), ajouter à Awa les notes 15, 17, 13, afficher son bulletin et la masse salariale (750000 F).

Solution

Stratégie : une hiérarchie Personne -> Eleve / Professeur exprime la relation EST UN ; la classe Ecole compose le tout avec un dictionnaire nom -> objet. Chaque méthode publique de Ecole commence par sa clause de garde (inscrit inconnu, note hors bornes). La méthode bulletin() démontre le polymorphisme interne : elle appelle se_presenter() sans savoir si l'objet est un élève ou un professeur.

Solution de l'exercice 22 : les personnes

```python
class Personne:
    def __init__(self, nom, age):
        self.nom = nom
        self.age = age
    def se_presenter(self):
        return f"{self.nom}, {self.age} ans"
class Eleve(Personne):
    def __init__(self, nom, age, niveau):
        super().__init__(nom, age)
        self.niveau = niveau
        self.notes = []
    def moyenne(self):
        if len(self.notes) == 0:
            return None
        return sum(self.notes) / len(self.notes)
class Professeur(Personne):
    def __init__(self, nom, age, matiere, salaire):
        super().__init__(nom, age)
        self.matiere = matiere
        self.salaire = salaire
    def augmenter(self, pourcentage):
        self.salaire += self.salaire * pourcentage / 100
```

Solution de l'exercice 22 : l'école

```python
class Ecole:
    def __init__(self, nom_ecole):
        self.nom_ecole = nom_ecole
        self.inscrits = {}          # nom -> Eleve ou Professeur
    def _trouver(self, nom):
        """Clause de garde commune : l'inscrit doit exister."""
        if nom not in self.inscrits:
            print(f"Personne nommee '{nom}' dans l'ecole.")
            return None
        return self.inscrits[nom]
    def inscrire_eleve(self, nom, age, niveau):
        self.inscrits[nom] = Eleve(nom, age, niveau)
    def recruter_prof(self, nom, age, matiere, salaire):
        self.inscrits[nom] = Professeur(nom, age, matiere, salaire)
    def ajouter_note(self, nom, note):
        eleve = self._trouver(nom)
        if eleve is None or not isinstance(eleve, Eleve):
            print(f"'{nom}' n'est pas un eleve.")
            return
        if 0 <= note <= 20:
            eleve.notes.append(note)
        else:
            print("Note invalide : elle doit etre entre 0 et 20.")
    def masse_salariale(self):
        total = 0
        for personne in self.inscrits.values():
            if isinstance(personne, Professeur):
                total += personne.salaire
        return total
    def bulletin(self, nom):
        eleve = self._trouver(nom)
        if eleve is None or not isinstance(eleve, Eleve):
            print(f"'{nom}' n'est pas un eleve.")
            return
        moyenne = eleve.moyenne()
        print(f"BULLETIN - {eleve.se_presenter()} - {eleve.niveau}")
        print(f"Notes : {eleve.notes}")
        print(f"Moyenne : {moyenne:.2f}")
# Scenario de test
ecole = Ecole("Lycee de la Rive")
ecole.inscrire_eleve("Awa", 17, "Terminale")
ecole.recruter_prof("HOUNKPE", 40, "Mathematiques", 350000)
ecole.recruter_prof("DJABON", 38, "Algorithmique", 400000)
for note in [15, 17, 13]:
    ecole.ajouter_note("Awa", note)
ecole.ajouter_note("Awa", 25)       # Note invalide
ecole.bulletin("Awa")
print(f"Masse salariale : {ecole.masse_salariale()} F")
```

Résultats attendus : une ligne « Note invalide » pour le 25 refusé, un bulletin affichant les notes [15, 17, 13] et la moyenne 15.00, puis « Masse salariale : 750000 F ». Vérification : (15 + 17 + 13) / 3 = 45 / 3 = 15. La méthode privée _trouver() factorise la garde « inscrit inconnu » : un seul point de maintenance. isinstance() distingue les élèves des professeurs dans les boucles, écho direct au polymorphisme du chapitre 17. Vous venez d'écrire votre premier système objet complet : hiérarchie, composition, encapsulation partielle, validations et rapport de sortie. Le Tome 2 prolongera ce système avec des tests automatiques complets puis une base de données.

PARTIE V

Erreurs, exceptions et débogage

Un programme fiable n'est pas un programme sans erreur : c'est un programme qui anticipe les erreurs, les signale proprement et se corrige vite. Cette partie vous apprend d'abord à lire les rapports d'erreur de Python, à contrôler les exceptions avec try / except / else / finally et raise, puis à traquer les bugs de façon méthodique avec les outils du développeur : print bien utilisé, assert, breakpoint() et une méthode de travail éprouvée. Ces compétences font la différence entre un script qui s'effondre au premier utilisateur distrait et un programme qui tient la route.

CHAPITRE 20

Les erreurs et les exceptions

Depuis le chapitre 5, vous avez déjà croisé des messages rouges et parfois énigmatiques. Il est temps de les comprendre et, mieux, de les maîtriser : les exceptions sont le mécanisme que Python met à votre disposition pour signaler une situation anormale et pour y répondre proprement. Ce chapitre vous apprend à classer les erreurs, à lire un traceback comme un professionnel, à attraper les exceptions dangereuses avec try / except, et à lever les vôtres avec raise pour protéger les règles de vos programmes.

20.1 Les trois familles d'erreurs

Avant de contrôler les erreurs, il faut savoir les reconnaître. Python range les incidents en trois familles de natures très différentes, qui exigent des réponses différentes. Les **erreurs de syntaxe** sont détectées avant même le premier mouvement du programme : le texte enfreint la grammaire du langage (un deux-points oublié, une parenthèse non fermée) et Python refuse de démarrer en désignant la ligne fautive. Les **erreurs d'exécution**, appelées **exceptions**, surviennent en pleine course : la syntaxe est correcte, mais une instruction devient impossible au moment où elle s'exécute, comme diviser par zéro ou ouvrir un fichier absent. Les **erreurs logiques**, enfin, sont les plus sournoises : le programme démarre, tourne, se termine sans le moindre message, mais calcule de mauvais résultats.

Code 20.1 : Trois familles, trois comportements

```python
# 1) Erreur de SYNTAXE : refusee avant toute execution
# if x > 5            # SyntaxError : il manque les deux-points
# 2) Erreur d'EXECUTION (exception) : la syntaxe est correcte
print(10 / 0)          # ZeroDivisionError : division par zero
# 3) Erreur LOGIQUE : le programme tourne mais se trompe
moyenne = 12 + 16 + 9 / 3    # priorite des operateurs oubliee !
print(moyenne)               # 31.0 au lieu de 12.33
```

Observez la troisième famille avec méfiance : la ligne moyenne = 12 + 16 + 9 / 3 ne provoque aucun plantage, Python applique simplement la priorité des opérateurs (la division d'abord), et le résultat est faux en silence. C'est précisément ce silence qui rend les erreurs logiques dangereuses : seul un contrôle du résultat (une valeur attendue, un test, un assert) les révèle. Les sections 21.2 et 21.3 vous donneront les outils pour cela ; le reste du chapitre 20 traite des deux premières familles, et surtout des exceptions, car une exception non maîtrisée arrête brutalement le programme sous les yeux de l'utilisateur.

20.2 Lire un traceback : le rapport d'accident

Quand une exception n'est pas interceptée, Python imprime un rapport appelé **traceback** (trace de la pile d'appels) puis arrête le programme. Le réflexe fondamental à ancrer dès maintenant : un traceback se lit **de bas en haut**. La dernière ligne annonce le type de l'exception et son message ; les lignes du dessus remontent la chaîne des appels qui y a mené, avec le nom du fichier, le numéro de ligne et la réimpression de l'instruction fautive. L'en-tête Traceback (most recent call last) signifie que l'appel le plus récent figure en bas : c'est donc le bas du rapport qui décrit l'instant exact de l'accident.

Code 20.2 : Un script fautif et son rapport d'accident

```python
# Contenu du fichier solde.py
def retirer(solde, montant):
    return solde - montant
retirer("100", 30)     # oups : une chaine au lieu d'un nombre
```

Code 20.3 : Le traceback produit par l'exécution de solde.py

```
Traceback (most recent call last):
  File "solde.py", line 5, in <module>
    retirer("100", 30)
  File "solde.py", line 2, in retirer
    return solde - montant
TypeError: unsupported operand type(s) for -: 'str' and 'int'
```

Lisez ce rapport comme un enquêteur. La dernière ligne condense l'essentiel : le type (TypeError, une opération entre types incompatibles) et le message (la soustraction ne sait pas retrancher un int d'une str). Au-dessus, Python retrace le trajet : ligne 5 du fichier solde.py, le module principal appelle retirer("100", 30) ; ligne 2, dans la fonction retirer, l'instruction return solde - montant fait échouer la soustraction. Dans les grands programmes, cette remontée traverse parfois des dizaines de fichiers : commencez toujours par la dernière ligne, puis remontez jusqu'au premier fichier qui vous appartient, car l'erreur naît le plus souvent dans votre propre code plutôt que dans les bibliothèques.

20.3 Le bestiaire des exceptions courantes

Reconnaître le type d'une exception au premier coup d'œil double la vitesse de votre débogage : chaque nom raconte une famille d'incident bien précise. Le tableau ci-dessous réunit les dix exceptions que vous rencontrerez le plus souvent dans les mois qui viennent ; chacune sera illustrée par un exemple dans les chapitres et exercices de ce Tome. Le meilleur entraînement consiste à provoquer volontairement chacune d'elles dans la console interactive pour lire son message.

Exception	Quand elle survient	Exemple déclencheur
NameError	Nom jamais défini (souvent une faute de frappe)	print(nom) avant toute affectation
TypeError	Opération entre types incompatibles	"5" + 2
ValueError	Type correct mais valeur inacceptable	int("douze")
IndexError	Indice hors des limites de la séquence	[10, 20, 30][5]
KeyError	Clé absente du dictionnaire	d["inconnue"]
AttributeError	Attribut ou méthode inexistante	"texte".push()
ZeroDivisionError	Division ou modulo par zéro	x / 0
FileNotFoundError	Fichier introuvable au chemin indiqué	open("absent.txt")
IndentationError	Indentation incohérente dans un bloc	espaces et tabulations mélangés
ModuleNotFoundError	Module inexistant ou mal orthographié	import matplotib (un t manque)

Tableau 20.1 : Les exceptions à connaître par cœur

20.4 try / except : attraper l'exception

Laisser une exception tuer le programme est acceptable pendant vos essais, mais pas devant un utilisateur. Le couple **try / except** installe un filet de sécurité : Python tente d'exécuter le bloc try ; si une exception du type attendu survient, il n'affiche pas de traceback brutal mais exécute à la place le bloc except, puis continue normalement après le filet. La saisie au clavier est l'exemple canonique : l'utilisateur tapera un jour ou l'autre « dix-huit » là où on attend un nombre, et le programme doit le reprendre avec patience.

Code 20.4 : Une saisie d'âge qui pardonne les erreurs

```python
while True:
    try:
        texte = input("Quel age avez-vous ? ")
        age = int(texte)          # risquee : peut lever ValueError
        break                     # conversion reussie : on sort
    except ValueError:
        print(f"'{texte}' n'est pas un nombre, reessayez.")
print(f"Merci, vous avez {age} ans.")
```

Déroulons la mécanique. Si l'utilisateur tape 27, la conversion int(texte) réussit, break fait sortir de la boucle : le bloc except est sauté, personne ne l'a remarqué. Si l'utilisateur tape dix-huit, int(texte) lève une ValueError ; Python abandonne immédiatement le reste du bloc try (le break ne s'exécute pas) et saute dans le bloc except, qui invite à réessayer : la boucle repart. Un même bloc try peut protéger plusieurs instructions risquées, et un même try peut être suivi de plusieurs blocs except, un par type d'exception, du plus précis au plus général.

Code 20.5 : Plusieurs exceptions, plusieurs plans B

```python
try:
    fichier = open("donnees.txt", encoding="utf-8")
    nombre = int(fichier.readline())
except FileNotFoundError:
    print("Fichier absent : j'utilise la valeur par defaut.")
    nombre = 0
except ValueError as erreur:
    print(f"Premiere ligne corrompue ({erreur}) : valeur 0.")
    nombre = 0
print(f"Valeur de depart : {nombre}")
```

La clause except ValueError as erreur mérite une pause : le mot-clé as récupère l'objet exception dans une variable, et son message (str(erreur)) décrit précisément la cause. C'est la matière première des journaux d'erreurs professionnels. Retenez enfin deux interdits de métier. Premièrement, bannissez except: pass, qui ensevelit l'erreur en silence et laisse le programme continuer avec des données fausses : une erreur doit au minimum être signalée. Deuxièmement, n'attrapez Exception à la sauvette que si vous savez quoi en faire : un filet trop large masque aussi les vrais bugs, comme une faute de frappe dans un nom de variable qui devient mystérieusement une « donnée manquante ».

ATTENTION

Deux abus à bannir absolument. except: pass ensevelit l'erreur en silence : le programme continue avec des données fausses et personne ne saura jamais pourquoi. À l'inverse, attraper Exception en bloc masque aussi les vrais bugs : une faute de frappe dans un nom devient une « donnée manquante » indétectable. Attrapez toujours le type le plus précis possible, et laissez remonter ce que vous ne savez pas traiter.

20.5 else et finally : la panoplie complète

Le couple try / except se complète de deux clauses optionnelles qui précisent le déroulé. La clause **else** s'exécute uniquement si le bloc try s'est déroulé sans aucune exception : elle sert à isoler le code qui ne doit tourner qu'en cas de succès, et à garder le bloc try minimal, concentré sur les seules instructions risquées. La clause **finally**, elle, s'exécute TOUJOURS, qu'il y ait eu exception ou non : c'est l'emplacement des travaux de nettoyage (fermer une connexion, libérer une ressource, afficher un bilan). L'ordre de lecture est donc try (on tente), except (on répare), else (on exploite le succès), finally (on range l'atelier).

Code 20.6 : Les quatre temps d'un try complet

```python
try:
    valeur = int("42")            # tentative risquee
except ValueError:
    print("Conversion impossible")     # en cas d'echec
else:
    print(f"Conversion reussie : {valeur + 1}")   # en cas de succes
finally:
    print("Ce bilan s'affiche dans tous les cas.")
# Affichage produit :
# Conversion reussie : 43
# Ce bilan s'affiche dans tous les cas.
```

Remplacez "42" par "quarante-deux" et rejouez : le message d'échec remplace le message de succès, mais le bilan du finally s'affiche dans les deux scénarios. Cette constance est sa raison d'être : même si le bloc except lui-même levait une nouvelle exception, le finally passerait avant l'arrêt. Vous connaissez déjà un nettoyage automatique de ce genre : la forme with open(...) du chapitre 14 ferme le fichier même en cas d'erreur, exactement comme un finally le ferait. Pour les fichiers, préférez donc with ; pour vos propres ressources et bilans, finally reste l'outil.

20.6 raise : lever ses propres exceptions

Python lève des exceptions, mais vous aussi : l'instruction **raise** déclenche une exception volontairement, avec un message de votre choix. C'est le gardien des règles métier : plutôt que d'accepter en silence une donnée absurde qui contaminera tout le calcul en aval, la fonction refuse immédiatement, nommément, avec la précision d'un douanier. Vous l'avez déjà vu à l'œuvre sans le savoir : le setter du chapitre 16 levait une ValueError pour refuser un solde négatif, et la classe Argent de l'exercice 21 levait une TypeError pour refuser une addition entre incompatible. Ce chapitre vous en donne enfin la théorie complète.

Code 20.7 : Une fonction qui refuse les données absurdes

```
def fixer_note(note):
    if not isinstance(note, (int, float)):
        raise TypeError("Une note doit etre un nombre.")
    if not 0 <= note <= 20:
        raise ValueError(f"Note {note} hors bornes [0 ; 20].")
    print(f"Note {note} enregistree.")
fixer_note(15)          # Note 15 enregistree.
# fixer_note(25)        # ValueError : Note 25 hors bornes [0 ; 20].
# fixer_note("dix")     # TypeError : Une note doit etre un nombre.
```

Cette technique porte un nom célèbre dans la culture Python : la préférence EAFP, « Easier to Ask Forgiveness than Permission » (plus facile de demander pardon que la permission). Plutôt que d'empiler des vérifications préalables avant chaque opération (style LBYL, « Look Before You Leap », regarder avant de sauter), le code Python idiomatique tente l'opération et rattrape l'exception si elle survient. Les deux styles ont leur place : LBYL quand le test est bon marché et lisible, EAFP quand la vérification préalable serait complexe ou soumise à des courses entre threads. Dans tous les cas, raise et try / except forment le couple des deux bouts du fil téléphonique : l'un signale, l'autre décide de la réponse.

20.7 Définir ses propres exceptions

La boucle est bouclée avec la POO : une exception est une **classe**, et créer la sienne revient à définir une fille d'Exception, héritage du chapitre 17 à l'appui, avec un nom qui se termine par Error par convention. Pourquoi s'embêter alors que ValueError existe déjà ? Parce qu'un type nommé parle : attraper SoldeInsuffisantError, c'est comprendre le problème sans lire le message, et le code appelant peut distinguer ce cas métier précis de toutes les autres ValueError possibles. C'est aussi une documentation vivante : la liste de vos classes d'exceptions décrit tout ce qui peut mal tourner dans votre application.

Code 20.8 : Une exception métier pour la banque

```python
class SoldeInsuffisantError(Exception):
    """Signale un retrait impossible sur un compte bancaire."""
def retirer(solde, montant):
    if montant > solde:
        raise SoldeInsuffisantError(
            f"Retrait de {montant} refuse : solde {solde}.")
    return solde - montant
try:
    reste = retirer(5000, 8000)
except SoldeInsuffisantError as erreur:
    print(f"Operation annulee : {erreur}")
    reste = 5000
print(f"Solde conserve : {reste} F")
```

La définition tient en deux lignes : class SoldeInsuffisantError(Exception) et une docstring qui dit quand la lever. Python lui donne gratuitement tout le comportement d'une exception : elle se lève avec raise, s'attrape avec except, transporte son message et se présente dans les tracebacks. Notez le partage des rôles : retirer() ne sait PAS ce qu'il faut faire en cas d'échec (avertir ? réessayer ? arrêter ?) et refuse d'en décider ; le code appelant, lui, connaît le contexte et réagit. Ce découplage est la vraie richesse du mécanisme : chaque couche du programme signale ce qu'elle observe et gère ce qu'elle maîtrise.

À RETENIR

Les erreurs se répartissent en trois familles : syntaxe (refus avant exécution), exceptions (plantage en course) et logique (résultat faux sans message). Le traceback se lit de bas en haut : dernière ligne = type et cause. try / except attrape une exception précise, else récompense le succès, finally nettoie toujours, as donne accès au message. raise déclenche une exception volontairement pour protéger les règles métier, et une exception personnalisée est une simple classe fille d'Exception.

CHAPITRE 21

Déboguer et fiabiliser son code

Écrire du code qui fonctionne est une compétence ; trouver pourquoi du code ne fonctionne PAS en est une autre, tout aussi indispensable. Tous les développeurs professionnels passent une part considérable de leur temps à déboguer, et ce qui les distingue des débutants n'est pas un don, mais une méthode et des outils. Ce chapitre vous transmet les deux : une démarche en quatre temps qui évite les corrections au hasard, puis les trois outils du programmeur Python, du plus humble (print bien utilisé) au plus puissant (le débogueur interactif), jusqu'aux réflexes d'écriture qui produisent du code où les bugs se sentent mal.

21.1 La méthode en quatre temps

Face à un bug, l'instinct pousse à modifier du code au hasard en espérant que ça marche : c'est le meilleur moyen d'aggraver le mal. La démarche professionnelle s'organise en quatre temps disciplinés, à suivre dans l'ordre.

- **Reproduire.** Trouvez une suite de gestes fiable qui déclenche le bug à coup sûr (des entrées précises, une commande exacte). Un bug qui refuse de se reproduire est un bug que vous ne pouvez pas vérifier avoir corrigé : la reproduction est le point de départ ET le point d'arrivée.
- **Isoler.** Réduisez le terrain : quelles fonctions suspectes, quelles lignes ? Commentez, élaguez, testez les petites pièces une à une dans la console, jusqu'à réduire le crime à quelques lignes.
- **Corriger la cause, pas le symptôme.** Un correctif qui « semble marcher » sans explication cache souvent un second bug. Si vous ne pouvez pas expliquer POURQUOI votre correction fonctionne, vous n'avez pas fini de chercher.
- **Vérifier.** Rejouez le scénario de reproduction : le bug a disparu ? Assurez-vous ensuite que la correction n'a rien cassé ailleurs (le cas limite d'avant fonctionne-t-il toujours ?).

Notez la symétrie du cycle : la reproduction qui a ouvert l'enquête sert aussi à la clore. Beaucoup de développeurs tiennent un cahier de débogage (même un simple fichier texte) où chaque incident reçoit une ligne : symptôme, cause trouvée, correctif appliqué. Relire ce cahier quelques semaines plus tard révèle vos schémas d'erreurs personnels, et prévenir vaut toujours mieux que guérir.

21.2 Le débogage par print, mais bien fait

Le print est l'outil de débogage le plus humble et le plus utilisé du monde ; mal employé, il noie l'écran de bruit, bien employé, il documente le déroulement du programme comme une caméra embarquée. Trois règles font toute la différence : étiqueter chaque print (un simple [1], [2], [3] pour retrouver sa trace), afficher non seulement les valeurs mais leurs types quand le doute plane, et observer l'état AVANT et APRÈS les opérations suspectes pour repérer l'instant où tout dérape.

Code 21.1 : Instrumenter une fonction étape par étape

```python
def moyenne_centree(notes):
    """Moyenne sans la plus petite et la plus grande note."""
    core = sorted(notes)[1:-1]
    return sum(core) / len(core)
```

```python
# La meme fonction, version instrumentee pour l'enquete
def moyenne_centree_debug(notes):
    print(f"[1] entree  : {notes}")
    tri = sorted(notes)
    print(f"[2] triee   : {tri}")
    core = tri[1:-1]
    print(f"[3] coeur   : {core}")
    resultat = sum(core) / len(core)
    print(f"[4] moyenne : {resultat:.2f}")
    return resultat
print(moyenne_centree_debug([12, 8, 18, 10]))
```

L'exécution révèle la mécanique interne : entrée [12, 8, 18, 10], triée [8, 10, 12, 18], cœur [10, 12] (les extrêmes 8 et 18 sont exclus), moyenne 11.00. L'instrumentation a aussi mis le doigt sur une faille que le code simple cachait : avec une liste de deux notes seulement, le cœur tri[1:-1] devient vide et la division lève une ZeroDivisionError. C'est le schéma type d'une enquête réussie : on cherchait un résultat faux, on découvre au passage un cas limite. Une fois le bug réglé, retirez ces prints (ou remplacez la fonction debug par la version propre) : le débogage est un échafaudage, pas une façade.

ASTUCE

Depuis Python 3.8, l'astuce print(f"{total=}") affiche d'un coup le nom ET la valeur : total=6300. Idéal pour instrumenter vite sans retaper le nom de chaque variable. Ajoutez un marqueur visible (### ou [DBG]) en préfixe pour retrouver tous vos prints d'enquête d'un coup d'œil, et n'oubliez pas de les retirer une fois le bug corrigé.

21.3 assert : les garde-fous du code

L'instruction **assert condition, message** proclame une hypothèse : « je suis certain que cette condition est vraie à cet endroit ». Si l'hypothèse tient, le programme continue sans un mot ; si elle tombe, Python lève immédiatement une AssertionError accompagnée du message. Les asserts servent à deux usages complémentaires : documenter et faire respecter les préconditions d'une fonction (l'appelant doit fournir une liste non vide, des notes entre 0 et 20), et constituer un mini-banc de tests que l'on rejoue après chaque modification.

Code 21.2 : Des hypothèses explicites et un banc d'essai

```python
def moyenne(notes):
    assert len(notes) > 0, "moyenne() exige une liste non vide"
    assert all(0 <= n <= 20 for n in notes), "note hors bornes"
    return sum(notes) / len(notes)
print(moyenne([12, 16, 9]))     # 12.33
# moyenne([])                   # AssertionError : liste non vide
# moyenne([12, 25])             # AssertionError : note hors bornes
# Le mini-banc d'essai : rejouable apres chaque modification
assert moyenne([10]) == 10
assert abs(moyenne([12, 16, 9]) - 12.33) < 0.01
```

```
print("Tous les tests passent.")
```

Le deuxième assert du banc d'essai mérite une explication : comparer des flottants avec == est risqué (chapitre 6 : 0.1 + 0.2 ne vaut pas exactement 0.3), donc on vérifie que la moyenne s'écarte de 12.33 de moins d'un centième. Retenez aussi la frontière de responsabilité : assert vérifie le développeur, pas l'utilisateur. Avec l'option d'optimisation python -O, Python ignore tous les asserts et le programme file à pleine vitesse : n'y confiez donc JAMAIS une validation dont dépend l'intégrité des données (une saisie utilisateur, un montant bancaire). Pour ces cas-là, le chapitre 20 vous a donné l'outil adéquat : raise ValueError.

21.4 Le débogueur intégré : breakpoint()

Quand le programme est long, rapide ou boucle des milliers de fois, les prints deviennent une pluie illisible. Il faut alors pouvoir ARRÊTER le vol à une altitude choisie et inspecter le cockpit : c'est le rôle du débogueur. La fonction **breakpoint()**, placée n'importe où dans le code, gèle le programme à cette ligne et ouvre une session interactive pdb dans le terminal. Vous y exécutez des commandes d'une lettre : n passe à la ligne suivante, p affiche la valeur de n'importe quelle expression, c reprend le vol, q abandonne. Le programme est vivant mais suspendu : toutes les variables locales sont accessibles, dans l'état exact de l'incident.

Code 21.3 : Poser un point d'arrêt avec breakpoint()

```python
def total_commandes(prix, remise):
    breakpoint()                 # le programme s'arrete ici
    sous_total = sum(prix)
    total = sous_total * (1 - remise / 100)
    return total
print(total_commandes([1200, 800], 10))
```

Code 21.4 : Une session pdb pas à pas (les commandes en gras)

```
> boutique.py(2)total_commandes()
-> sous_total = sum(prix)
(Pdb) p prix
[1200, 800]
(Pdb) n
(Pdb) p sous_total
2000
(Pdb) c
1440.0
```

Commande	Effet
p expression	Affiche la valeur d'une expression dans l'état actuel
n (next)	Exécute la ligne suivante sans entrer dans les fonctions
s (step)	Exécute la ligne suivante en entrant dans la fonction appelée
c (continue)	Reprend l'exécution jusqu'au prochain point d'arrêt
l (list)	Affiche le code autour de la position courante
q (quit)	Abandonne la session et le programme

Tableau 21.1 : Les commandes pdb de première urgence

Relisez la session du code 21.4 : arrivé à la ligne du breakpoint, l'enquêteur demande p prix (le paramètre vaut [1200, 800]), avance d'une ligne avec n, constate que sous_total vaut 2000, puis débloque tout avec c : le programme termine et imprime 1440.0 (2000 moins 10 % de remise). Quelques secondes de session ont confirmé ce que trois prints auraient dévoilé plus bruyamment. Dans vos environnements graphiques (VS Code, Thonny, PyCharm), les mêmes concepts existent en version visuelle : clic dans la marge pour poser un point d'arrêt, panneau Variables pour inspecter, boutons pour avancer pas à pas. Apprendre pdb, c'est apprendre le vocabulaire que tous ces outils partagent.

21.5 Les pièges classiques du débutant

Certaines erreurs reviennent chez presque tous les apprenants, avec les mêmes symptômes et les mêmes remèdes. Ce tableau de chasse condense les pièges que vous avez déjà (ou bientôt) rencontrés dans ce Tome ; gardez-le sous la main avant de déclarer un programme « hanté ». La plupart se débusquent en quelques secondes une fois qu'on sait quoi regarder.

Symptôme	Cause probable	Remède
SyntaxError sur une ligne if	= (affectation) écrit là où == (comparaison) était voulu	Relire la ligne : if x == 5 :
IndexError dans une boucle	range(len(x) + 1) ou suppression d'éléments en itérant	for element in liste, ou itérer sur une copie
La liste a changé « toute seule »	liste2 = liste1 copie la référence, pas la liste	liste2 = liste1[:] ou list(liste1)
0.1 + 0.2 ne vaut pas 0.3	Précision limitée des flottants	round(x, 2), ou comparaison à un epsilon
Boucle d'un tour de trop ou de moins	La borne de fin de range est exclusive	print(list(range(a, b))) pour visualiser
Résultat faux sans aucun plantage	Erreur logique : ordre des opérations, nom trompeur	Instrumentation print (21.2) puis assert (21.3)

Tableau 21.2 : Les six pièges les plus fréquents

Le dernier piège, alias de liste, mérite qu'on le rejoue une fois de plus car il frappe même des étudiants avancés. Écrire copie = prix_articles ne duplique AUCUNE donnée : les deux noms désignent le même objet, et pop() sur « la copie » ampute le panier original. Le remède liste1[:] fabrique une vraie duplication. Rappel du chapitre 9 : cette subtilité ne concerne que les objets modifiables (listes, dictionnaires, ensembles, objets de vos classes) ; les nombres, chaînes et tuples, immuables, sont à l'abri.

21.6 Écrire du code facile à déboguer

Le meilleur débogage est celui qu'on n'a pas à faire. La prévention tient à des réflexes déjà enseignés dans ce Tome, rassemblés ici comme une check-list de fin de rédaction : découpez en petites fonctions à responsabilité unique (chapitre 12), car une fonction de six lignes se teste en une ligne ; donnez aux variables et fonctions des noms qui racontent leur rôle, car le nom trompeur est une cause d'erreur logique silencieuse ; documentez chaque fonction avec une docstring honnête (ce qu'elle reçoit, ce qu'elle renvoie) ; bannissez les variables globales qui laissent des données changer sans témoin ; et validez vos hypothèses avec assert ou raise plutôt que de faire confiance au hasard.

Code 21.5 : Une fonction solide et son banc d'essai

```python
def appliquer_remise(prix, pourcentage):
    """Prix final apres remise (pourcentage entre 0 et 100)."""
    if not 0 <= pourcentage <= 100:
        raise ValueError("Pourcentage hors bornes [0 ; 100].")
    return prix * (1 - pourcentage / 100)
# Le banc d'essai rejouable : trois cas normaux, un cas limite
assert appliquer_remise(1000, 10) == 900
assert appliquer_remise(1000, 0) == 1000
assert appliquer_remise(1000, 100) == 0
try:
    appliquer_remise(1000, 150)
    assert False, "la remise 150 aurait du etre refusee"
except ValueError:
    print("Cas limite bien gere : OK")
```

Cette fonction concentre tout le chapitre : une validation métier qui refuse l'absurde avec raise (chapitre 20), une docstring qui contracte les responsabilités, et un banc d'essai qui couvre les cas normaux ET le cas limite, rejouable après chaque modification du code. Ce banc, c'est l'embryon des tests unitaires que le Tome 2 développera avec l'outil pytest : la philosophie y est déjà entière, « un code sans test est un code qui attend son bug ». Combinez ces réflexes avec la méthode en quatre temps et les trois outils d'enquête, et vous n'affronterez plus jamais un message d'erreur les mains vides.

À RETENIR

Déboguer, c'est reproduire, isoler, corriger la cause, puis vérifier : jamais de correctif au hasard. Le print bien fait s'étiquette, montre les types et observe l'état avant/après. assert proclame une hypothèse et sert de mini-test rejouable (mais vérifie le développeur, pas l'utilisateur : pour la validation réelle, raise). breakpoint() suspend le programme dans pdb : p pour inspecter, n pour avancer, c pour reprendre. Et la meilleure prévention reste du code propre : petites fonctions, noms parlants, docstrings, zéro variable globale.

Exercices corrigés de la Partie V

Dernière série : cinq exercices pour transformer les erreurs en alliées. Deux basiques pour prendre en main try / except et le bestiaire des exceptions, deux intermédiaires pour blinder vos classes et vos conversions, puis un défi d'enquête où vous traquerez cinq bugs plantés dans un programme réel. Comme toujours, cherchez chaque solution au moins dix minutes avant de lire la correction.

Ex. 23 La saisie blindée

BASIQUE

Écrivez une fonction demander_nombre() qui réclame à l'utilisateur un entier entre 1 et 100 et ne lâche pas l'affaire avant d'en obtenir un valide : un texte non convertible déclenche un message d'erreur courtois et une nouvelle demande, un entier hors bornes aussi. Utilisez la boucle while True avec try / except, et réservez le contrôle des bornes à la clause else. Testez en simulant les saisies « douze », puis 150, puis 42.

Solution

Stratégie : le try / except protège la seule instruction risquée (la conversion int) ; la clause else ne s'exécute que si la conversion a réussi, c'est donc elle qui juge les bornes. Le patron while True : ... return est la façon idiomatique de boucler jusqu'à une donnée acceptable.

Solution de l'exercice 23

```python
def demander_nombre():
    """Redemande jusqu'a obtenir un entier entre 1 et 100."""
    while True:
        try:
            texte = input("Entrez un nombre entre 1 et 100 : ")
            nombre = int(texte)
        except ValueError:
            print(f"'{texte}' n'est pas un entier, reessayez.")
        else:
            if 1 <= nombre <= 100:
                return nombre
            print(f"{nombre} est hors bornes [1 ; 100].")
n = demander_nombre()
print(f"Merci : vous avez choisi {n}.")
```

Suivez les trois saisies de test. « douze » : int lève ValueError, le message d'accueil s'affiche, la boucle repart. 150 : la conversion réussit, else juge les bornes et refuse, la boucle repart. 42 : conversion réussie, bornes respectées, return met fin à la boucle. La finesse de l'exercice est le partage des rôles : except gère le type, else gère la valeur. Si vous aviez tout mis dans le try, une ValueError sur les bornes serait impossible à distinguer d'une saisie illisible ; la clause else garde chaque responsabilité à sa place.

Ex. 24 Le détective des exceptions

BASIQUE

Sans exécuter les lignes, annoncez pour chacune le type d'exception levée (ou son comportement normal) et justifiez en une phrase : 1) int("3.14") ; 2) [10, 20, 30][5] ; 3) print(bonjour) où bonjour n'a jamais été définie ; 4) "salut" + 3 ; 5) d = {"a": 1} puis d["b"]. Ensuite, vérifiez vos pronostics dans la console : la lecture du message de Python confirme-t-elle votre diagnostic ?

Solution

Stratégie : classer chaque ligne par le type de la donnée et par l'opération demandée, puis recouper avec le tableau 20.1. La vérification en console fait partie de l'exercice : le réflexe « lire le message » est aussi important que le pronostic.

Ligne	Verdict	Explication
int("3.14")	ValueError	Le type est bon (une chaîne) mais la valeur n'est pas un entier littéral ; float("3.14") aurait réussi.
[10, 20, 30][5]	IndexError	La liste compte trois éléments, indices 0 à 2 ; l'indice 5 dépasse les limites.
print(bonjour)	NameError	Le nom bonjour n'existe dans aucune portée : Python ne peut pas le résoudre.
"salut" + 3	TypeError	L'addition entre une chaîne et un int n'a pas de sens ; "salut" + str(3) aurait réussi.
d["b"] sur {"a": 1}	KeyError	La clé "b" est absente ; d.get("b") aurait renvoyé None sans bruit (chapitre 11).

Tableau des pronostics de l'exercice 24

Si vos cinq diagnostics sont justes, le tableau 20.1 est acquis. Sinon, notez dans votre cahier de débogage les confusions persistantes : ValueError et TypeError se distinguent par le niveau du problème (la VALEUR est mauvaise, ou le TYPE est mauvais), tandis que IndexError et KeyError se distinguent par la structure touchée (une séquence numérotée, ou un dictionnaire à clés). Ces quatre-là reviendront toute votre carrière : mieux vaut les reconnaître en un clin d'œil dès maintenant.

Ex. 25 Le compte bancaire blindé

INTERMÉDIAIRE

Reprenez la classe CompteBancaire du chapitre 15 et rendez-la intransigeante : définissez deux exceptions personnalisées, MontantInvalideError (montant négatif, nul ou non numérique) et SoldeInsuffisantError (retrait dépassant le solde). La méthode deposer et retirer doivent vérifier le montant et lever la bonne exception ; ajoutez une méthode virer_vers(autre_compte, montant) qui effectue le virement en attrapant SoldeInsuffisantError pour afficher « Virement annulé » sans rien modifier. Scénario : Awa (50 000 F) vire 20 000 à Roland (10 000 F) ; Roland tente de retirer 50 000 ; on tente de déposer -100 chez Awa ; enfin, on affiche les deux soldes (30 000 et 30 000).

Solution

Stratégie : une méthode privée _verifier() centralise la validation des montants (chapitre 12 : une seule source de vérité) ; les deux exceptions personnalisées sont des filles d'Exception définies en deux lignes chacune. Le virement attrape précisément SoldeInsuffisantError : si le retrait échoue, le dépôt n'a jamais lieu, le virement reste atomique comme au chapitre 19.

Solution de l'exercice 25

```python
class MontantInvalideError(Exception):
    """Le montant n'est pas un nombre strictement positif."""
class SoldeInsuffisantError(Exception):
    """Le retrait depasse le solde disponible."""
class CompteBancaire:
    def __init__(self, titulaire, solde=0):
        self.titulaire = titulaire
        self.solde = solde
    def _verifier(self, montant):
        if not isinstance(montant, (int, float)) or montant <= 0:
            raise MontantInvalideError(f"Montant refuse : {montant}")
    def deposer(self, montant):
        self._verifier(montant)
        self.solde += montant
    def retirer(self, montant):
        self._verifier(montant)
        if montant > self.solde:
            raise SoldeInsuffisantError(
                f"{self.titulaire} : demande {montant}, solde {self.solde}")
        self.solde -= montant
    def virer_vers(self, autre, montant):
        try:
            self.retirer(montant)
            autre.deposer(montant)
        except SoldeInsuffisantError as erreur:
            print(f"Virement annule : {erreur}")
# Scenario de test
awa = CompteBancaire("Awa", 50000)
```

```
roland = CompteBancaire("Roland", 10000)
awa.virer_vers(roland, 20000)          # reussit en silence
try:
    roland.retirer(50000)
except SoldeInsuffisantError as erreur:
    print(f"Refuse : {erreur}")
try:
    awa.deposer(-100)
except MontantInvalideError as erreur:
    print(f"Refuse : {erreur}")
print(awa.solde, roland.solde)         # 30000 30000
```

Les deux classes d'exceptions rendent chaque échec parlant et traitable séparément : le code appelant décide s'il veut pardonner un montant invalide (rectifier la saisie) ou un solde insuffisant (proposer un découvert), sans jamais confondre les deux situations. Remarquez la chaîne du virement : retirer lève, virer_vers attrape, l'argent ne peut ni apparaître ni disparaître. Et si demain une nouvelle règle métier arrive (plafond de dépôt, compte bloqué), il suffira d'une classe d'exception de plus : l'architecture grandit par ajout, jamais par modification, la marque d'un bon design.

Ex. 26 Le convertisseur à trois étages INTERMÉDIAIRE

Écrivez une fonction convertir(texte) qui tente de transformer une chaîne en int ; si l'échec est ValueError, tente float ; si l'échec recommence, levez une ValueError avec un message explicite. Puis faites défiler la liste ["42", "3.14", "abc", " 7 ", "1e3", ""] avec un try / except / else / finally par élément : except affiche l'échec, else affiche la conversion réussie, finally affiche le compteur de progression. Bilan attendu : 4 réussites, 2 échecs.

Solution

Stratégie : boucler sur les deux fonctions de conversion (int et float sont des objets comme les autres, chapitre 15 !) en rattrapant chaque ValueError pour passer à l'étage suivant ; épuisé, on lève avec un message qui cite la donnée fautive. Le finally sert ici de compteur de progression : il s'exécute dans tous les cas, succès comme échec.

Solution de l'exercice 26

```
def convertir(texte):
    """Chaine -> int, sinon float, sinon ValueError explicite."""
    for usine in (int, float):
        try:
            return usine(texte)
        except ValueError:
            continue
    raise ValueError(f"'{texte}' n'est ni un entier ni un reel.")
donnees = ["42", "3.14", "abc", "  7 ", "1e3", ""]
reussites, echecs = 0, 0
for texte in donnees:
    try:
        valeur = convertir(texte)
    except ValueError as erreur:
        print(f"ECHEC : {erreur}")
        echecs += 1
    else:
        print(f"OK    : '{texte}' -> {valeur}")
        reussites += 1
    finally:
        traites = reussites + echecs
        print(f"      (traite : {traites}/{len(donnees)})")
print(f"Bilan : {reussites} reussites, {echecs} echecs.")
```

Décompte vérifié : "42" réussit en int ; "3.14" échoue en int puis réussit en float ; "abc" épuise les deux étages et lève ; " 7 " réussit car int accepte les espaces d'entourage ; "1e3" échoue en int mais devient 1000.0 en float ; "" échoue partout. Le retour anticipé return usine(texte) dans le try est le trait élégant du patron : dès qu'un étage réussit, la fonction s'arrête là, et le continue ne concerne que les échecs. Ce convertisseur en cascade est un patron très répandu (lecture de fichiers de configuration, nettoyage de données avant import) : gardez-le dans votre boîte à outils.

Ex. 27 Opération débogage : cinq bugs dans le panier DÉFI

Le programme ci-dessous doit : calculer le total d'un panier avec une remise en pourcentage, offrir les frais de port si le total dépasse 10 000 F, afficher chaque article avec son prix, ne PAS modifier le panier reçu, refuser toute remise hors bornes [0 ; 100] avec une ValueError, et renvoyer le total après remise. Or il cumule CINQ bugs (dont un de robustesse : une exigence manquante). Déboguez-le avec la méthode du chapitre : reproduire, isoler, corriger, vérifier, en utilisant l'outil adapté à chaque famille d'erreur.

Le programme bogué à auditer

```
# panier.py : calcule le total d'un panier avec remise
def total_panier(prix_articles, remise_pourcent):
    total = sum(prix_articles)
    total = total * remise_pourcent / 100
    if total > 10000
        print("Frais de port offerts")
    for i in range(len(prix_articles) + 1):
        print(f"Article {i} : {prix_articles[i]} F")
    copie = prix_articles
    copie.pop()
    return total
panier = [5000, 3000, 2500]
total = total_panier(panier, 20)
print(f"A payer : {total} F")
print(f"Le panier contient {len(panier)} articles")
```

Solution

Stratégie : commencer par lancer le programme, car chaque famille d'erreur se débusque avec son outil : les bugs de syntaxe refusent de démarrer et désignent leur ligne, l'IndexError fournit un traceback à lire de bas en haut, et les erreurs logiques exigent l'instrumentation print du code 21.1. Une fois la liste dressée, on corrige tout, puis on rejoue le scénario complet pour vérifier.

Solution de l'exercice 27 : le programme corrigé

```python
def total_panier(prix_articles, remise_pourcent):
    """Total apres remise, sans modifier le panier recu."""
    if not 0 <= remise_pourcent <= 100:
        raise ValueError("Remise hors bornes [0 ; 100].")
    total = sum(prix_articles)
    total = total * (1 - remise_pourcent / 100)
    if total > 10000:
        print("Frais de port offerts")
    for i in range(len(prix_articles)):
        print(f"Article {i} : {prix_articles[i]} F")
    copie = prix_articles[:]      # vraie copie : le panier est protege
    print(f"Controle interne : {len(copie)} articles traites")
    return total
panier = [5000, 3000, 2500]
total = total_panier(panier, 20)
print(f"A payer : {total:.0f} F")
print(f"Le panier contient {len(panier)} articles")
# Affichage : trois articles listes, puis
# A payer : 8400 F  et  Le panier contient 3 articles
```

Bug	Famille	Outil qui le débusque	Correction
Remise appliquée à l'envers : on paye 20 % au lieu de 80 %	Logique	print instrumenté + valeur attendue (8400)	total * (1 - remise / 100)
Deux-points oubliés après le if	Syntaxe	Python refuse de démarrer et cite la ligne	if total > 10000 :
range(len(...) + 1) sort de la liste	Exécution	Traceback IndexError, lu de bas en haut	range(len(...)), ou for prix in liste
copie = prix_articles puis pop() ampute le panier	Logique (alias)	Le print final affiche 2 articles au lieu de 3	copie = prix_articles[:]
Aucune validation de la remise exigée	Robustesse	Relecture du cahier des charges (section 21.6)	raise ValueError si hors bornes

Tableau de l'enquête de l'exercice 27

Vérification finale : le programme corrigé liste les trois articles, n'annonce pas de frais de port (8400 ne dépasse pas 10 000), affiche « A payer : 8400 F » (10 500 moins 20 %) et « Le panier contient 3 articles » : le panier original est intact, la remise 120 lèverait une ValueError. Le bilan de l'enquête est instructif : chaque famille d'erreur a été confondue par un outil différent, et aucun correctif n'a été posé au hasard. Vous venez de vivre un cycle complet de débogage professionnel ; rejouez-le sur vos propres programmes, et les messages rouges de Python ne seront plus jamais des ennemis, mais des collègues bavards qui vous disent exactement où porter secours.

Conclusion du Tome 1

Faites le compte de ce que vous savez désormais faire. Vous savez construire un algorithme en pseudo-code, avec des conditions, des boucles, des tableaux et des fonctions. Vous savez le traduire en Python et l'exécuter : variables et types, if / elif / else, for et while, listes, tuples, dictionnaires et ensembles, compréhensions de listes, fonctions avec closures et décorateurs, chaînes et fichiers. Et vous savez protéger vos programmes : lecture des tracebacks, try / except / else / finally, raise et exceptions personnalisées, débogage méthodique avec print, assert et breakpoint(). Vous savez aussi concevoir des objets : classes et instances, attributs de classe, encapsulation avec @property, héritage simple et multiple avec le MRO, méthodes spéciales et surcharge d'opérateurs, jusqu'à assembler un mini-système bancaire et un système de gestion d'école. C'est le socle complet d'un développeur Python débutant confirmé.

Trois conseils pour consolider cet acquis. Pratiquez chaque jour, même quinze minutes : la régularité construit plus que l'intensité. Relisez le code des corrections en vous demandant « pourquoi cette ligne ? » à chaque instruction : la lecture critique est le second meilleur professeur après l'erreur. Enfin, inventez vos propres projets (un gestionnaire de budget, un quiz de révision, un carnet de lectures) car c'est en résolvant des problèmes qui vous tiennent à cœur que la programmation devient vôtre.

Le **Tome 2** prolongera ce parcours vers la maturité : modules et packages pour organiser de grands projets, POO avancée (classes abstraites, itérateurs et générateurs, méthodes magiques étendues), tests unitaires avec pytest, interfaces graphiques, manipulation de bases de données et projets filés. Les fondations de ce Tome 1 y seront le prérequis assumé : gardez-le à portée de main.

Annexe : Aide-mémoire Python

Cette annexe condense les références essentielles du Tome 1. Gardez-la à portée de main pendant vos exercices : elle réunit les conversions, les opérateurs, les méthodes des structures de données, la syntaxe des structures de contrôle et le vocabulaire de la programmation orientée objet.

Besoin	Syntaxe	Exemple
Entier vers texte	str(n)	str(42) -> "42"
Texte vers entier	int(s)	int("42") -> 42
Texte vers nombre réel	float(s)	float("3.14") -> 3.14
Réel arrondi	round(x, ndigits)	round(3.14159, 2) -> 3.14
Type d'une valeur	type(x)	type(42) -> <class 'int'>
Saisie clavier (toujours str)	input(invite)	age = int(input("Age : "))

Tableau A.1 : Types et conversions

Famille	Opérateurs	Exemple
Arithmétiques	+ - * / // % **	7 // 2 -> 3 ; 7 % 2 -> 1 ; 2**10 -> 1024
Comparaison	== != < <= > >=	note >= 10
Logiques	and or not	0 <= n and n <= 20
Appartenance	in not in	"a" in "chaine"
Affectation composée	+= -= *= /=	somme += valeur

Tableau A.2 : Opérateurs

Structure	Méthodes principales	Rappel
Liste	append, insert, remove, pop, sort, reverse, index, count	Indexation depuis 0, tranches [a:b]
Dictionnaire	get, keys, values, items, pop, update	Clés uniques, accès par d[cle]
Ensemble	add, discard, union |, intersection &, difference -	Pas de doublon, pas d'ordre
Chaîne	upper, lower, strip, replace, split, join, find, startswith	Immuables : récupérer le résultat

Tableau A.3 : Méthodes des structures de données

Structure	Syntaxe Python	Équivalent pseudo-code
Condition	if cond: ... elif cond2: ... else: ...	SI ... SINON SI ... SINON
Boucle comptée	for i in range(debut, fin, pas):	POUR i DE debut A fin-1
Boucle conditionnelle	while cond:	TANT QUE cond
Sortie de boucle	break / continue	Sortie anticipée / tour suivant
Compréhension	[f(x) for x in seq if cond]	Nouvelle liste construite en une ligne
Gestion d'erreur	try / except / else / finally : ... raise ...	(protection à l'exécution)
Fonction	def nom(parametres): ... return valeur	FONCTION nom(...) RETOURNER ...
Classe	class Nom: def __init__(self, ...): ...	(spécifique POO)

Tableau A.4 : Syntaxe des structures de contrôle

Terme	Définition en une phrase
Classe	Le plan de construction : elle décrit les attributs et les méthodes communes à une famille d'objets.
Instance (objet)	Une réalisation concrète du plan, avec ses propres valeurs d'attributs.
Attribut d'instance	Une donnée propre à chaque objet, créée dans __init__ via self.
Attribut de classe	Une donnée partagée par toutes les instances, définie dans le corps de la classe.
Méthode	Une fonction définie dans la classe, dont le premier paramètre est self.
self	Le nom conventionnel de l'objet courant, transmis automatiquement par Python.
Encapsulation	Cacher les données brutes (_attribut) derrière @property et des méthodes de contrôle.
@property / .setter	Transforme une méthode en lecture d'attribut et valide chaque écriture.
@classmethod	Reçoit la classe (cls) en paramètre : fabrique des objets par des chemins alternatifs.
Héritage	class Fille(Mere) récupère attributs et méthodes de la mère ; super() l'initialise.
Polymorphisme	Le même appel de méthode produit des comportements différents selon l'objet.
Composition	Un objet en contient un autre (relation A UN) et lui délègue du travail.
Méthode dunder	Méthode spéciale (__str__, __eq__, __add__...) branchée sur la syntaxe native.
Closure	Fonction interne qui emporte les variables capturées de sa fonction englobante.
Décorateur	Fonction qui enrichit une autre fonction ; la syntaxe @nom la lui applique.
MRO	L'ordre de recherche des méthodes en cas d'héritage multiple, visible via __mro__.
Exception	Un objet signalant une erreur d'exécution ; attrapée avec try / except, déclenchée avec raise.
Traceback	Le rapport d'erreur de Python : il se lit de bas en haut pour localiser la faute.
Débogage	La démarche qui localise et corrige un bug : reproduire, isoler, corriger, vérifier.

Tableau A.5 : Le vocabulaire de la POO